SKALA

AI Campus 데이터분석 및 AIOps

4. 머신러닝 및 딥러닝 이해

Part C – DL Architecture

문제 해결의 연대기

2026.08

배기주 / SK AX

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX



문제 해결의 연대기

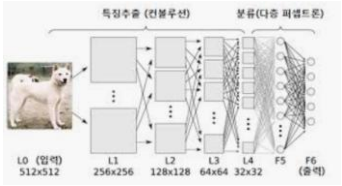
Part C – DL Architecture

1. 귀납적 편향의 설계
2. 표현 학습이라는 다른 길
3. CNN의 진화
4. Attention 등장
5. 규모와 비용의 시대

딥러닝 네트워크는 …만 구분 했었다

CNN
Convolutional Neural Network

이미지 분류

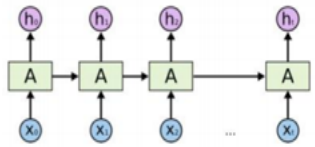


- Architecture
- Key word : 이미지 처리, Convolution Layer, Pooling Layer

- 이미지나 영상의 공간적 정보를 추출하는데 특화된 구조
- Convolution Layer 통해 입력 데이터의 패턴 학습

RNN
Recurrent Neural Network

자연어 처리 & 시계열 예측

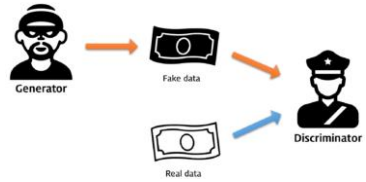


- Architecture
- Key word : 시퀀스 데이터, 시간 의존성

- 과거의 상태를 현재 상태에 반영하여 시간적인 패턴을 학습하는 네트워크 구조

GAN
Generative Adversarial Network

데이터 생성

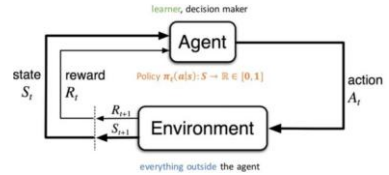


- Model
- Key Word : 생성자/판별자, 생성 모델, 경쟁적 학습

- 2개의 신경망이 경쟁적으로 학습
- 생성자는 실제에 가깝게 데이터 생성, 판별자는 그 데이터를 구분하는 모델

Reinforcement Learning

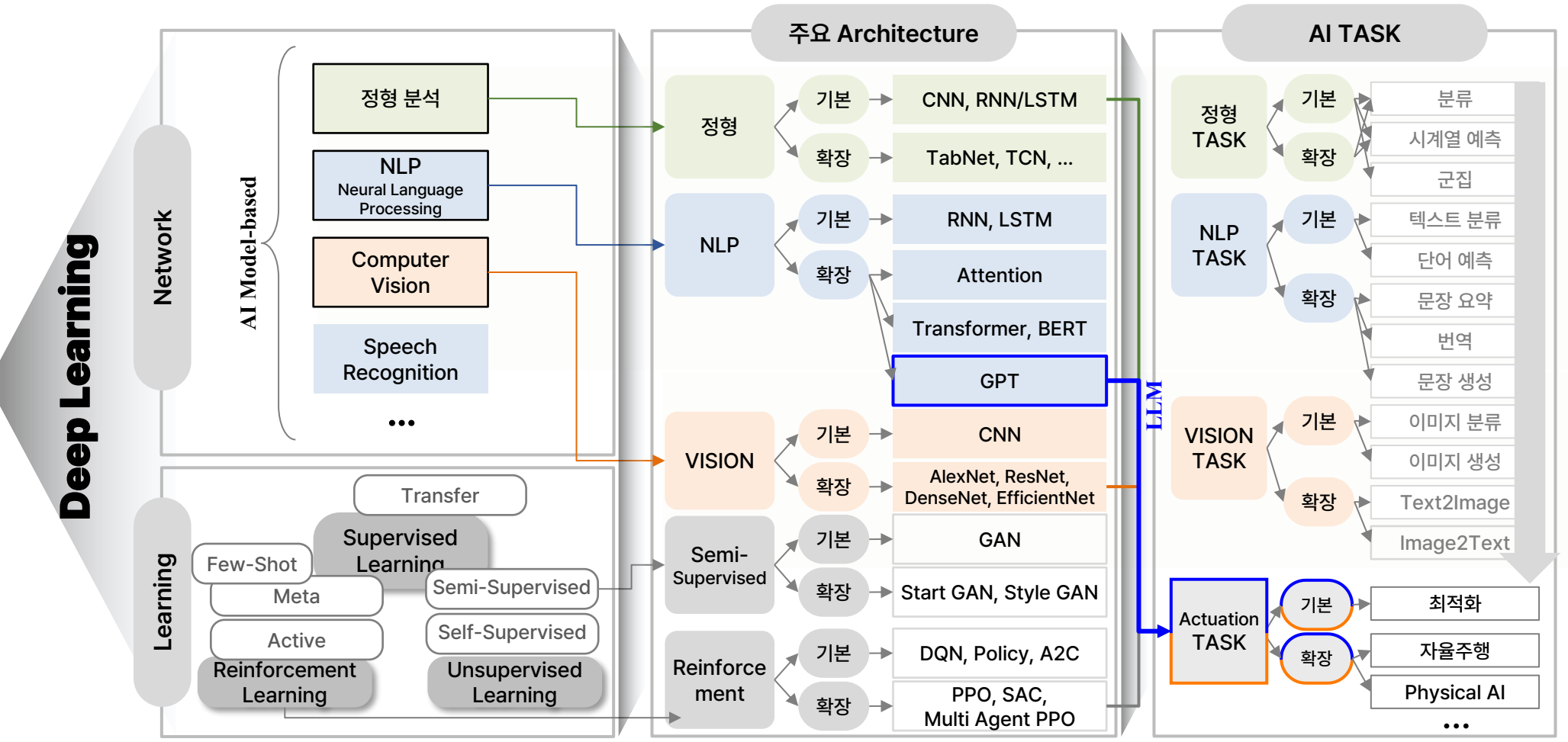
제어/최적화



- Algorithm
- Key word : Agent, Reward, Policy, Environment

- 에이전트가 환경과 상호작용하며 보상을 통해 최적의 행동을 학습하는 방법론
- 보상 피드백을 통해 의사결정 학습

Deep Learning, Expansion



DL Architecture

Expansion History

CNN-based Architecture

COMPUTER VISION		
YEAR	**Architecture**	**Contribution**
1990	**CNN**	• 이미지 처리에 강점을 가진 신경망 • 합성곱 필터로 이미지 특징 자동 추출하는 방법론 제시
1998	LeNet-5	• 초기 CNN 중 하나 • MNIST 위해 설계, 현대 CNN의 기초 다짐
2012	**AlexNet**	• GPU 기반 ReLU, Dropout 사용한 Large CNNs • ImageNet에서 혁신적인 성과 & 딥러닝 대중화
2014	VGGNet	• 작은 필터(3*3)로 아키텍처 단순화 • 객세인식 성능 향상
2014	GoogleNet (Inception)	• Inception 모듈 도입 • 계산 자원 효율화, 높은 정확도 유지
2015	**ResNet**	• Residual Learning으로 Deep Network(최대 152층) • 기울기 소실 문제 해결
2017	DenseNet	• 각 층을 모든 다른 층과 연결 • 파라미터 수를 줄이면서 정확도 향상
2017	YOLO You Only Look Once	• 실시간 객체 탐지 • 이미지 내 객체를 빠르고 효율적으로 탐지
2018	EfficientNet	• 네트워크 깊이, 폭, 해상도를 균형있게 조정 • 적은 파라미터로 성능 극대화
2021	**ViT** Vision Transformer	• Transformer를 이미지 분류에 적용한 첫 사례로 전통적은 CNN 성능 능가 (2~5%) • Vision Task에서도 효과적일 수 있음을 입증

TRANSFORMERs

NLP		
YEAR	**Architecture**	**Contribution**
1985	**RNN**	• 시퀀스 데이터를 처리할 수 있는 첫번째 신경망 • 순차적 데이터 처리에 탁월한 성능 발휘
1997	LSTM Long Short-Term Moemry	• RNN의 기울기 소실 문제 해결 • 언어/음성과 같은 시퀀스에서 뛰어난 성능 발휘
2014	Word2Vec	• 단어 임베딩 혁신 • 단어의 의미지 관계를 연속적인 벡터로 학습
2015	Seq2Seq	• 인코더-디코더 확장 • 기계번역 및 순차 데이터 작업에서 성능 향상
2017	**Transformer**	• Self-Attention 도입, RNN/LSTM 대체 • 병렬 처리와 함께 뛰어난 성능 구현
2018	**BERT**	• Transformer 디코더 구조 확장 (Bidirectional) • Pre-training, Transfer Learning
2019	GPT-2	• Transformer 디코더 구조 확장 (Unidirectional) • 문맥에 맞는 텍스트 생성 가능
2020	GPT-3	• 1750억개 파라미터 가진 대형 Transformer • 미세조정 없이 다양한 NLP Task 수행
2021	DALL-E	• 텍스트 설명으로 이미지 생성 • Cross-Modal, Multi-Modal
2021	CLIP	• 이미지와 텍스트 데이터 쌍을 학습 • 비전 작업에서 Zero Shot 학습 가능하게 함
2023	**GPT-4**	• GPT-3 후속버전 • 향상된 성능으로 깊이 있는 문맥 이해와 자연스러운 텍스트 생성 가능

Architecture

- 신경망의 구조적 설계 방식
- Layer 구성, 층 간의 연결 방식, 데이터 흐름 등을 정의

Algorithm

- 문제를 해결하기 위한 절차나 계산 방법
- 딥러닝에서는 학습 방법, 최적화 절차, 또는 데이터 처리 방법 등을 알고리즘이라고 함
  (Gradient Descent, Backpropagation, Optimizer 등)

Model

- 데이터를 입력 받아 예측이나 분류 등의 작업을 수행하는 학습된 모델
- 학습이 완료된 후 목적에 맞는 작업을 수행

AI Campus 데이터분석 및 AIOps > Part C – DL Architecture

1A. 귀납적 편향의 설계

데이터의 구조를 코드에 새기다

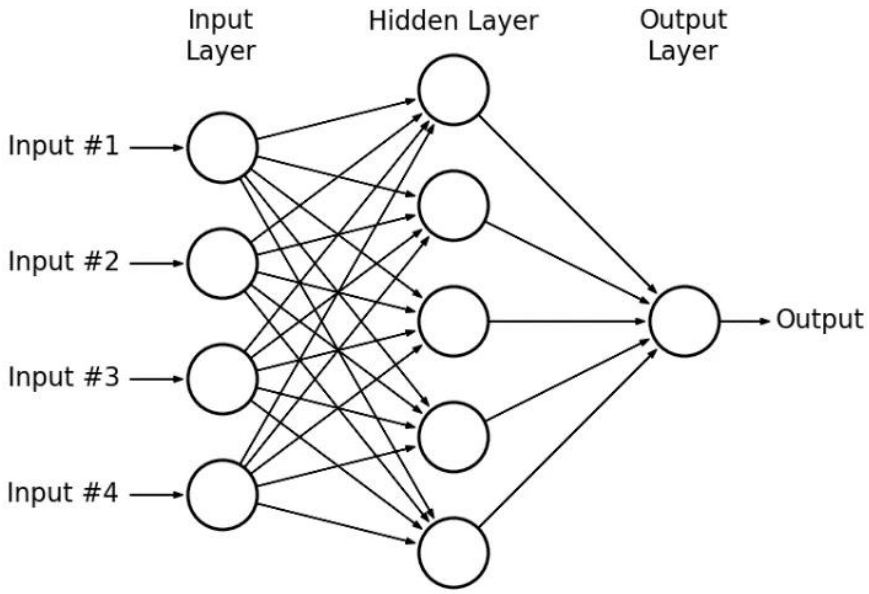
- MLP 한계
- CNN – 공간적 지역성과 가중치 공유

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

1. MLP

Multi-Layer Perceptron의 특징이자 한계 : 1차원 벡터



MLP 특징 :

- 뉴런들이 층으로 쌓여 있음
- 한 층의 모든 뉴런이 다음 층의 모든 뉴런과 빠짐없이 연결된 구조 (fully-connected)
- Input은 1차원 벡터 : 독립적인 속성일 때 유의미 (tabular)

그러나 이미지 데이터를 적용하면...

❑ 공간 구조 소실
- 1차원 벡터로 펼쳐지면 공간 구조가 사라지게 됨
- 28*28 → 784가 되면 어떤 픽셀이 어떤 픽셀 옆에 있었는지 정보가 없어짐

→ 격자 유지 : 가까운 픽셀 관계 확인

❑ 파라미터 폭발
- Fully-connected 구조
- 입력 784, Hidden 1000개면 연결만 약 78만 개
- 학습 자체가 무거워짐

→ 파라미터 절약

❑ 위치 의존
- Fully-connected 구조
- 입력 i번째 자리가 특정 가중치에 묶이게 됨 : 입력 위치마다 별도 가중치 학습
- 학습 과정에서 위치가 바뀌면 처음부터 다시 배워야 함

→ 위치 무관 재사용 : 배운 패턴을 어디서나 적용

필요한 새로운 성질을 구조에 미리 반영 = 귀납적 편향

- 합리적으로 예측하고 일반화하기 위해
- 사전에 부여하는 추가적인 가정이나 제약 조건

Source : https://www.researchgate.net/figure/A-hypothetical-example-of-Multilayer-Perceptron-Network_fig4_303875065

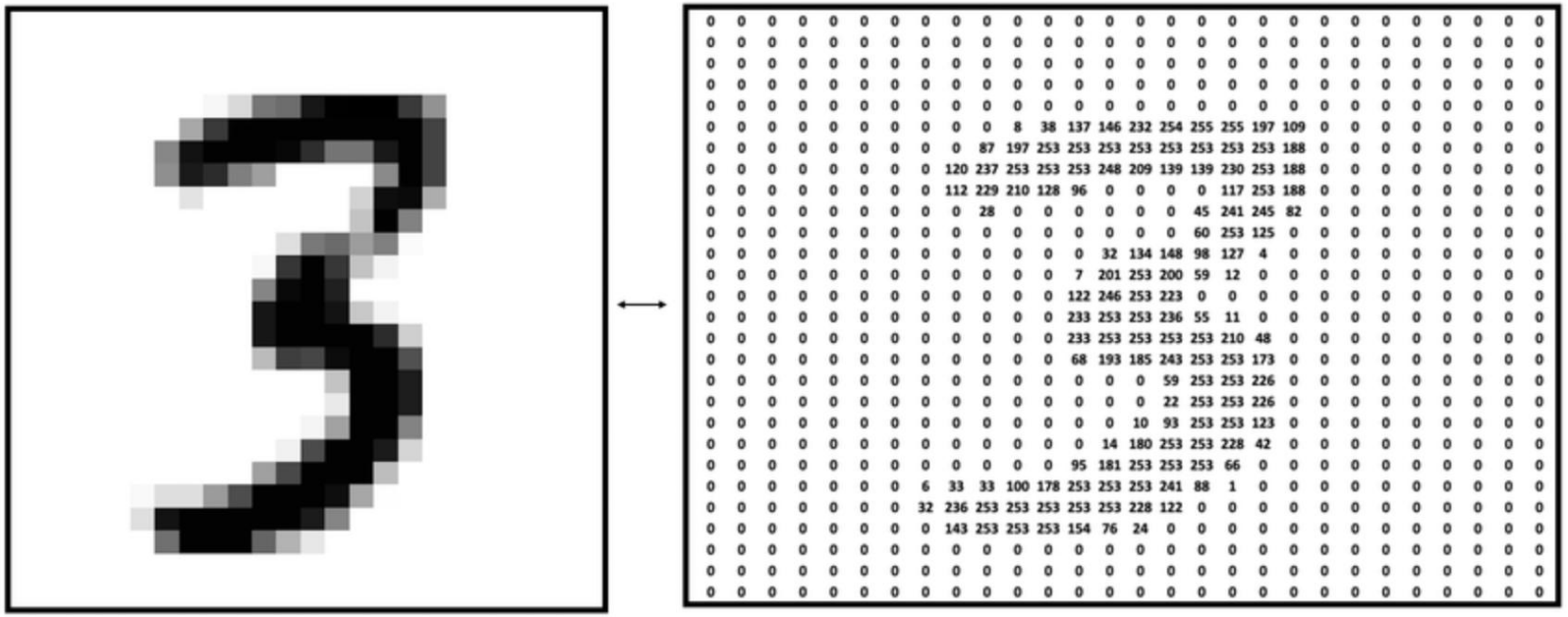
MNIST

Modified National Institute of Standards and Technology database



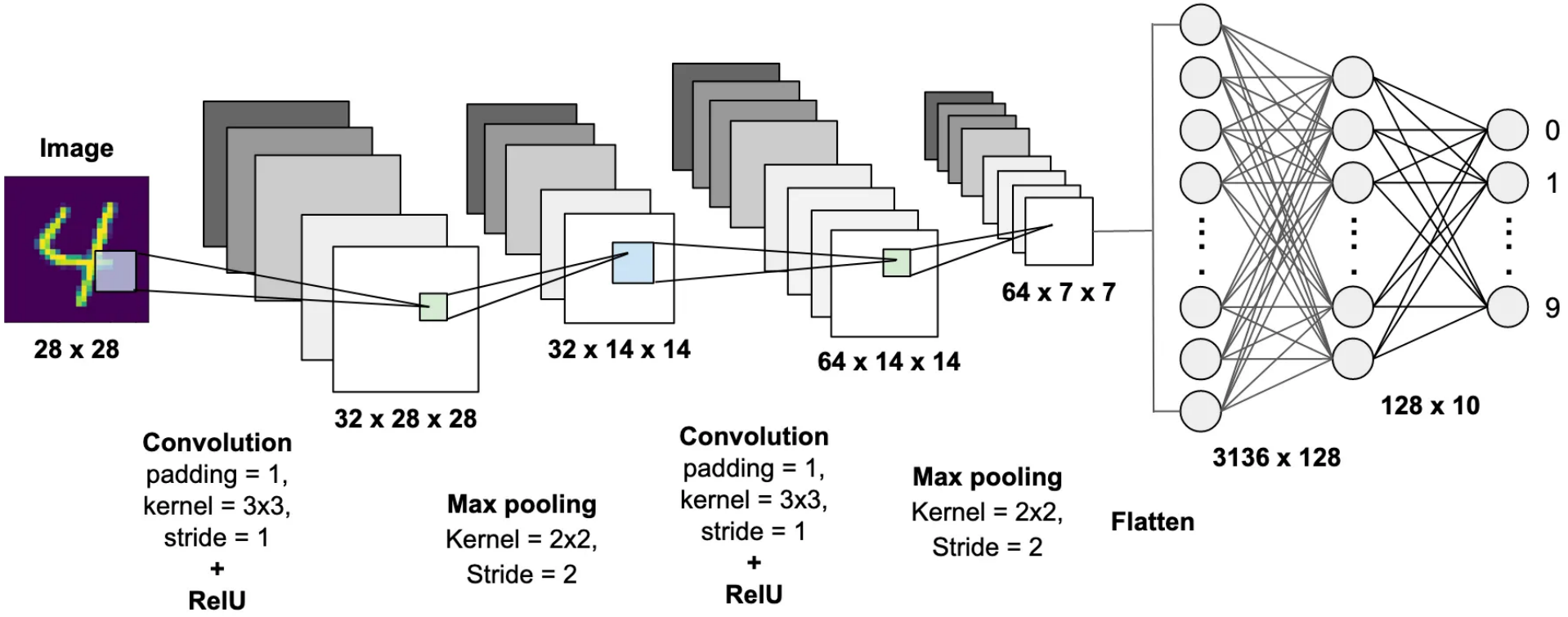
MNIST

Modified National Institute of Standards and Technology database



CNN Architecture

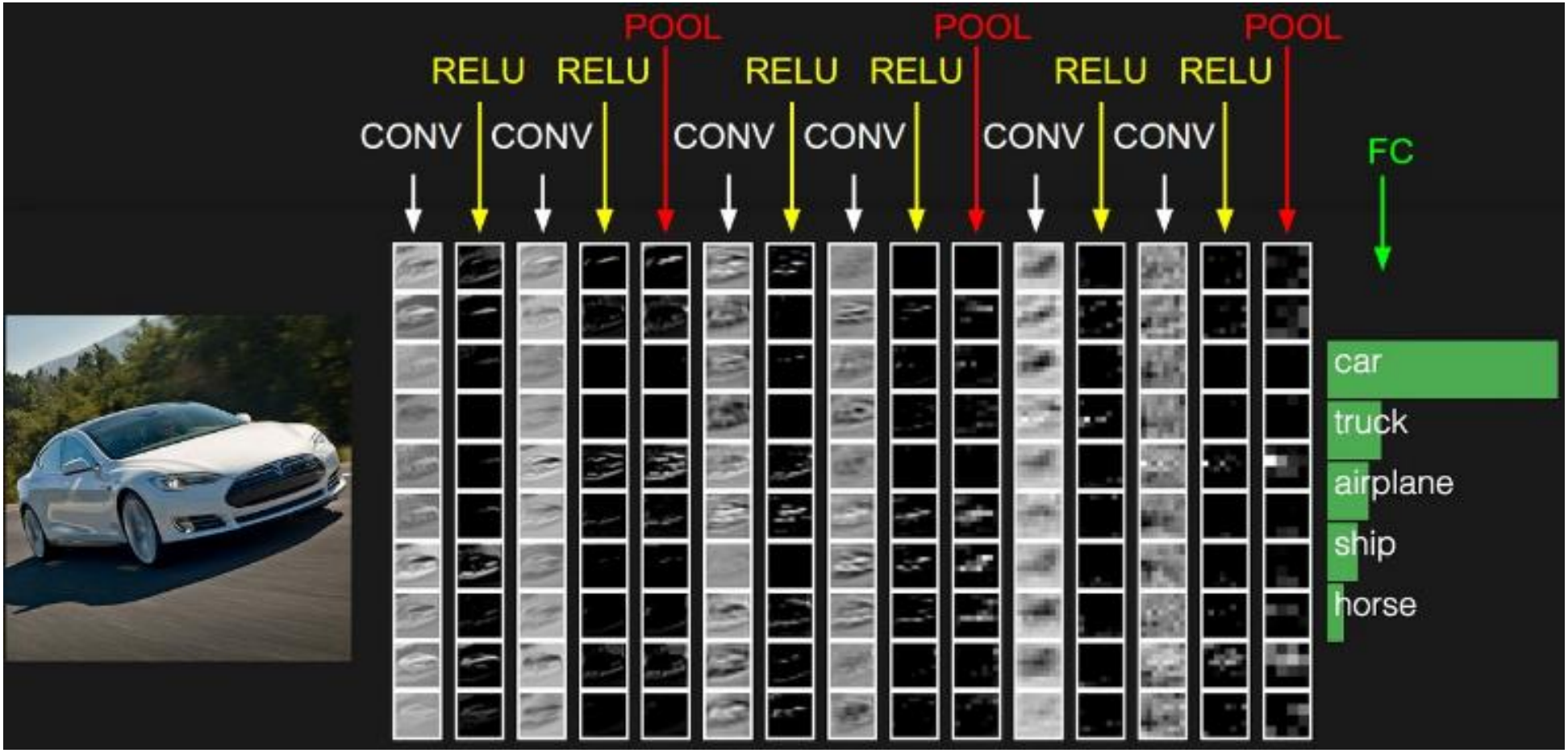
to classify MNIST



Source : https://medium.com/data-science/mnist-handwritten-digits-classification-using-a-convolutional-neural-network-cnn-af5fafbc35e9

CNN Architecture

to classify image (target = car)



Source: http://cs231n.github.io/convolutional-networks/

2. Convolution

용어의 이해

- [프랑스어] 두 개의 것이 합쳐져서 하나가 되는 것 (합성적)
- [수학/IT] 두 함수에 대한 수학적 연산.

  하나의 함수가 다른 함수에 작용하는 필터 역할 담당

  (esp. 신호 처리)

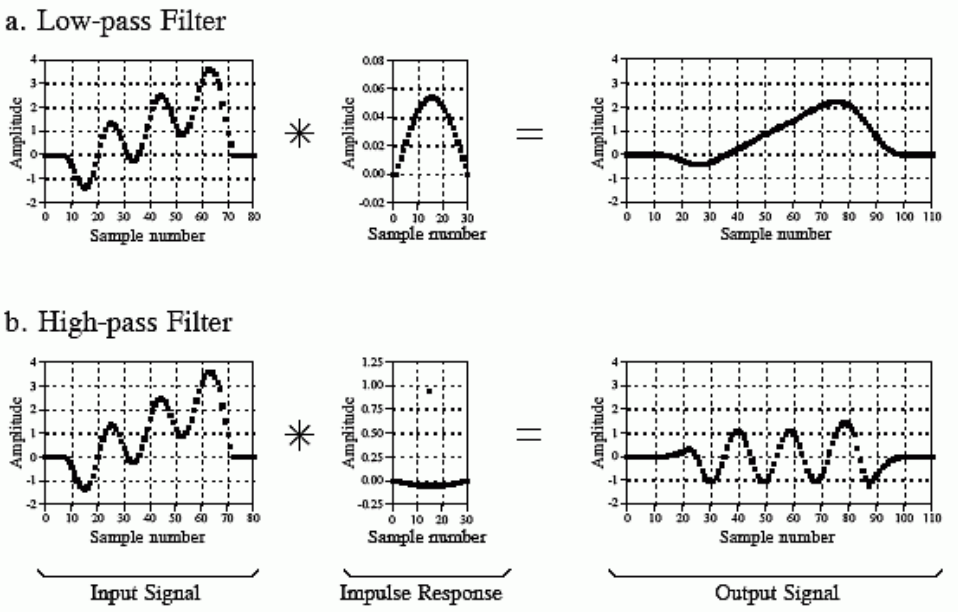


FIGURE 6-3
Examples of low-pass and high-pass filtering using convolution. In this example, the input signal is a few cycles of a sine wave plus a slowly rising ramp. These two components are separated by using properly selected impulse responses.

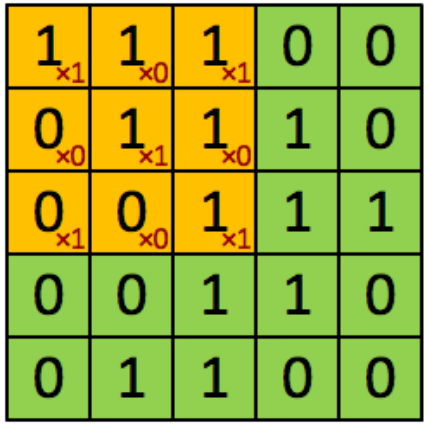
Source : https://www.dspguide.com/ch6/2.htm

2. Convolution

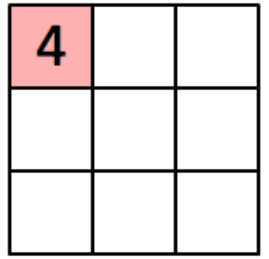
Key Point

용어의 이해

- [프랑스어] 두 개의 것이 합쳐져서 하나가 되는 것 (합성적)
- [수학/IT] 두 함수에 대한 수학적 연산. 하나의 함수가 다른 함수에 작용 하는 필터 역할 담당
- [DL] **작은 필터**(돋보기)가 이미지의 한 부분과 뒤섞이며 **새로운 특징 (이미지)**를 만들어 냄



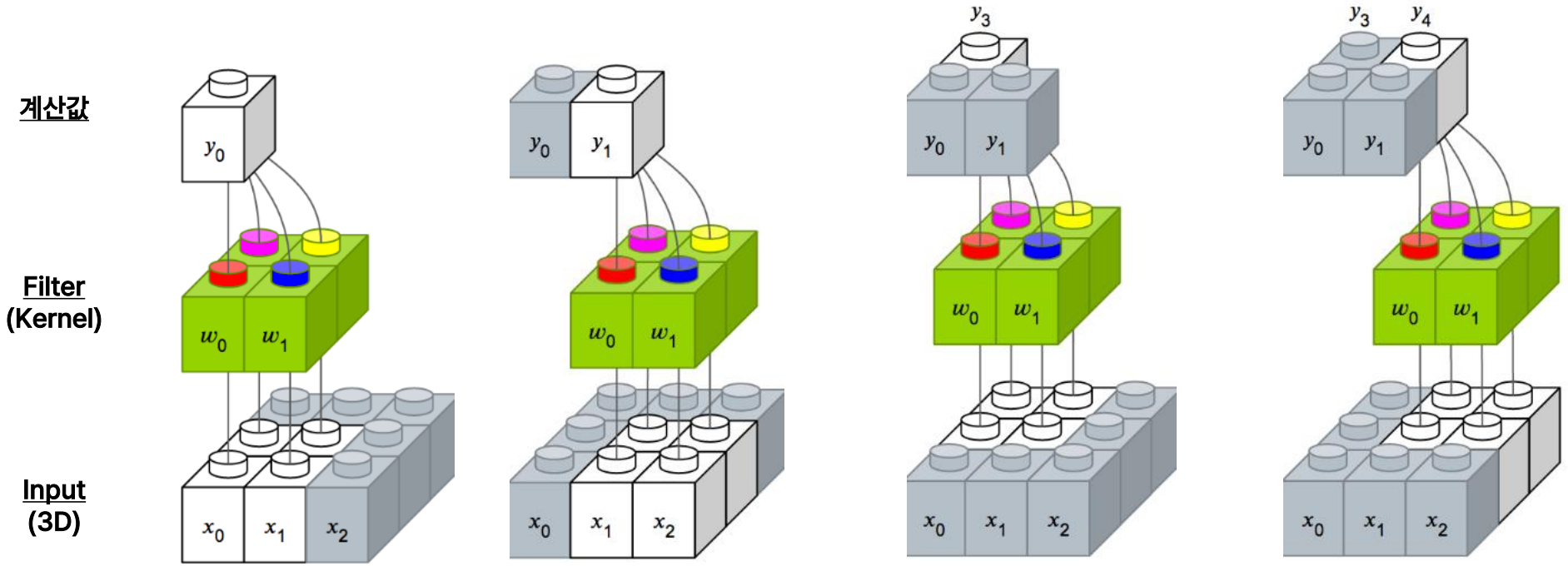
Image



Convolved Feature

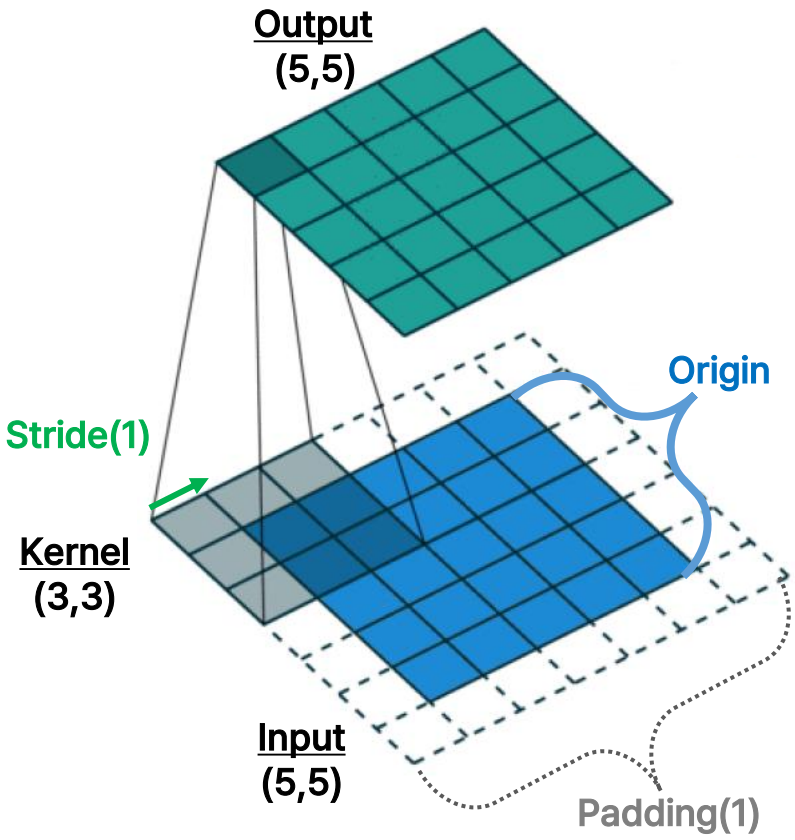
3. Convolution Layer

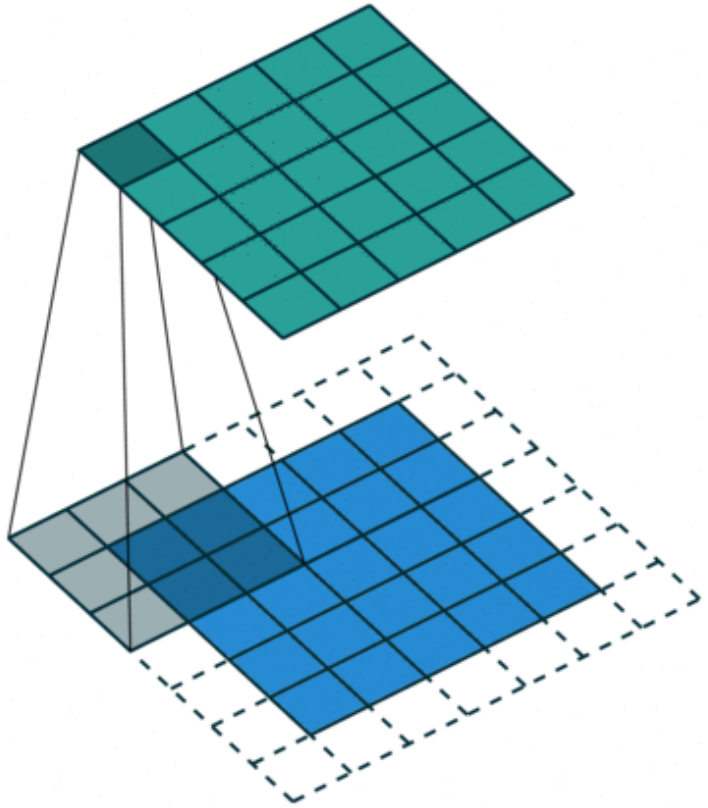
- 3차원 이미지 정보에서 이미지 특징 (Feature Map)을 뽑아내는 계층
- Input 이미지와 Filter 간의 연산



Source : https://tykimos.github.io/Keras/2017/01/27/CNN_Layer_Talk/

3. Convolution Layer

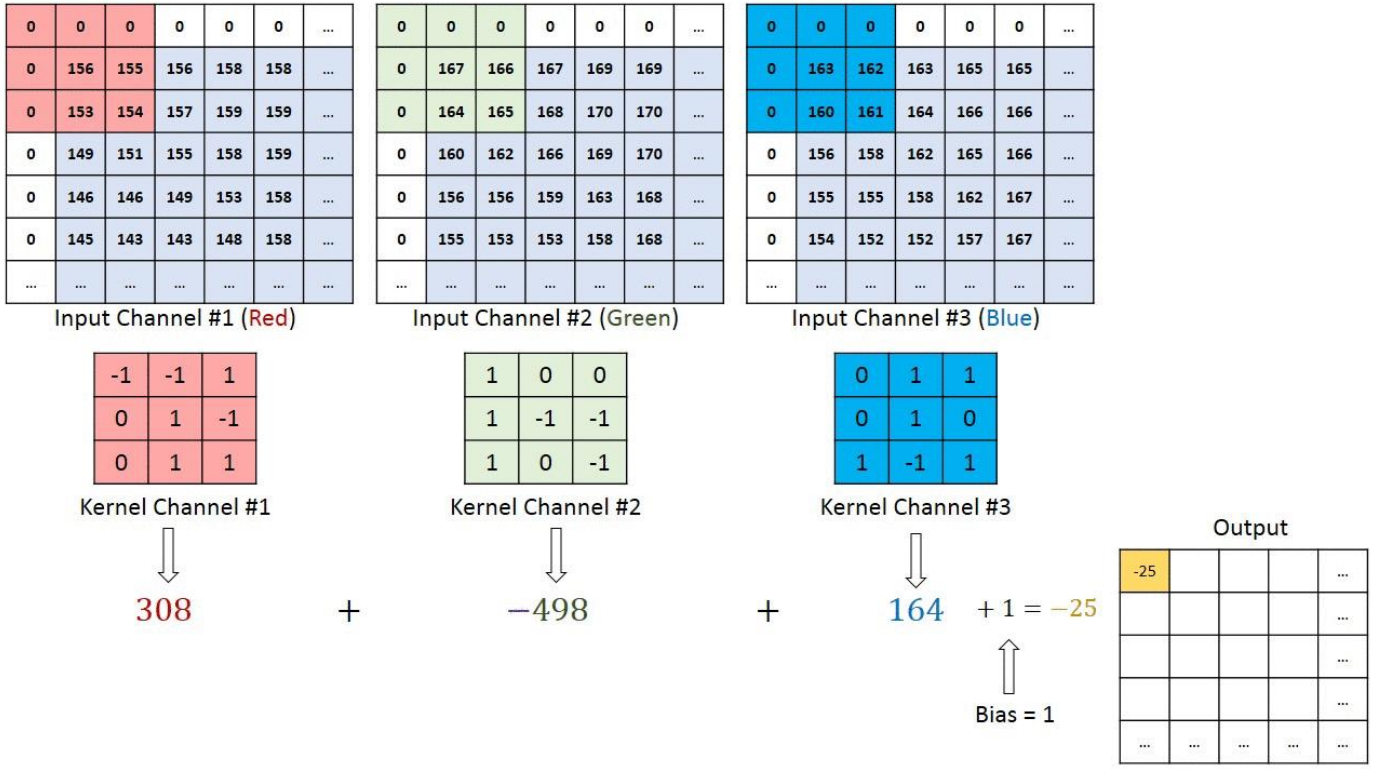




Source : https://github.com/vdumoulin/conv_arithmetic

3-(1). Kernel

- Kernel = Filter
- 합성곱 연산을 수행하는 작은 창, 이미지 위를 움직이며 합성곱 연산을 수행하여 특징을 뽑아내는 역할



Source : https://saturncloud.io/blog/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way/

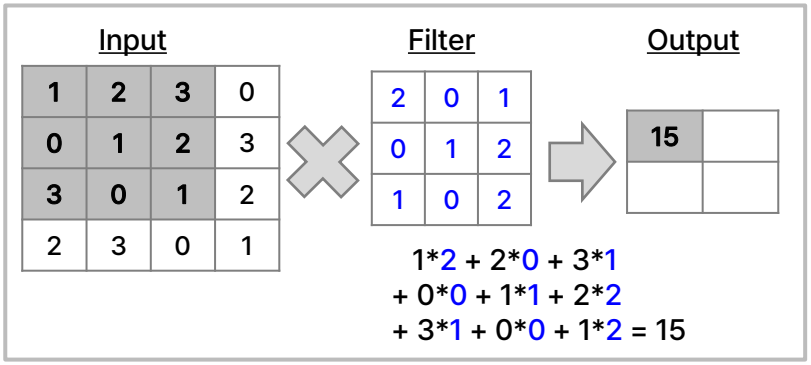
3-(2). Filter

합성곱 연산, 필터 연산

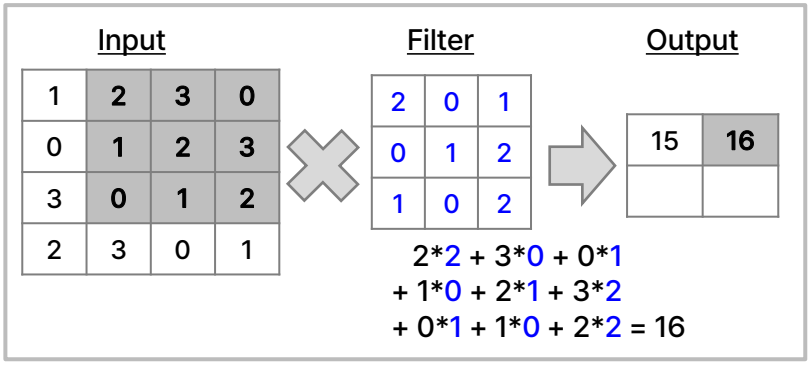
- 합성곱 연산은 Input map에 Filter window 크기 만큼 적용하여 연산
- 연산은 Window 크기를 유지하면서 Stride 만큼 움직이면서 반복 수행

※ Filter (3, 3)을 1 stride 적용 시

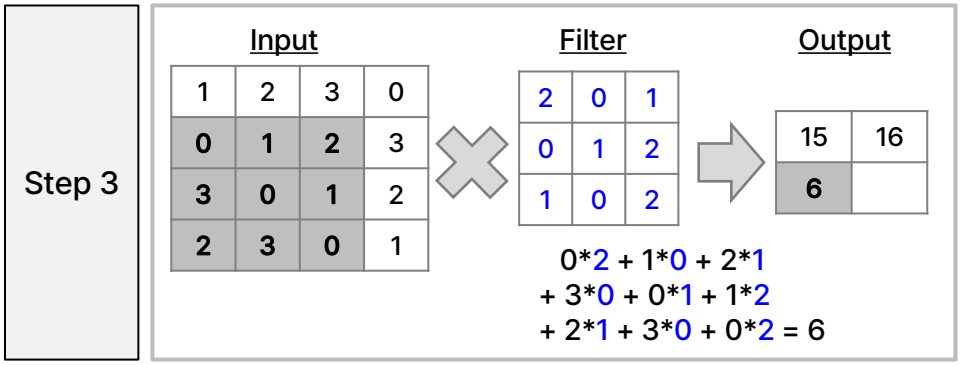
Step 1



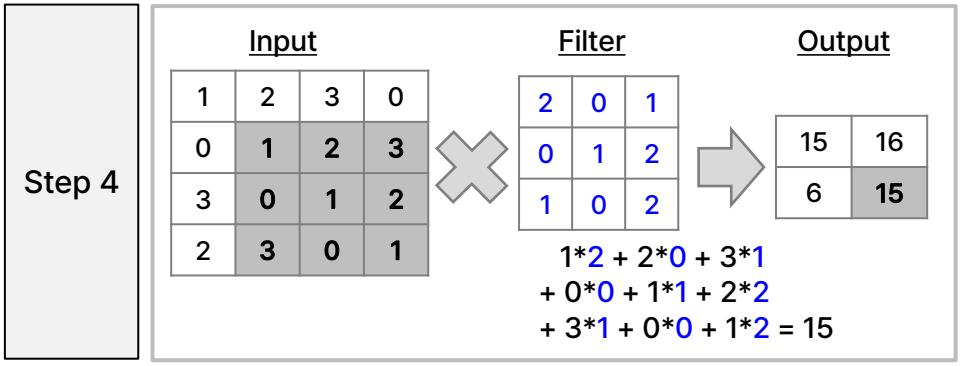
Step 2



Step 3



Step 4



3-(3). Padding

- [영어사전] (폭신하게 만들거나 형태를 잡기 위해 안에 대는) 속, 충전재
- 합성곱 연산 후 변형되는 Output size를 보정하기 위해 Input 데이터에 특정값 (주로 0) 채움

※ Input Padding 1 처리하고 Filter (3,3)을 Stride 1 적용

Input Feature Map

0	0	0	0	0	0
0	1	2	3	0	0
0	0	1	2	3	0
0	3	0	1	2	0
0	2	3	0	1	0
0	0	0	0	0	0

(4, 4) &
Zero-padding 1

Filter

2	0	1
0	1	2
1	0	2

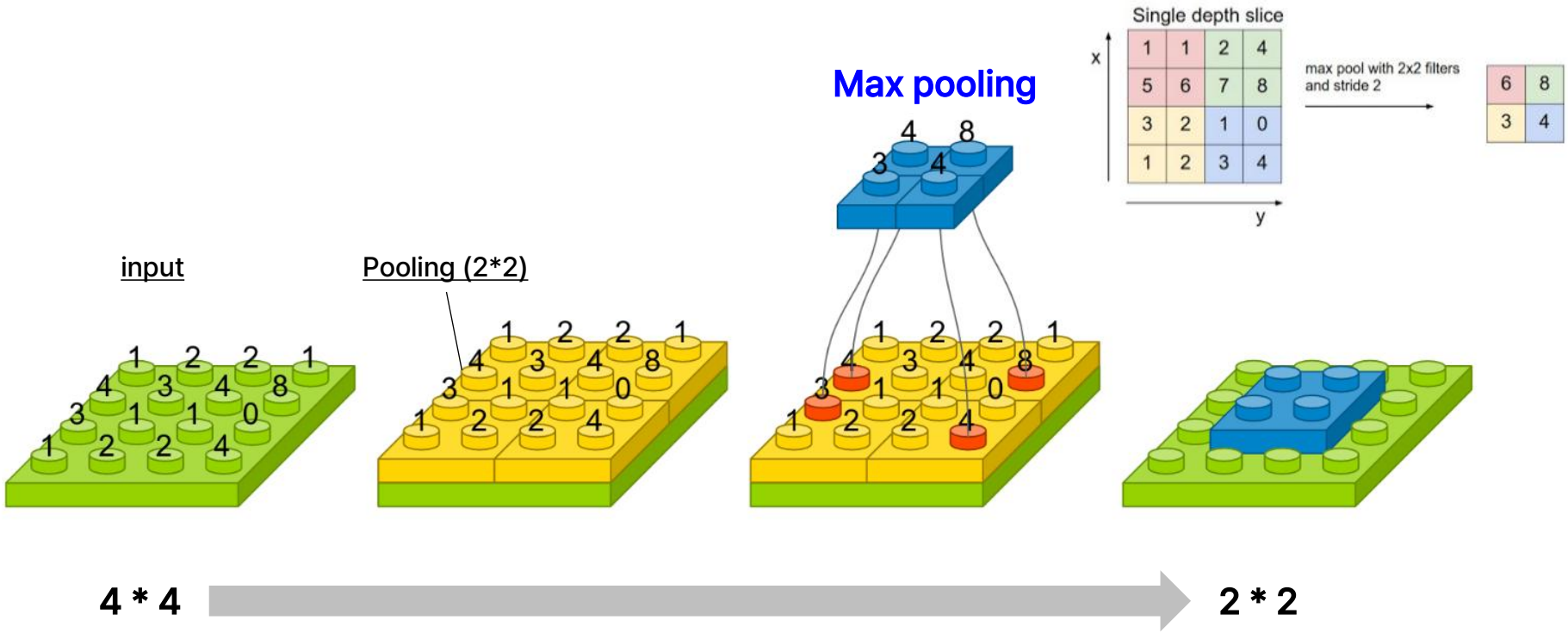
Output Feature Map

7	**12**	**10**	**2**
4	**15**	**16**	**10**
10	**6**	**15**	**6**
8	**10**	**4**	**3**

(1,1) 합성곱 연산
0*2 + 0*0 + 0*1
+ 0*0 + 1*1 + 2*2
+ 0*1 + 0*0 + 1*2 = 7

4. Pooling Layer

- 대표값을 추출하여 작은 이미지 생성
- 사진 축소 통한 좋은 이미지 획득. 대표적으로 Max-pooling 사용 (학습 파라미터 없음)

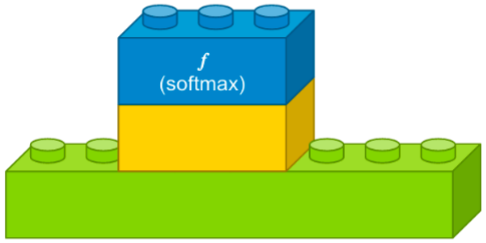


Source : https://tykimos.github.io/Keras/2017/01/27/CNN_Layer_Talk/

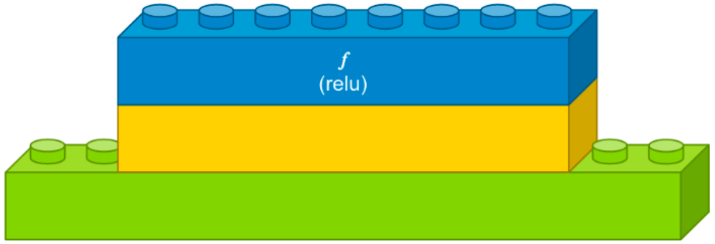
5. Flatten Layer

- Flatten Layer, FC Layer, Fully-Connected Layer, Affine Layer
- 분석한 이미지 데이터를 라벨 분류 하기 위해 1차원으로 바꿔주는 역할

Dense Layer

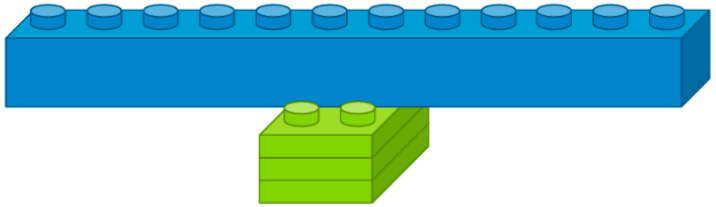


Softmax 적용 (Classification)
뉴런 개수는 라벨 개수와 동일



Activation Function 적용

Flatten Layer
(Fully-Connected Layer)

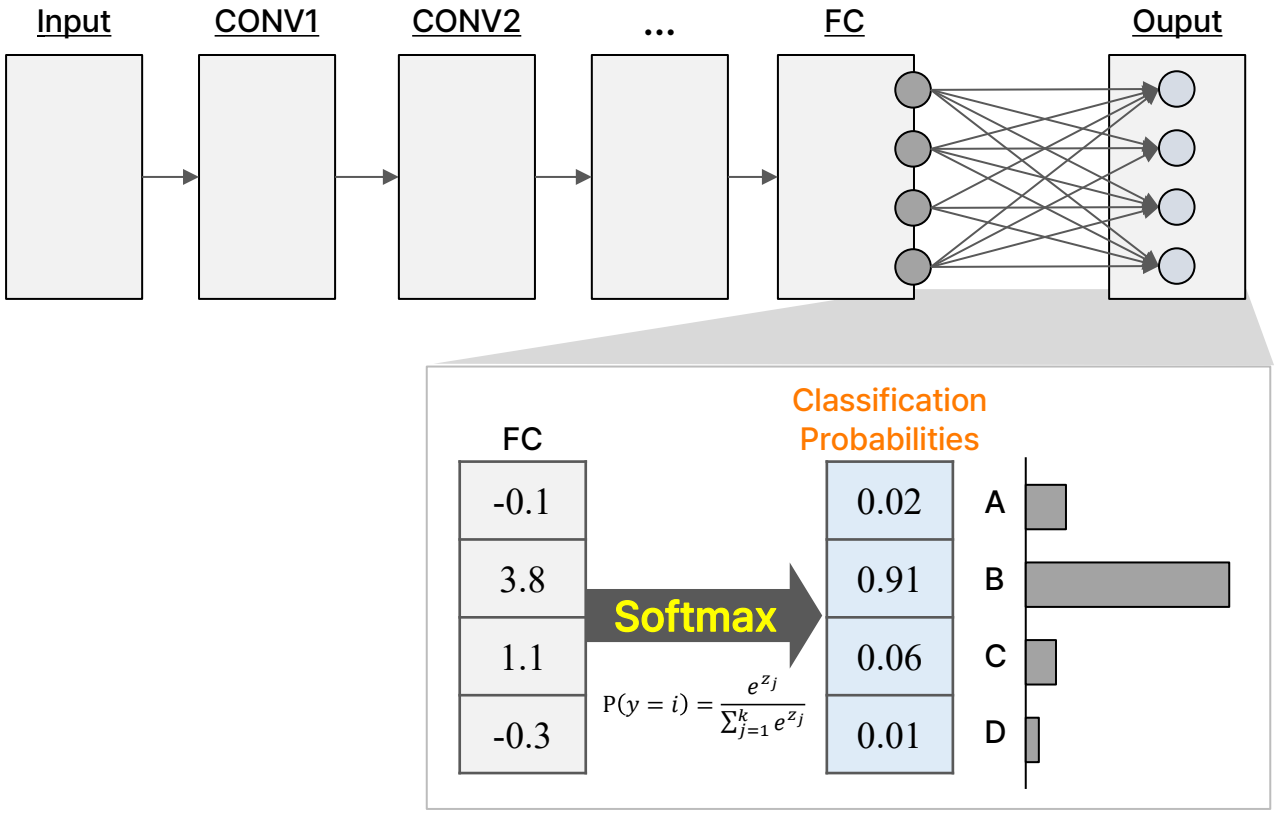


N차원 데이터를 1차원으로 바꿈

Source : https://tykimos.github.io/Keras/2017/01/27/CNN_Layer_Talk/

6. Softmax Function

- Softmax, Argmax, Argument Max : Max값을 가짐을 증명하다
- CNN Architecture 통해 Max로 분류 되었음을 증명하다. Max값의 위치가 몇 번째인지 증명하다



Softmax

- 모델의 출력값을 확률로 계산 (SUM = 1)
- 가장 큰 값의 분류로 예측
- 뉴런 개수만큼 모두에게 영향 미침
  (feat. Fully Connected Layer)

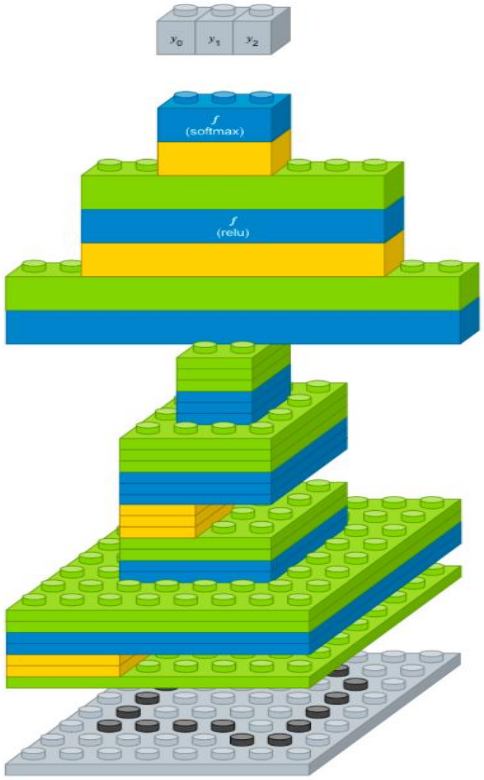
➔ 분류 문제에서 모델 결과값을 확률값으로 만드는데 사용되는 함수

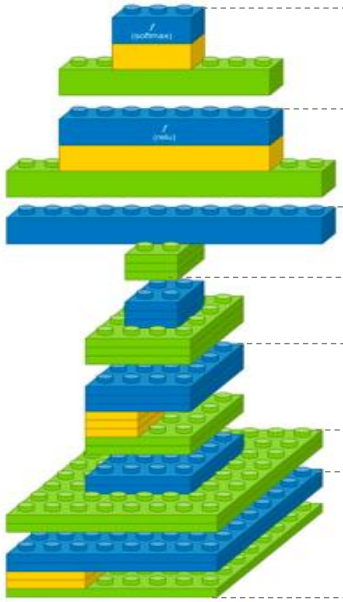
정리하면...

이미지 특징을 뽑아내고(Conv), 중요한 것만 추려내고(Pooling), 펼쳐서(FC), 최종 분류(Dense)하는 구조

Convolution Layer | Pooling Layer
Hidden Layer | Output Layer

[(Conv $*\alpha$ + Activation) $*\beta$ + Pooling] $*\gamma$ + FC Layer + Dense Layer





Dense Layer
Dense(Input 8 → 3 Neurons), Softmax

Dense Layer
Dense(Input 12 → 8 Neurons), Relu

Flatten Layer

Output Layer

Pooling Layer

Convolution Layer
Input 4*4, Kernel 2*2, Filter 3ea, ReLU

Pooling Layer

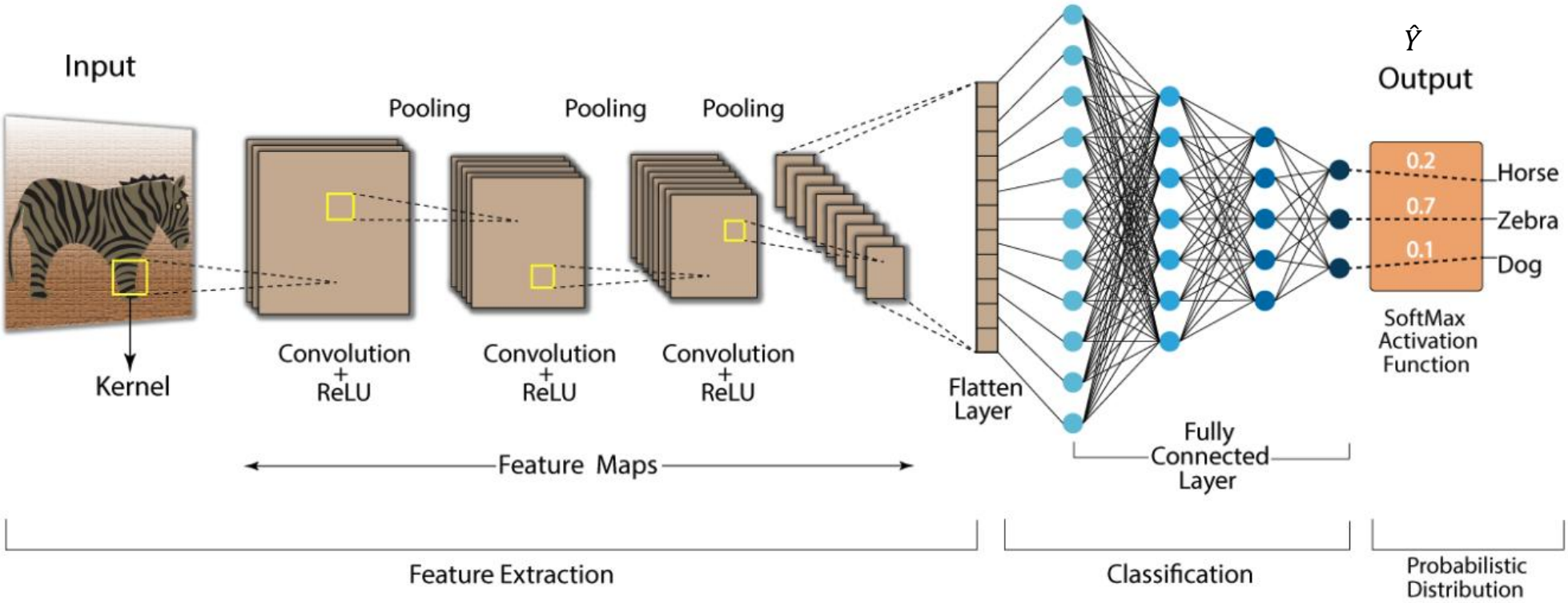
Convolution Layer
Input 8*8, Kernel 3*3, Filter 2ea, ReLU

Hidden Layer

Input Layer

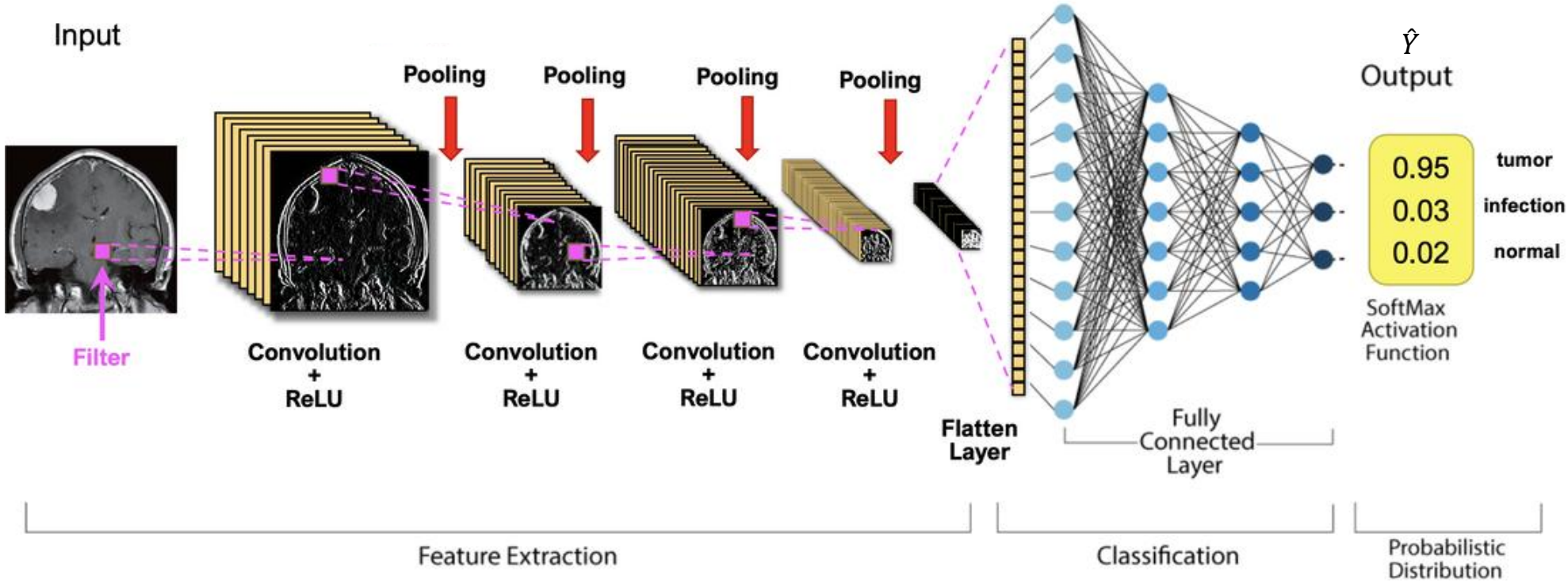
Source : https://tykimos.github.io/2017/01/27/CNN_Layer_Talk/#google_vignette

[참고] Simple CNN



Source : https://developersbreach.com/convolution-neural-network-deep-learning/

[참고] Simple CNN

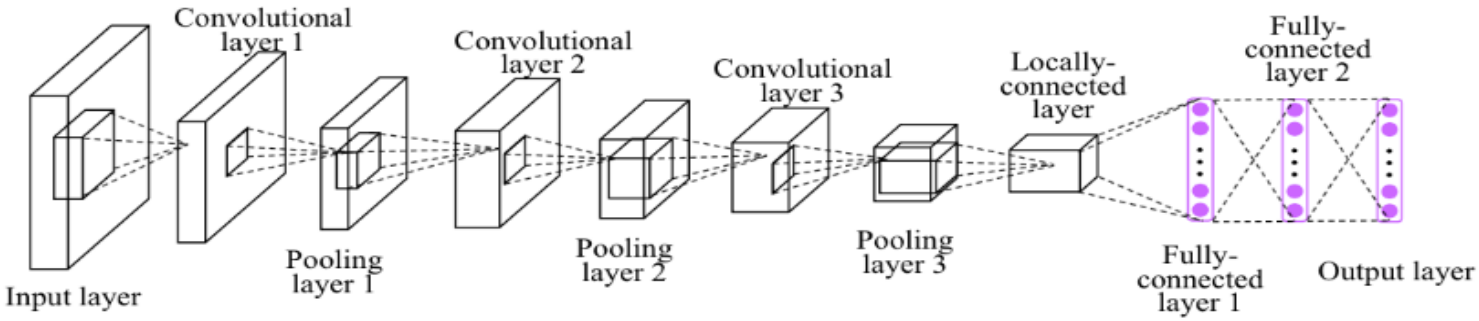


Source : https://mriquestions.com/convolutional-network.html

CNN

SUMMARY

이미지 특징을 뽑아내고(Conv), 중요한 것만 추려내고(Pooling), 펼쳐서(FC), 최종 분류(Dense)



Convolution Neural Network

Convolution Layer

- 3차원 이미지 정보에서 이미지 특징(feature map)을 뽑아내는 계층
- 이미지와 필터 간의 연산. 원본 이미지 변형 없음

합성곱 연산 (필터 연산)

- Input featrue map에서 filter window 만큼 적용하여 연산
- 연산은 window 크기만큼 유지하면서 stride sliding 반복 수행

Padding

- 합성곱 연산 결과 변형되는 output size 보정
- Intput feature map에 특정값(주로 0)을 채움

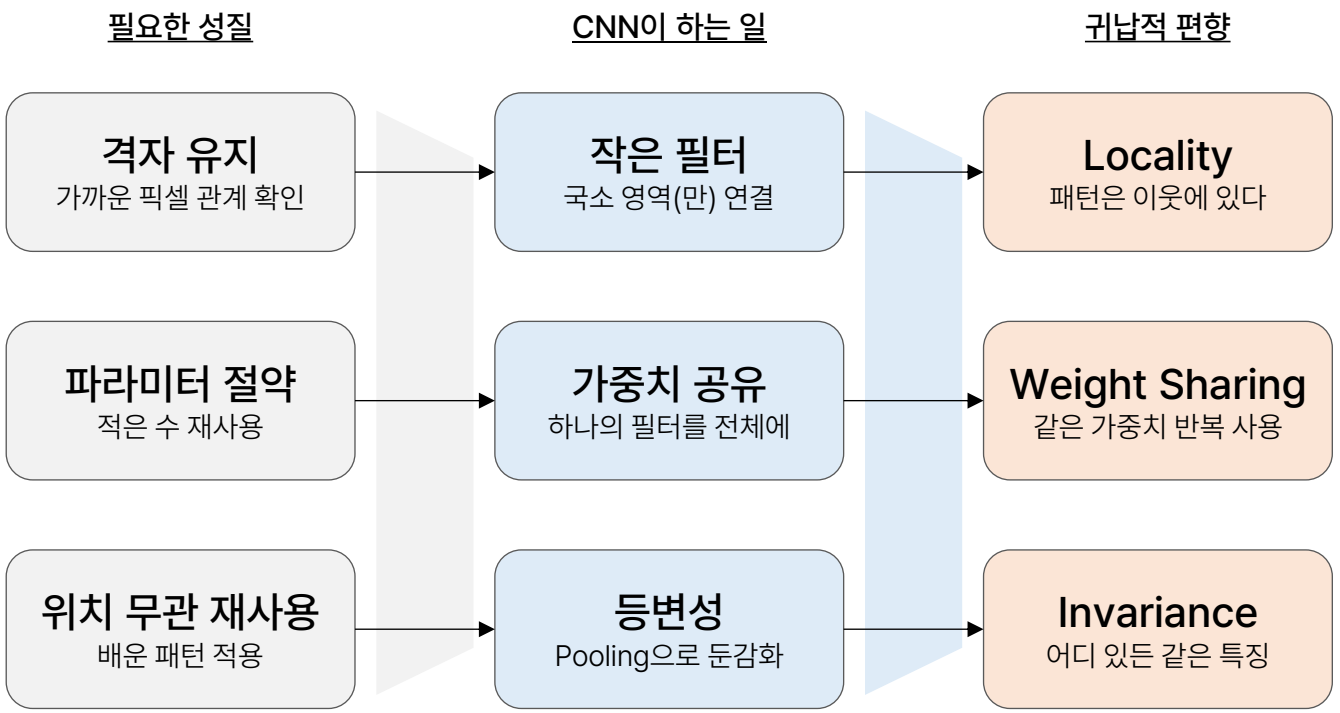
Pooling Layer

- Convolution Layer의 합성곱 연산 결과에서 추가적으로 공간을 줄이는 연산
- Max-pooling 주로 사용됨. 학습 Parameter 없음

CNN

SUMMARY

이미지는 지역적이고, 특징은 어디서나 똑같이 쓸모 있으며, 물체의 정체는 위치와 무관하다



의미 있는 패턴(모서리, 질감 등)은 멀리 떨어진 픽셀이 아니라 **가까운 픽셀들** 사이에 있음

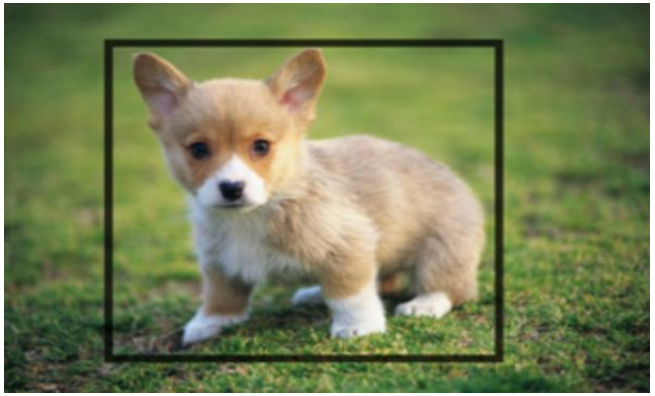
한 위치에서 쓸모 있는 특징 검출기는 **다른 위치에서도** 똑같이 쓸모 있음

물체의 정체는 **위치 정보와 무관**
: 입력 위치가 이동하면 특징맵도 같이 이동하지만, 풀링을 거치면서 위치 변화에 둔감해짐

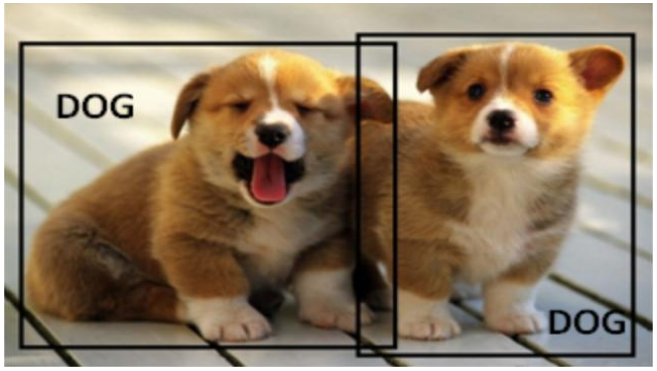
[참고] Classification



Object **classification** is the task of identifying as a dog.



Object **Localization** involves the class label as well as a bounding box to show there the object is located.



Object **Detection** involves localization of multiple objects.



Object **Segmentation** involves the class label as well as an outline of the object in interest.

Source : https://adeshpande3.github.io/adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks-Part-2/

AI Campus 데이터분석 및 AIOps > Part C – DL Architecture

1B. 귀납적 편향의 설계

데이터의 구조를 코드에 새기다

- RNN – 순차성과 시간적 의존성
- LSTM – 무엇을 기억하고 무엇을 잊을 것인가

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

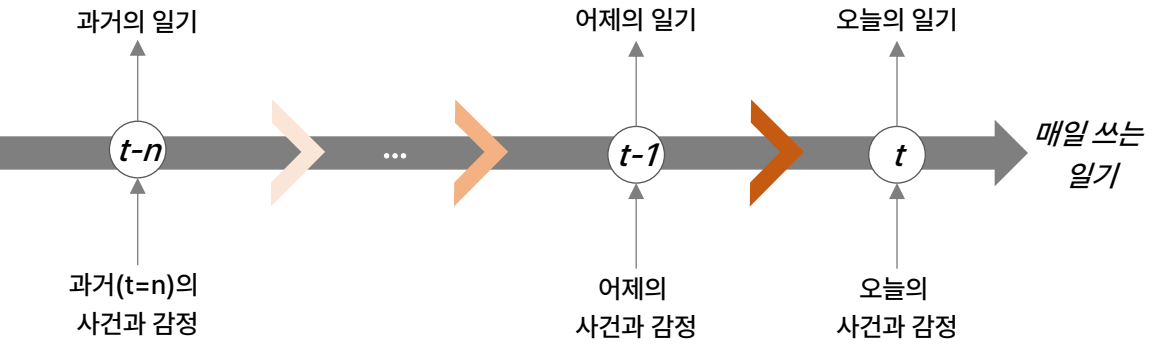
SK Inc. AX

1. Recurrent

시간에 따라 변화하는 데이터를 이해하기 위해, 과거의 정보를 기억하며 현재를 처리하는 구조

비유적 설명 : 일기

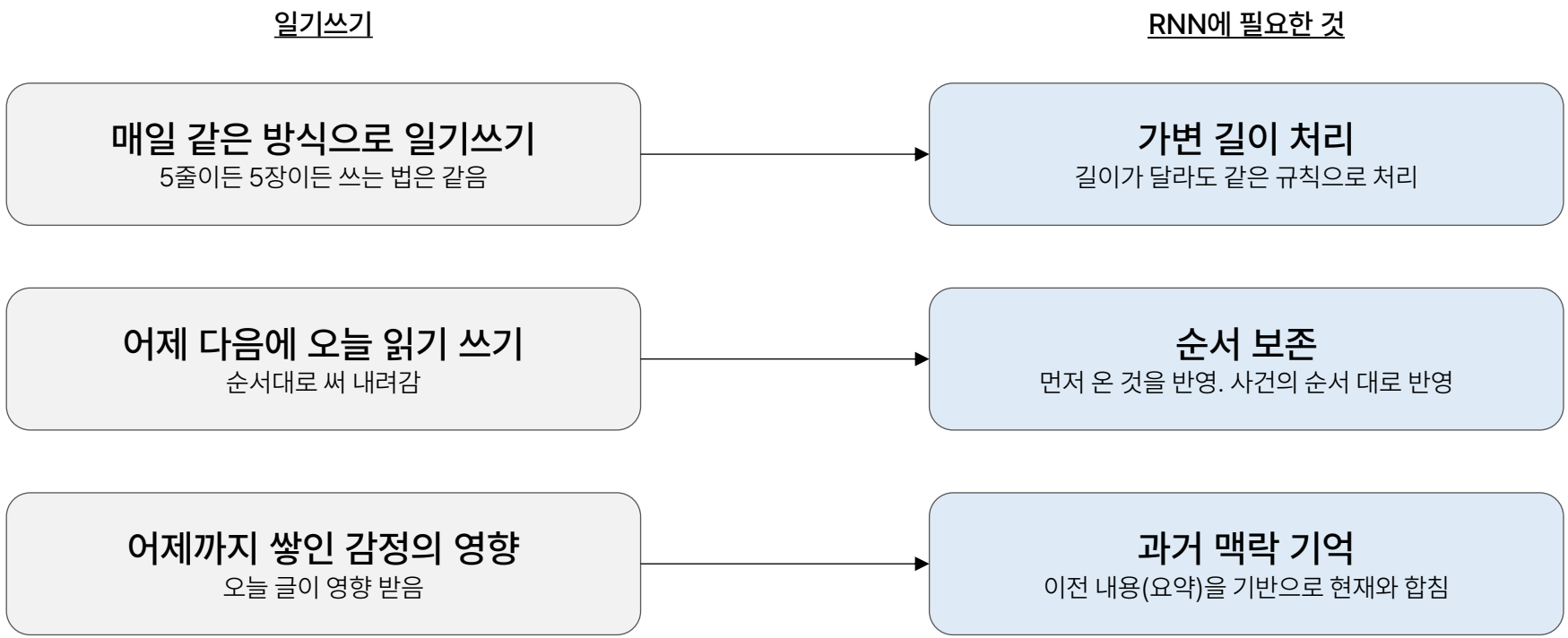
- 매일 밤 일기를 쓸 때, 하루를 돌아보며 사건이나 쌓인 감정을 정리
- 하루하루의 감정이나 사건들은 다음 날의 글에 조금씩 영향을 미침
- 만약 과거의 일이 너무 오래되어 기억이 흐릿해지거나 잊어버릴 수 있음





1. Recurrent

매일 같은 방식으로, 어제 다음 오늘을, 쌓인 기억으로 쓰는 일기



1. Recurrent

용어의 이해

Recurrent

- [영어사전] 되풀이되는, 반복되는, 재발하는, 회귀하는
- 일기에서 매일 반복되는 습관

Recurrent Neural Network

- **시퀀스(Sequence) 데이터 처리에 특화된 인공 신경망**

  시퀀스 데이터란 음성, 텍스트 또는 시간에 따라 측정되는 값으로, 데이터가 순차적으로 생성되며 데이터들 간의 순서나 관계가 중요하다는 특징을 가짐

  이를 효과적으로 고려하기 위해 이전 시점의 정보를 내부에 저장하는 일종의 기억 장치 가짐

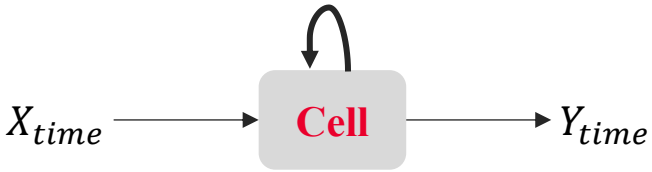
- 문장의 의미는 이전 단어들(기억 상태)에 따라 결정되는데, **RNN은 앞 내용을 기억하여 뒷 내용을 예측하는 구조**

2. RNN

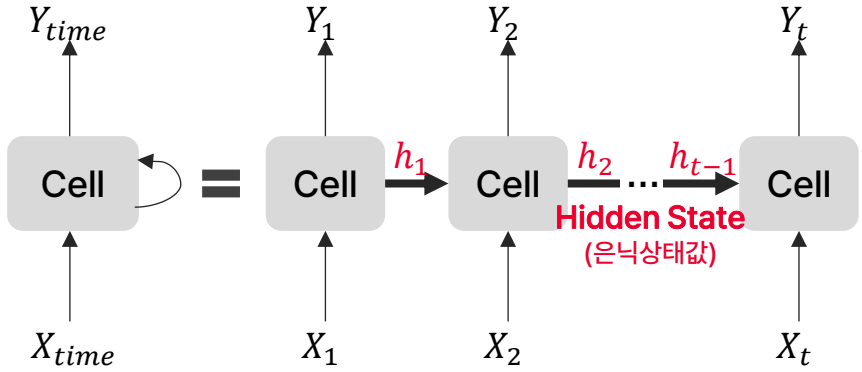
- RNN : 메모리를 저장하는 네트워크
- Cell, Memory Cell : Hidden Layer에서 Activation Function 통해 결과를 내보내는 역할 (내부 연산 단위)
- Hidden State : 여태까지 들어온 (과거의) Input 정보 저장. 이를 가지고 다음 단어 예측 (결과값, 결과 상태값)

Cell, Memory Cell

- 이전의 값을 기억하려고 하는 일종의 메모리 역할을 수행하는 노드
- 각각의 시점에서 바로 이전 시점에서의 출력값을 자신의 입력으로 사용



(동일한 Cell을 시간 축을 따라 복사해서 펼쳐 놓은 것처럼 구성)

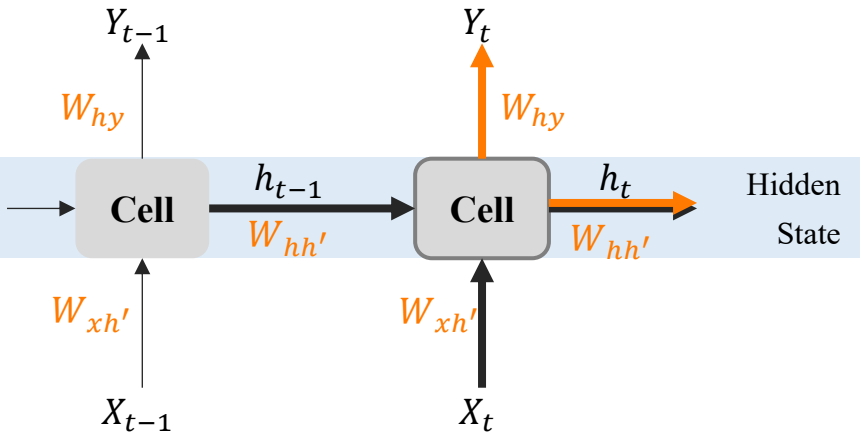


2. RNN

Key Point

Hidden State

- Hidden State : 여태까지 들어온 (과거의) Input 정보 저장. 이를 가지고 다음 단어 예측 (결과값, 결과 상태값)
- 현재 시점(h_t)은 이전 시점(h_{t-1})을 받아서 현재값(X_t)과 함께 다음 상태로 넘김
  → 이를 통해 과거의 정보가 현재로 전달되게 됨



하나의 층 안의 모든 시점에서 동일한 가중치 값($W_{hh'}$)으로 공유

➔ 학습에 필요한 가중치 수를 줄일 수 있음
➔ 흐름에 따른 연관성을 고려하여 예측할 수 있음
(※ Hidden Layer가 2개 이상인 경우, Layer 내에서 동일하나 각각은 다름)

$$Y_t = W_{hy}h_t + b_y$$
$$h_t = \tanh(W_{hh'}h_{t-1} + W_{xh'}X_t + b_h)$$

- Y_t : 현재 상태에서 최종 출력을 만듦
- h_t : 이전 상태와 현재 입력을 결합해 새로운 상태를 만듦
- $W_{xh'}$: 입력데이터 X를 셀에 보내는데 사용되는 가중치
- $W_{hh'}$: 이전 시점 셀의 Hidden State값을 현재 시점 셀로 보내는데 사용되는 가중치
- W_{hy} : 현재 시점 셀의 Hidden State값을 출력 데이터 Y로 보내는데 사용되는 가중치

[참고] RNN, 쉽게 이해하기

$$Y_t = W_{hy} h_t + b_y$$
$$h_t = \tanh(W_{hh'} h_{t-1} + W_{xh'} X_t + b_h)$$

- Y_t : 현재 상태에서 최종 출력을 만듦
- h_t : 이전 상태와 현재 입력을 결합해 새로운 상태를 만듦
- $W_{xh'}$: 입력데이터 X를 셀에 보내는데 사용되는 가중치
- $W_{hh'}$: 이전 시점 셀의 Hidden State값을 현재 시점 셀로 보내는데 사용되는 가중치
- W_{hy} : 현재 시점 셀의 Hidden State값을 출력 데이터 Y로 보내는데 사용되는 가중치

(e.g.)
T = 10, Hidden size = 128, Input size = 64 라면 :

▪ 공유하지 않는다면 :
- 시점마다 별도의 가중치($W_{hh'}, W_{xh'}$) 필요
- 10 * (128*128 + 64*128) = 246,000개

  한 시점의 $W_{hh'}$ 크기 (hidden → hidden)　한 시점의 $W_{xh'}$ 크기 (input → hidden)

▪ 공유한다면 :
- 시점 수와 무관하게 1세트만 존재
- 64 *128 + 128*128 = 24,600개

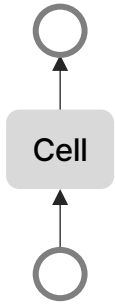
➔ 시퀀스가 아무리 길어져도 파라미터 수는 고정

2-(1). RNN, 다양한 구성

- RNN은 입력과 출력의 길이에 대해 자유로운 구조로 다양하게 설계할 수 있음
- 연속 데이터의 특징을 고려하여 Link의 방향도 단방향 또는 양방향으로 활용할 수 있음

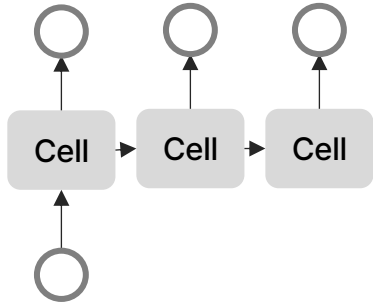
One-To-One

Hidden Layer가 1개인 구조



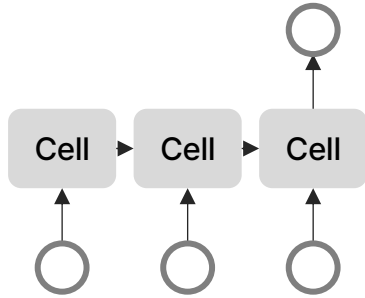
One-To-Many

하나의 입력에 대해 여러 개의 출력을 내보내는 구조



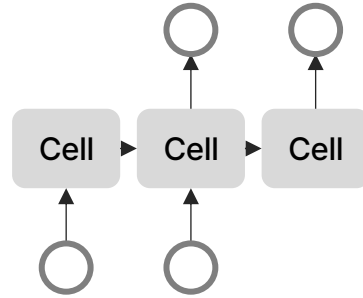
Many-To-One

여러 개의 입력에 대해 단 한개의 출력을 내보내는 구조



Many-To-Many

여러 개의 입력에 대해 여러 개의 출력을 내보내는 구조



2-(1). RNN, 다양한 구성

- RNN은 입력과 출력의 길이에 대해 자유로운 구조로 다양하게 설계할 수 있음
- 연속 데이터의 특징을 고려하여 Link의 방향도 단방향 또는 양방향으로 활용할 수 있음

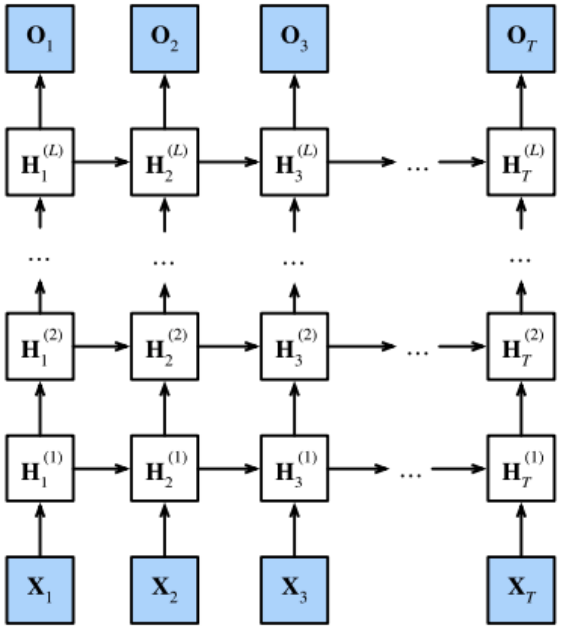


Fig. 9.3.1: Architecture of a deep RNN.

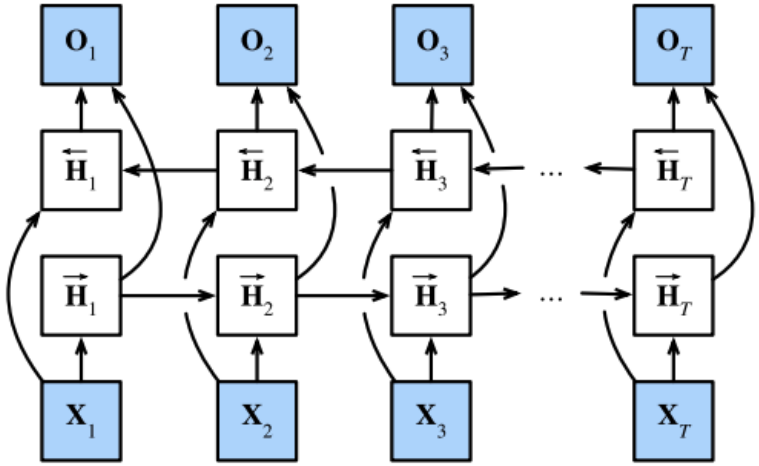
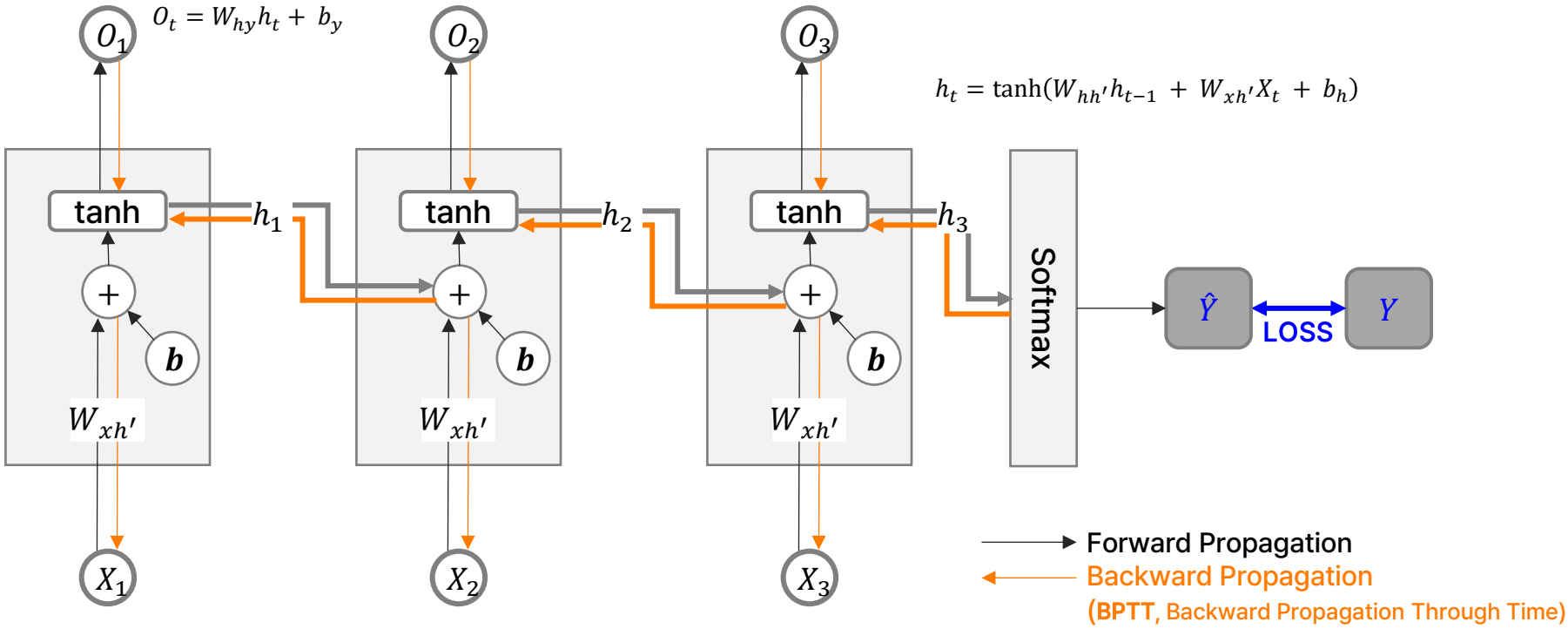


Fig. 9.4.2: Architecture of a bidirectional RNN.

Source : https://classic.d2l.ai/chapter_recurrent-modern/deep-rnn.html

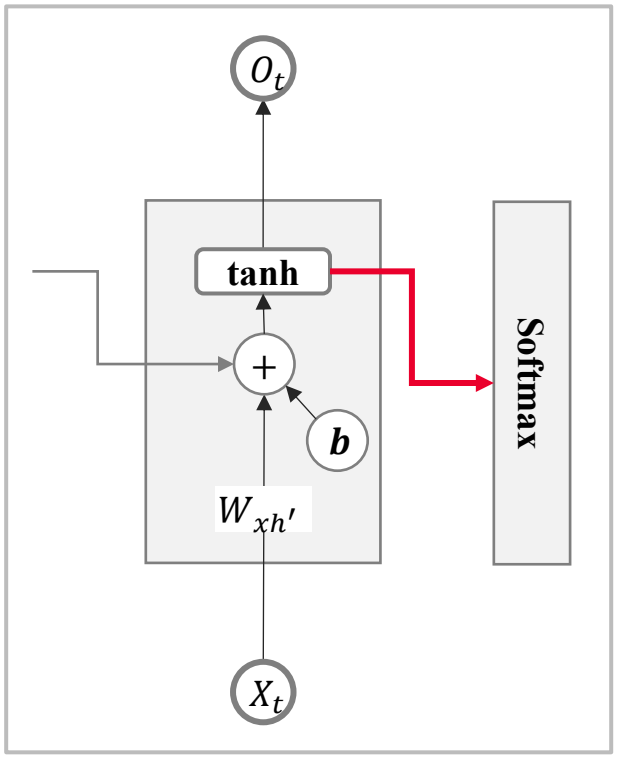
2-(2). RNN Architecture

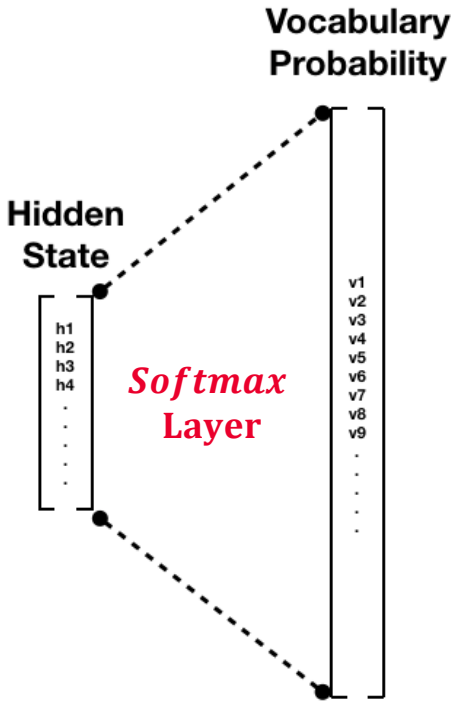
- 시퀀스 데이터의 순서를 고려하여 데이터들 간의 순서나 관계 특징을 저장 (Hidden State)
- 여태까지 들어온 (과거의) Input 정보로 다음 단어 예측
- Hidden State를 오래 유지해야 할 경우, Gradient Vanishing/Exploding 문제 발생 → Archi 개선 (LSTM, GRU 등)

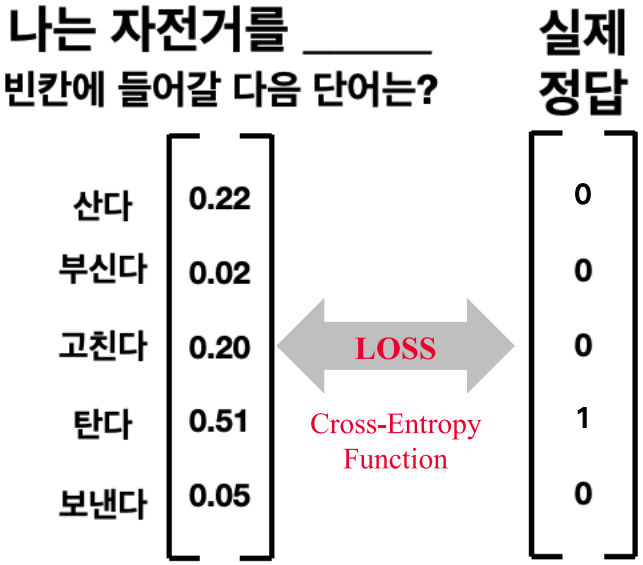


2-(3). Softmax in RNN

- RNN는 시퀀스 데이터의 특징을 저장하고, 이를 기반으로 미래를 예측
- 시계열 데이터인 경우 Softmax 없이 예측값 그대로 Output Layer로 전달
- 텍스트 데이터인 경우 학습한 N개의 단어 중 하나를 예측하므로 Softmax 사용 ➔ Classification



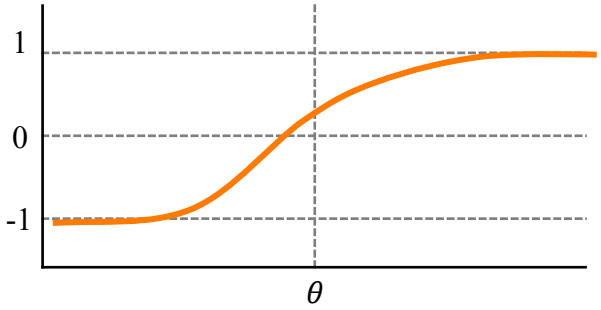




2-(4). RNN with tanh

값의 트랜드 변화에 따라 tanh가 어떤 의미를 보이는지 해석해 보세요 !!

【 Tanh 】



- 정의 : $\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
- 출력 : (-1, 1)
- 특징
  - 출력이 -1과 1 사이로 제한
  - Sigmoid와 유사하지만 출력값이 0 중심으로 분포되어 Gradient vanishing 문제가 덜 발생
- 활용
  - RNN, LSTM 에서 많이 사용
  - 출력값이 양수와 음수로 나뉘는 문제

$$\boldsymbol{h_t = tanh(W_{hh'} \cdot h_{t-1} + W_{xh'} \cdot X_t + b)}$$

$W_{hh'} = 0.3,\ W_{xh'} = 0.01,\ b = 0.1$

Case 1 : X_1 = 200, X_2 = 300, X_3 = 400 → Y=550

- $h_1 = \tanh(0.3 * 0 + 0.01 * 200 + 0.1) = \tanh(2.1) \approx 0.97$
- $h_2 = \tanh(0.3 * 0.97 + 0.01 * 300 + 0.1) = \tanh(3.39) \approx 0.998$
- $h_3 = \tanh(0.3 * 0.998 + 0.01 * 400 + 0.1) = \tanh(4.4) \approx 0.9997$

Case 2 : X_1 = 200, X_2 = 50, X_3 = 130 → Y=180

- $h_1 = \tanh(0.3 * 0 + 0.01 * 200 + 0.1) = \tanh(2.1) \approx 0.97$
- $h_2 = \tanh(0.3 * 0.97 + 0.01 * 50 + 0.1) = \tanh(0.891) \approx 0.712$
- $h_3 = \tanh(0.3 * 0.712 + 0.01 * 130 + 0.1) = \tanh(1.614) \approx 0.924$

Case 3 : X_1 = 200, X_2 = -130, X_3 = 50 → Y=70

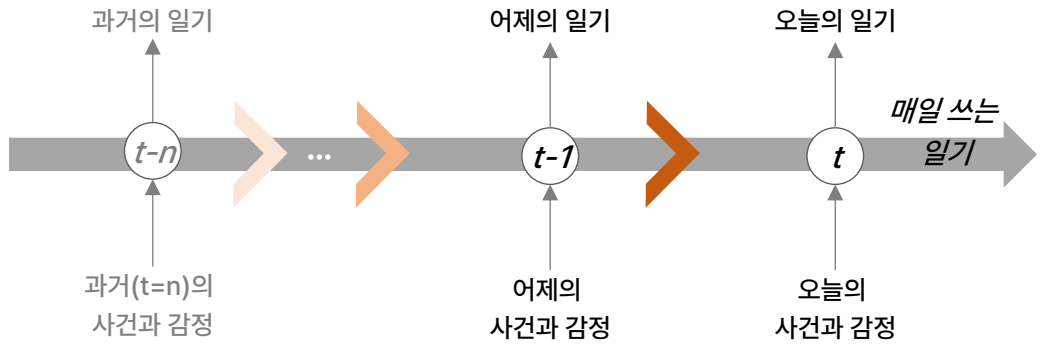
- $h_1 = \tanh(0.3 * 0 + 0.01 * 200 + 0.1) = \tanh(2.1) \approx 0.97$
- $h_2 = \tanh(0.3 * 0.97 + 0.01 * -130 + 0.1) = \tanh(-0.909) \approx -0.721$
- $h_3 = \tanh(0.3 * -0.721 + 0.01 * 50 + 0.1) = \tanh(0.384) \approx 0.366$

3. LSTM

비유적 설명 : 일기 쓰기

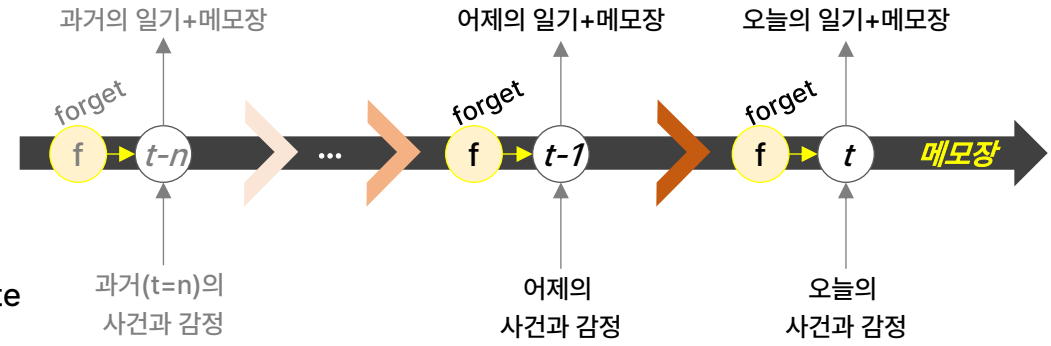
RNN = 매일 쓰는 일기

- 하루하루의 감정이나 사건들은 다음 날의 글에 조금씩 영향을 미침
- 시간이 오래 지나면 과거의 내용이 희미해 지거나 잊혀짐
  → 기울기 소실 문제
- 한계점 : 오래된 기억은 잘 보존되지 않음 → 중요한 사건도 사라질 수 있음



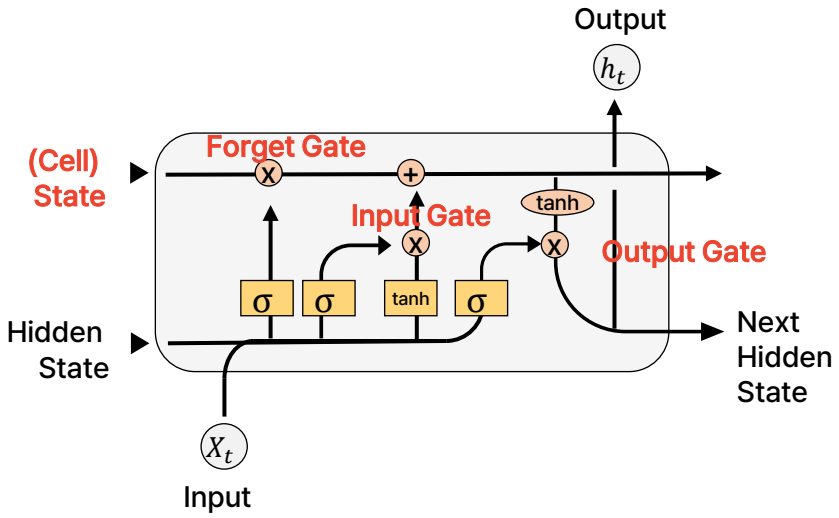
LSTM = 일기 + 중요 메모 노트

- 중요한 사건을 따로 기록하는 메모장을 가진 일기
- 매일 일기를 쓸 때 :
  - 오래된 메모 중에서 불필요한 것은 지움 = forget gate
  - 오늘 일 중 중요한 내용을 메모장에 추가 = input gate
  - 메모장 속 오래된 중요한 내용 참고하여 오늘 일기 작성 = output gate



3. LSTM, Long Short-Term Memory

이전 상태를 유지하는 Cell State와 해당 정보를 조절하는 Gate를 활용하여 과거 정보를 선택적으로 기억하고 활용할 수 있도록 설계



RNN은 과거로 거슬러 갈수록 정보가 사라지거나 무시되기 쉽지만,

LSTM은 Gate 활용하여 Cell State가 장기 기억을 유지할 수 있도록 설계
→ Long-term dependency 문제 해결
→ (여전히) 긴 시퀀스에서는 한계 존재 ➔ Attention 등장 배경

vanilla RNN

$$h_t = \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh_{(c_t)}$$

- c_t : 과거 기억 중 일부(f) 유지 + 새로운 입력(g)을 반영(i)
- h_t : 현재 기억(c_t)에서 필요한 정보(o)를 꺼내서 출력

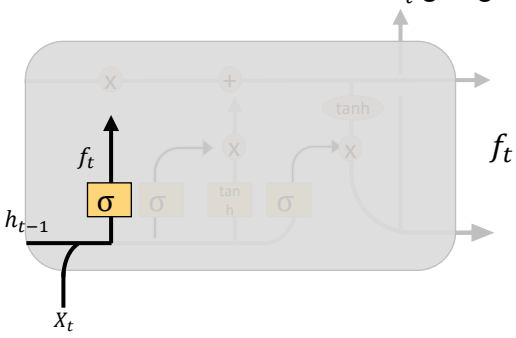
[Remind]
- Cell State : 장기 기억을 저장하는 공간
- Hidden State : 특정 시점의 출력을 나타내는 상태

Source : http://colah.github.io/posts/2015-08-Understanding-LSTMs/

[참고] LSTM Gates

Forget Gate

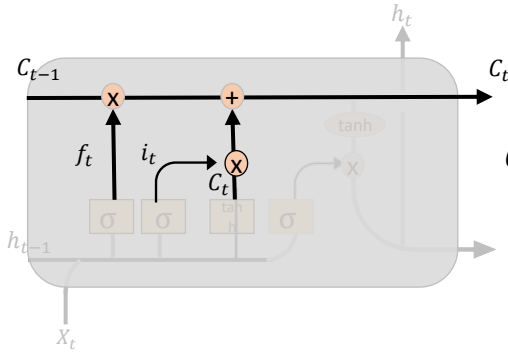
- 이전 기억 중 버릴 것을 선택
- Decide what information we're going to **throw away** from the cell state



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

Update (cell state)

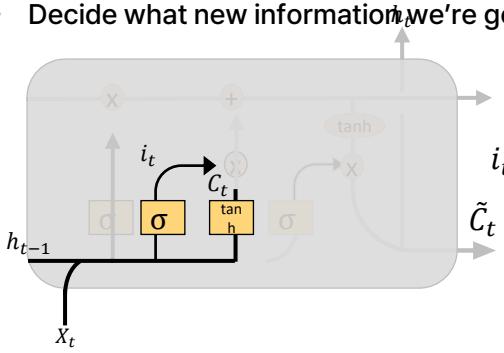
- Update, scaled by how much we decide to update



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Input Gate

- 새로운 정보를 저장할지 선택
- Decide what new information we're going to **store** in the cell state

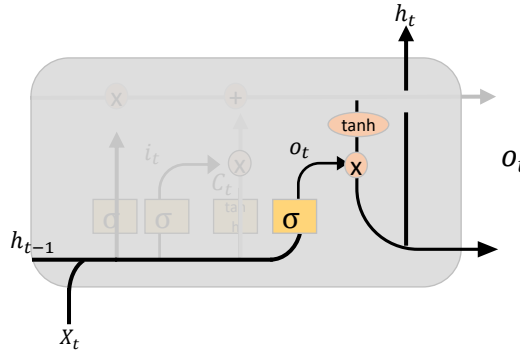


$$i_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_i\right)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_C)$$

Output Gate (hidden state)

- Output based on the updated state



$$o_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_o\right)$$

$$h_t = o_t * \tanh(C_t)$$

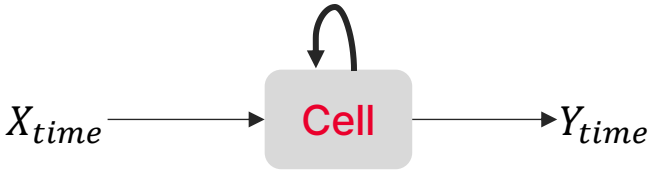
Source : http://colah.github.io/posts/2015-08-Understanding-LSTMs/

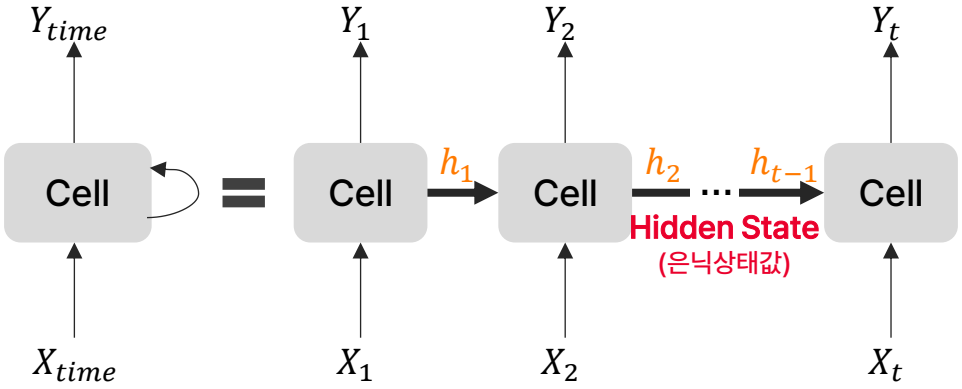
2. RNN

현재 입력 뿐만 아니라 이전 시점의 상태(Hidden State)를 기억하여 시간에 따른 데이터 흐름과 패턴을 학습하는 신경망

Cell, Memory Cell

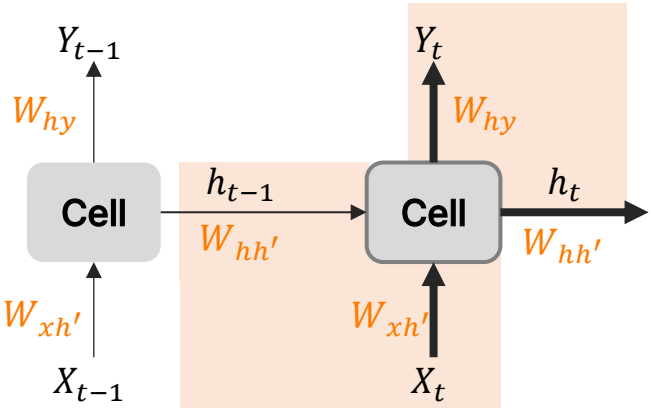
- Hidden Layer에서 Activation Function 통해 결과를 내보내는 역할
- 이전의 값을 기억하려고 하는 일종의 메모리 역할을 수행하는 노드 (내부 연산)
- 각각의 시점에서 바로 이전 시점에서의 출력값을 자신의 입력으로 사용





$$Y_t = W_{hy}h_t + b_y$$

$$h_t = \tanh(W_{hh'}h_{t-1} + W_{xh'}X_t + b_h)$$



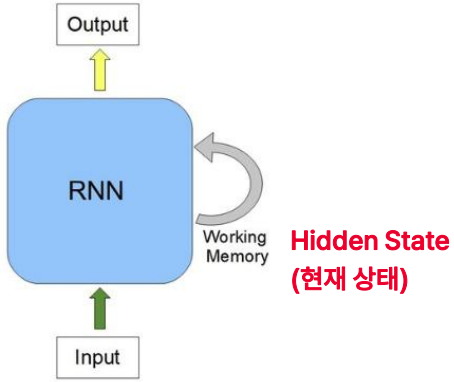
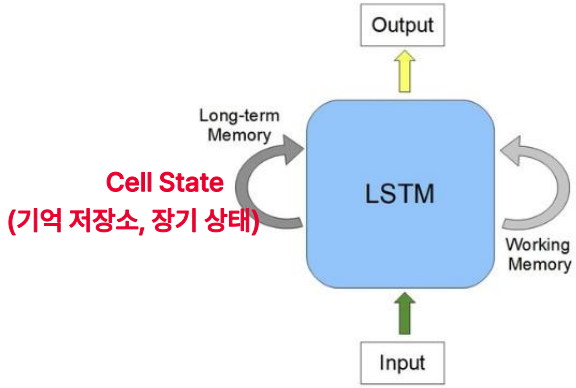
하나의 층 안의 모든 시점에서 동일한 가중치 값으로 공유
➔학습에 필요한 가중치 수를 줄일 수 있음
➔흐름에 따른 연관성을 고려하여 예측할 수 있음
(※ Hidden Layer가 2개 이상인 경우, Layer 내에서 동일하나 각각은 다름)

- Y_t : 현재 상태에서 최종 출력을 만듦
- h_t : 이전 상태와 현재 입력을 결합해 새로운 상태를 만듦

SUMMARY

RNN vs LSTM

RNN은 짧은 기억력, LSTM은 긴 기억력으로 과거 정보를 더 잘 활용하도록 설계된 신경망

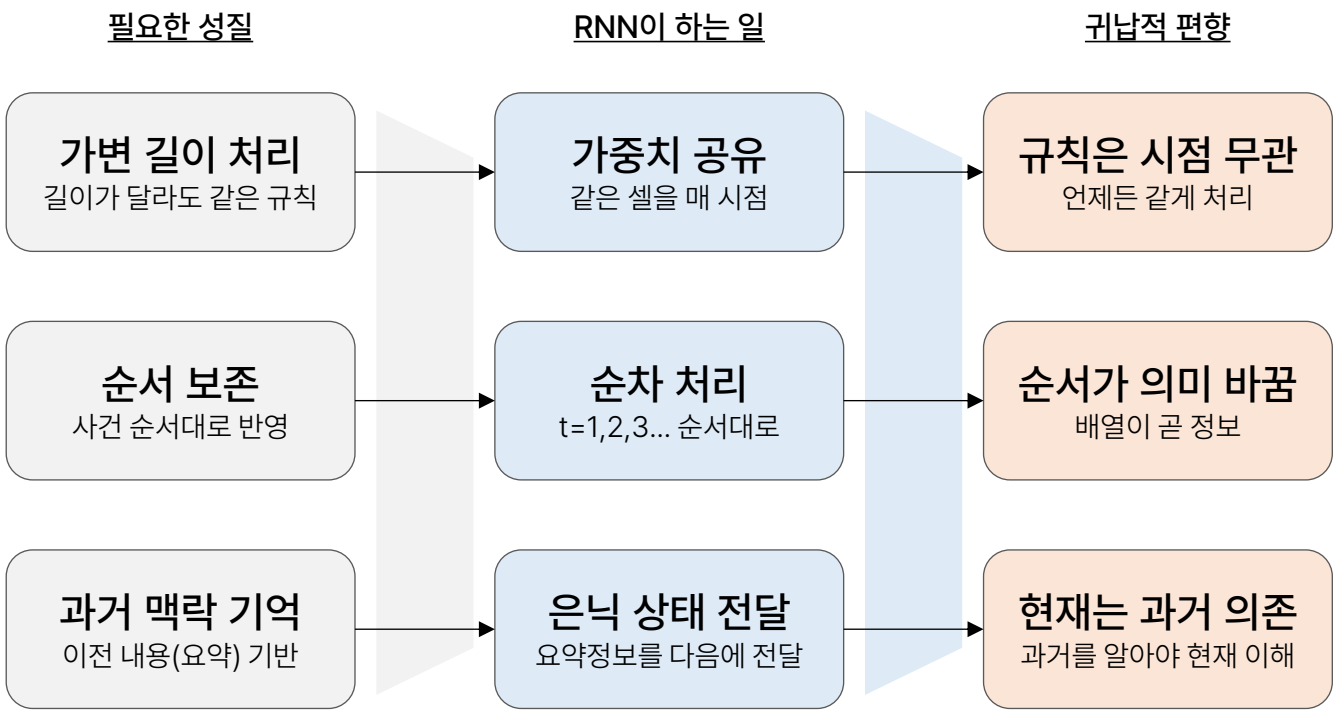
구분	RNN	LSTM
기억 능력	• 과거 정보를 짧게 기억	• RNN 보다는 과거 정보를 길게 기억 가능
구조	 Hidden State (현재 상태) • 현재 상태와 입력값을 받아 Hidden State 통해 출력 • tanh 함수로 단순 계산	 Cell State (기억 저장소, 장기 상태) Hidden State (현재 상태) • Cell Satet(기억 저장소)와 Gate를 활용하여 상태 출력 • Forget/Input/Output Gate로 정보 조절 → 계산량이 많고, RNN 보다 학습 시간 길어짐
한계점	• 어느 정보 과거를 기억하지만, 긴 시퀀스 학습 어려움 (Vanishing Gradient) • 시간 순서 의존 → 연산 속도 느림 (병렬처리 한계)	• 계산량 많고, 긴 시퀀스에서 목적을 달성하기 어려움 • 시간 순서 의존 → 연산 속도 느림 (병렬처리 한계)
설명	• 순간순간 정보를 이어가는 방식	• 순간순간 정보를 이어가는 방식에 더해서 • 중요한 정보를 따로 저장하는 기억 저장소(Cell) 통해 과거 중요한 정보를 오래 활용할 수 있도록 함

Source : https://www.researchgate.net/figure/RNN-v-s-LSTM-a-RNNs-use-their-internal-state-memory-to-process-sequences-of-inputs_fig1_341131167

SUMMARY

2. RNN

순서가 의미를 만들고 현재는 과거에 의존하며, 규칙은 시점과 무관함



그래서 학습 파라미터가 시점 수만큼 늘지 않음
→ 파라미터 절약 + 가변 길이 동시 해결

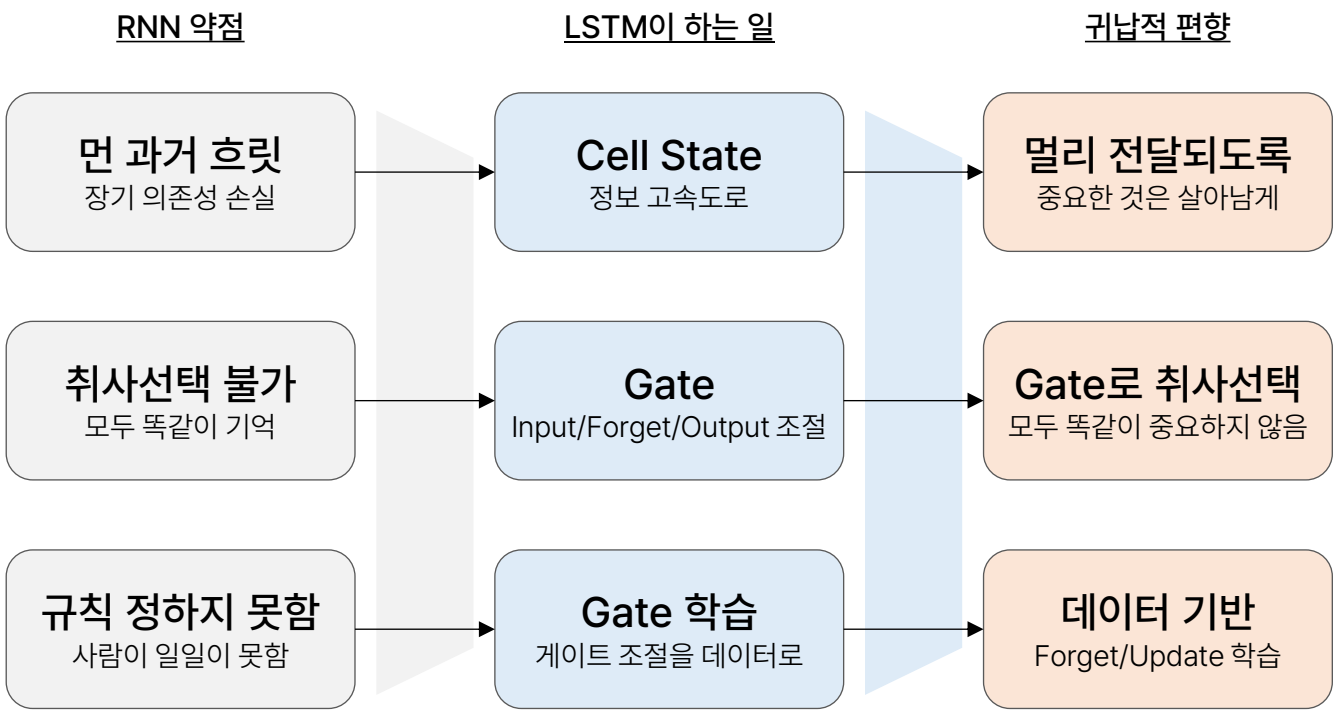
단어가 같아도 순서가 다르면 뜻이 다름 (의미 명확)
(e.g.) 사람이 사과를 먹었다 ≠ 사과가 사람을 먹었다

앞 문장을 기억해야 현재 문장을 이해할 수 있음
(e.g.) 그것이 뜻하는 바는..

SUMMARY

3. LSTM

RNN 골격을 그대로 두고, 무엇을 얼마나 오래 기억하고 무엇을 잊을지를 데이터가 스스로 정하게 하는 장치를 더한 아키텍처



Hidden State와 별개로, 고속도로 같은 경로(Cell State)를 두어 정보가 멀리까지 살아남도록

각각의 Gate가 이번 정보를 얼마나 받아들일지, 과거를 얼마나 잊을지, 무엇을 내보낼지를 학습하여 조절

Gate는 고정 규칙이 아니라 학습되는 값으로, 문제에 맞는 기억 전략을 스스로 익힘

AI Campus 데이터분석 및 AIOps > Part C – DL Architecture

2. 표현 학습이라는 다른 길

정답 없이 배우다

- 지도 학습의 한계
- Auto-Encoder – 압축과 복원 통한 표현 학습
- Seq2Seq

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

1. 지도 학습의 한계

Supervised는 입력 마다 "정답"이 있어야 학습이 가능함

Label 한계점

비용
사람이 일일이, 전문가는 더 비싼 비용 필요

애매한 문제
무엇이 "정답"인지 명확하게 정의하기 힘듦

레이블 이외 정보 손실
레이블로 구분되지 못한 데이터는 버려짐

그래서 필요한 발상

***정답 없이**, 데이터에서 **스스로** 학습하자*

입력 자신을 정답으로 삼자
(Unsupervised, Self-Supervised)

2. Auto-Encoder

- Auto : 자동으로
- Encoder : 데이터를 압축 (특징만 남김)
- Decoder : 압축된 데이터를 원래대로 복원

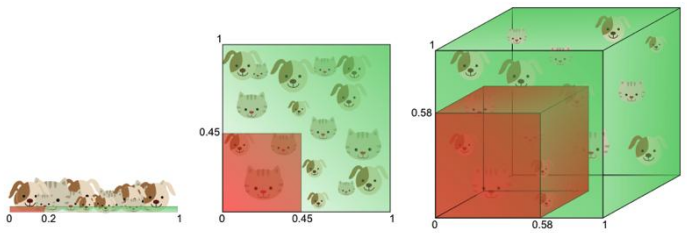
➔ 데이터를 자동으로 압축했다가, 다시 복원하는 신경망

➔ 데이터를 압축하고 복원하는 과정을 스스로 학습하는 신경망

2-(1). AE 필요성

Data Size

- 기본적으로 벡터로 표현되는데, 비정형 데이터는 더 많은 데이터로 표현됨
  (eg, 28*28 흑백 이미지 = 784개 숫자로 표현)
- 딥러닝 네트워크에 적용하기 위해서는 차원이 필요
- 차원이 커지면 데이터 크기가 커지고, 더불어 더 많은 데이터가 요구됨
- '차원의 저주'로 연결될 수 있음 : 차원이 증가하면서 학습데이터 수가 차원수보다 적어져서 성능이 저하되는 현상



데이터에서 중요한 특징만 뽑아서, 적은 숫자로 표현할 수 없을까?

기존 알고리즘 한계

- 기존 알고리즘 중 차원을 축소하여 분석하는 PCA (주성분 분석) 방법 있음
- PCA는 대체적으로 직선 관계(선형)에 적합 : 분산이 가장 큰 방향으로 데이터를 선형적으로 압축

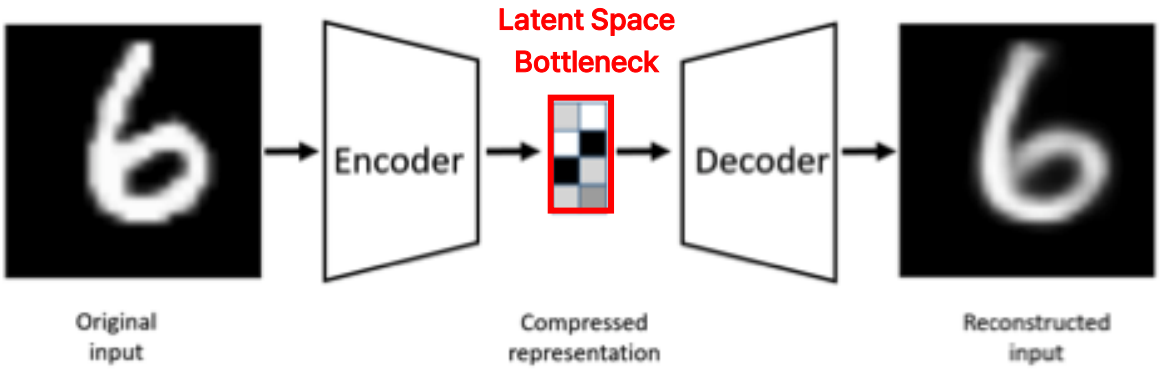
비선형 데이터에서도 잘 압축하는 방법은?

Auto-Encoder는...

- 입력 → 압축(Encoder) → 복원(Decoder) → 출력
- 입력과 출력이 최대한 같아지도록 학습
- 이 과정에서 중간에 중요한 특징만 남게 되는 압축을 학습시키는 것이 핵심!!

Source : https://velog.io/@chiroya/22-차원의-저주

2-(2). AE Architecture



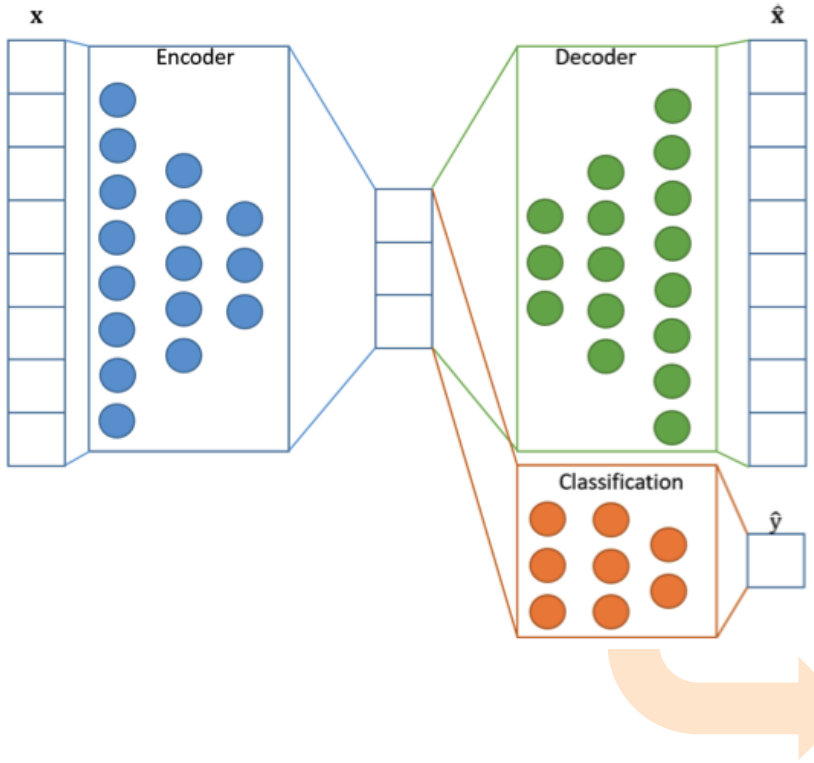
- Encoder : 데이터를 점점 줄여서 압축함 → 중요한 특징만 남김
- **Latent Space** : 가장 압축된 형태 = 가장 작은 차원으로 특징 표현
- Decoder : 압축된 정보를 바탕으로 다시 원본처럼 복원

중요한 특징만 남기고 불필요한 정보를 줄이는 구조

Source : https://arxiv.org/pdf/2003.05991

2-(2). AE Architecture - 확장

Architecture for Classification



Auto-Encoder는...

단순한 압축 도구가 아니라, " 데이터의 특징"에 집중한 네트워크

"압축된 특징이 결국 중요한 정보"

Latent Space에서 Classification(분류기)로도 보낼 수 있음

← **압축을 통해 특징을 잘 뽑아내었다면,**
해당 특징으로 Classification 문제도 잘 해결할 수 있음

Source : https://arxiv.org/pdf/2003.05991

2-(3). Vanilla AE 한계점

Vanilla AE는 복원은 수행하지만, 좋은 표현·생성·일반화를 보장하지 못함

▪ 압축 표현(latent)이 "쓸모"를 보장하지 않음

- AE 학습 목표는 복원 → 무엇을 잘 복원 했는지로 볼지에 따라 표현이 달라짐
- 복원이 분류, 검색 같은 과제에 유용하다는 보장이 없음
- '복원이 잘 됨 = 표현이 좋음' 은 성립하지 않음

▪ Latent 민감성

- Latent 공간이 불규칙 : 임의의 데이터를 추가해도, 의미 있는 출력이 나온다는 보장 없음 → 생성 모델로 사용할 수 없음
- Hyper-para 민감성 : Latent 차원을 너무 크게 잡으면 복사, 너무 작게 잡으면 정보가 과하게 버려짐
  → 적절한 압축 정도를 찾아야 하지만 정답이 없어 검증이 까다로움

▪ 분포 밖 입력에 취약

- 학습 데이터의 분포를 복원하도록 → 학습 때 보지 못한 종류의 입력은 잘 복원하지 못함
- Anomaly Detection 활용되기도 하지만, 학습 분포를 벗어나면 신뢰할 수 없는 한계점도 있음 – 동전의 양면

2-(4). AE 확장

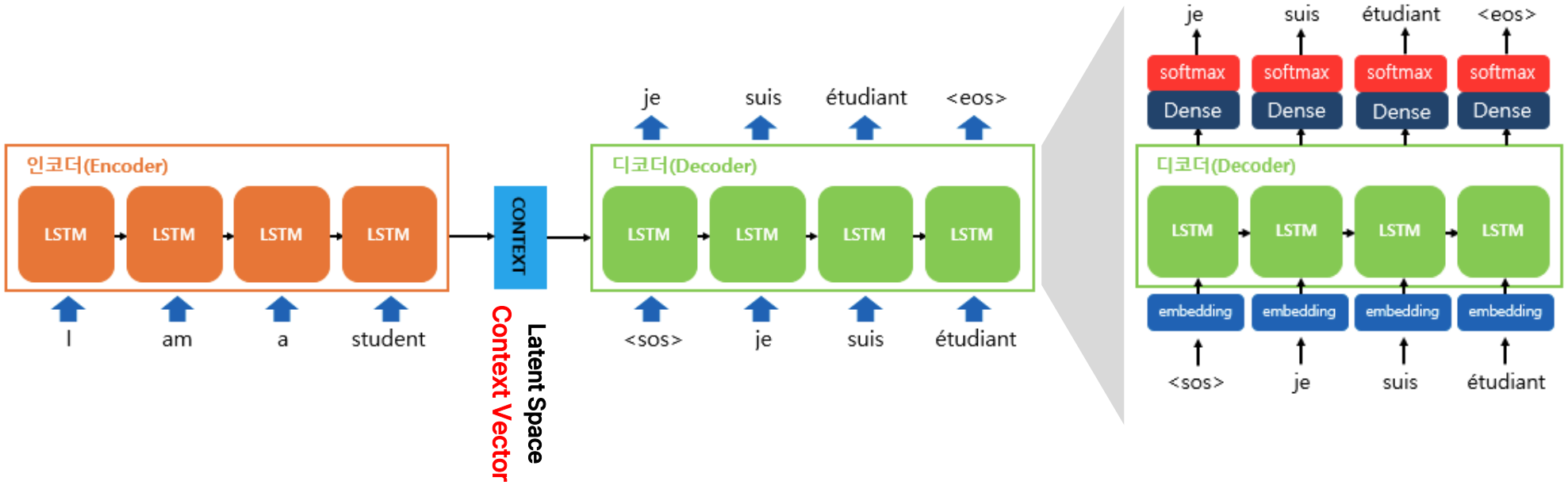
다양한 확장 및 발전 : 그냥 압축기가 아님!!!

Autoencoder 종류	주요 특징	활용	비고
Vanilla AE	• 기본형 • 입력을 압축했다가 복원	• 차원 축소 • 특징 추출	• 완전연결층(MLP)로 구성되는 경우 많음
Denoising AE	• 일부러 입력에 노이즈 추가 • 좀 더 깨끗한 데이터로 복원	• 노이즈 제거 • 강인한 특징 학습 (Robust Features/Representations)	• 입력에 손상을 주어도 본질적인 특징을 뽑도록 학습
Sparse AE	• Latent Space에 희소성 제약 • 활성 뉴런수 제한	• 중요 특징만 뽑아내는 압축 강조	• L1 정규화 등으로 활성 노드수 줄임
VAE (Variational AE)	• Encoder 출력이 작은 변화에도 민감하지 않도록 제약	• 견고한 특징 학습 (Robust)	• 작은 변화 (노이즈)에 덜 민감하게 개선 • 단순 압축을 넘어 데이터 생성까지 가능
Convolutional AE	• CNN 기반 AE • 공간적 구조(이미지 특징) 반영	• 이미지 압축 • 이미지 노이즈 제거	• CNN 네트워크 사용하여 이미지 데이터에도 적합하도록 개선
Seq2Seq	• 시계열 데이터, 순차적 데이터에도 처리되도록 반영	• 시계열 데이터의 특징 압축 • 시계열 데이터의 패턴 학습	• RNN, LSTM, GRU 등 사용되도록 개선

3. Seq2Seq

AE는 입력을 그대로 복원한다면, Seq2Seq는 입력을 다른 시퀀스로 변환하는 네트워크

- 입력 : 시간에 따라 변하는 시계열 데이터, 문장(자연어) 등
- 출력 : 입력과 길이가 다를 수도 있는 새로운 시퀀스 (순서가 있는 데이터). 주로 기계 번역, 텍스트 요약, 챗봇(초기 버전) 등에 활용
- Encoder의 마지막 Hidden State 하나에 입력 전체를 압축하여, 고정 길이 병목으로 인한 정보 손실 발생



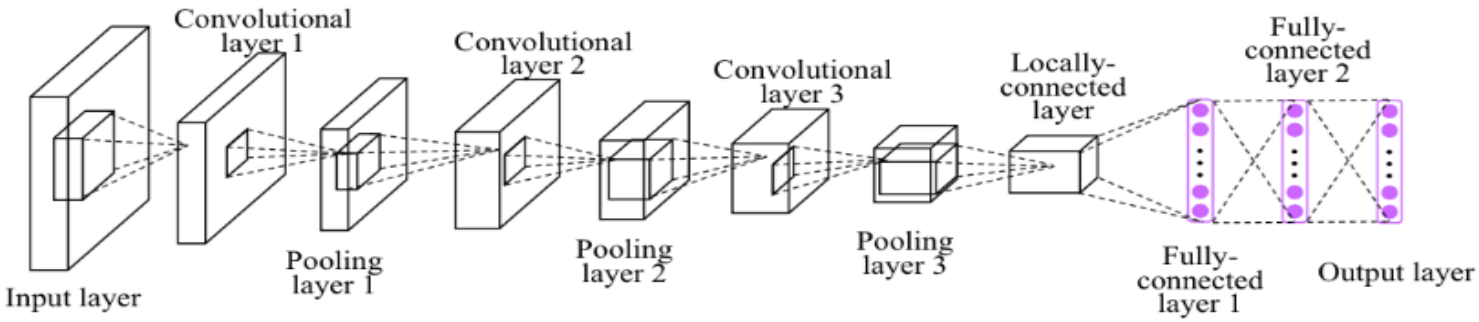
Source : https://wikidocs.net/24996

Part-C. DL Architecture (1990~2015)

SUMMARY

CNN

이미지 특징을 뽑아내고(Conv), 중요한 것만 추려내고(Pooling), 펼쳐서(FC), 최종 분류(Dense)



Convolution Neural Network

Convolution Layer

- 3차원 이미지 정보에서 **이미지 특징(feature map)을 뽑아내는 계층**
- **이미지와 필터 간의 연산**. 원본 이미지 변형 없음

합성곱 연산 (필터 연산)

- Input featrue map에서 **filter window 만큼** 적용하여 연산
- 연산은 window 크기만큼 유지하면서 **stride sliding** 반복 수행

Padding

- 합성곱 연산 결과 변형되는 **output size 보정**
- **Intput** feature map에 특정값(주로 0)을 채움

Pooling Layer

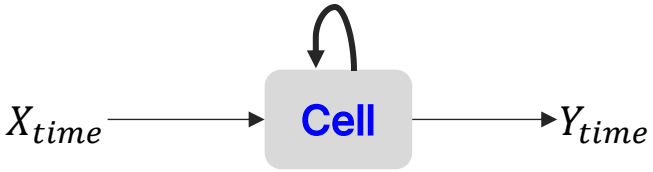
- Convolution Layer의 합성곱 연산 결과에서 **추가적으로 공간을 줄이는 연산**
- **Max-pooling** 주로 사용됨. 학습 Parameter 없음

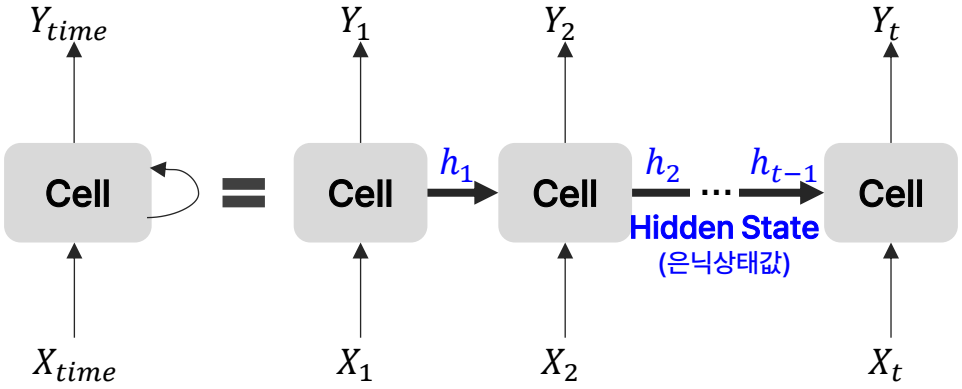
RNN

현재 입력 뿐만 아니라 이전 시점의 상태(Hidden State)를 기억하여 시간에 따른 데이터 흐름과 패턴을 학습하는 신경망

Cell, Memory Cell

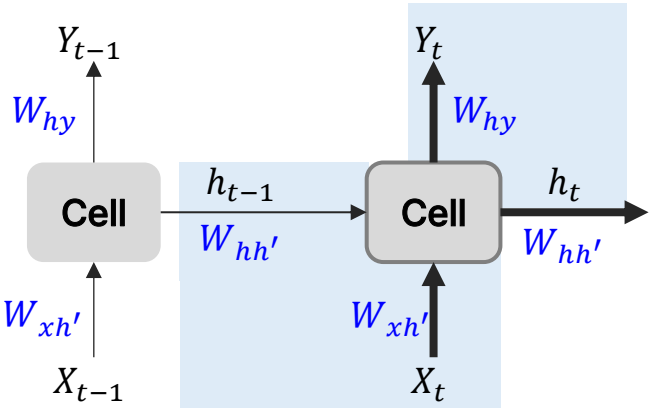
- Hidden Layer에서 Activation Function 통해 결과를 내보내는 역할
- 이전의 값을 기억하려고 하는 일종의 메모리 역할을 수행하는 노드 (내부 연산)
- 각각의 시점에서 바로 이전 시점에서의 출력값을 자신의 입력으로 사용





$$Y_t = W_{hy}h_t + b_y$$

$$h_t = \tanh(W_{hh'}h_{t-1} + W_{xh'}X_t + b_h)$$

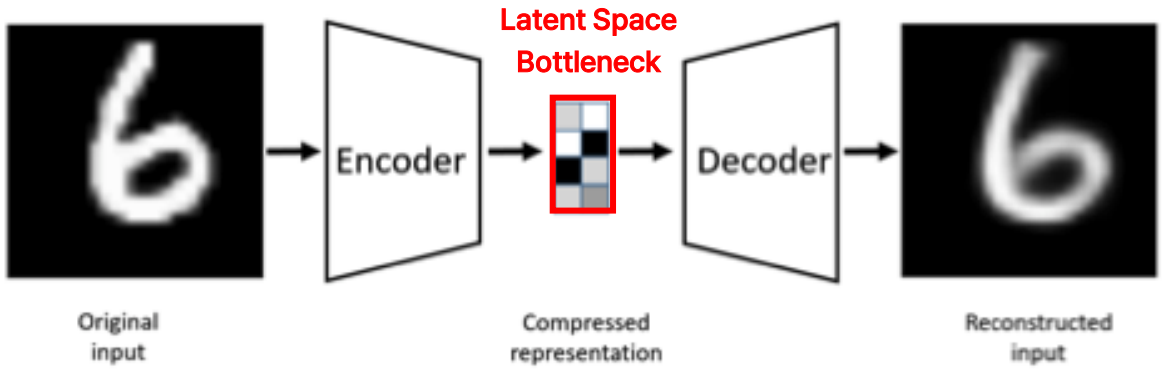


하나의 층 안의 모든 시점에서 **동일한 가중치 값**으로 공유
➔학습에 필요한 가중치 수를 줄일 수 있음
➔흐름에 따른 연관성을 고려하여 예측할 수 있음
(※ Hidden Layer가 2개 이상인 경우, Layer 내에서 동일하나 각각은 다름)

- Y_t : 현재 상태에서 최종 출력을 만듦
- h_t : 이전 상태와 현재 입력을 결합해 새로운 상태를 만듦

Auto-Encoder

Architecture



- Encoder : 데이터를 점점 줄여서 압축함 → 중요한 특징만 남김
- **Latent Space** : 가장 압축된 형태 = 가장 작은 차원으로 특징 표현
- Decoder : 압축된 정보를 바탕으로 다시 원본처럼 복원

→ 중요한 특징만 남기고 불필요한 정보를 줄이는 구조

→ 단순한 압축 도구가 아니라, " 데이터의 특징"에 집중한 네트워크, **"압축된 특징이 결국 중요한 정보"**

Source : https://arxiv.org/pdf/2003.05991

AI Campus 데이터분석 및 AIOps > Part C – DL Architecture

3. CNN의 진화

더 깊게, 더 안정적으로

- AlexNet – 딥러닝 부흥과 GPU 학습
- ResNet – residual connection과 깊이의 한계 돌파

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

DL Architecture

Expansion History

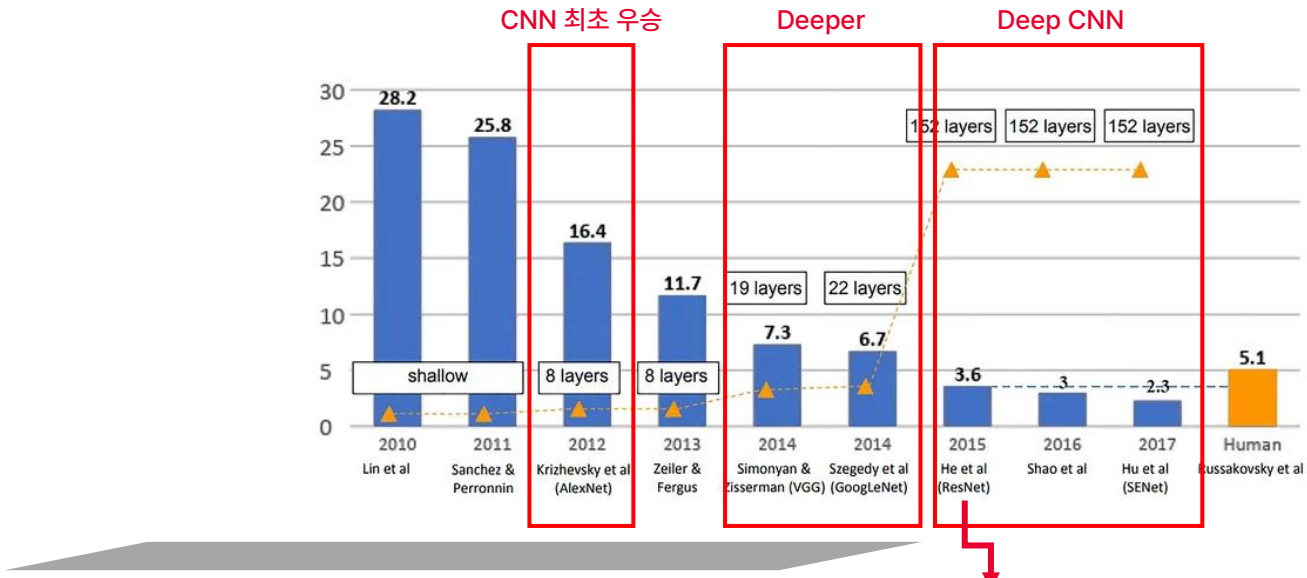
CNN-based Architecture

COMPUTER VISION		
YEAR	**Architecture**	**Contribution**
1990	**CNN**	• 이미지 처리에 강점을 가진 신경망 • 합성곱 필터로 이미지 특징 자동 추출하는 방법론 제시
1998	LeNet-5	• 초기 CNN 중 하나 • MNIST 위해 설계, 현대 CNN의 기초 다짐
2012	**AlexNet**	• GPU 기반 ReLU, Dropout 사용한 Large CNNs • ImageNet에서 혁신적인 성과 & 딥러닝 대중화
2014	VGGNet	• 작은 필터(3*3)로 아키텍처 단순화 • 객세인식 성능 향상
2014	GoogleNet (Inception)	• Inception 모듈 도입 • 계산 자원 효율화, 높은 정확도 유지
2015	**ResNet**	• Residual Learning으로 Deep Network(최대 152층) • 기울기 소실 문제 해결
2017	DenseNet	• 각 층을 모든 다른 층과 연결 • 파라미터 수를 줄이면서 정확도 향상
2017	YOLO You Only Look Once	• 실시간 객체 탐지 • 이미지 내 객체를 빠르고 효율적으로 탐지
2018	EfficientNet	• 네트워크 깊이, 폭, 해상도를 균형있게 조정 • 적은 파라미터로 성능 극대화
2021	**ViT** Vision Transformer	• Transformer를 이미지 분류에 적용한 첫 사례로 전통적은 CNN 성능 능가 (2~5%) • Vision Task에서도 효과적일 수 있음을 입증

TRANSFORMERs

NLP		
YEAR	**Architecture**	**Contribution**
1985	**RNN**	• 시퀀스 데이터를 처리할 수 있는 첫번째 신경망 • 순차적 데이터 처리에 탁월한 성능 발휘
1997	LSTM Long Short-Term Moemry	• RNN의 기울기 소실 문제 해결 • 언어/음성과 같은 시퀀스에서 뛰어난 성능 발휘
2014	Word2Vec	• 단어 임베딩 혁신 • 단어의 의미지 관계를 연속적인 벡터로 학습
2015	Seq2Seq	• 인코더-디코더 확장 • 기계번역 및 순차 데이터 작업에서 성능 향상
2017	**Transformer**	• Self-Attention 도입, RNN/LSTM 대체 • 병렬 처리와 함께 뛰어난 성능 구현
2018	**BERT**	• Transformer 디코더 구조 확장 (Bidirectional) • Pre-training, Transfer Learning
2019	GPT-2	• Transformer 디코더 구조 확장 (Unidirectional) • 문맥에 맞는 텍스트 생성 가능
2020	GPT-3	• 1750억개 파라미터 가진 대형 Transformer • 미세조정 없이 다양한 NLP Task 수행
2021	DALL-E	• 텍스트 설명으로 이미지 생성 • Cross-Modal, Multi-Modal
2021	CLIP	• 이미지와 텍스트 데이터 쌍을 학습 • 비전 작업에서 Zero Shot 학습 가능하게 함
2023	**GPT-4**	• GPT-3 후속버전 • 향상된 성능으로 깊이 있는 문맥 이해와 자연스러운 텍스트 생성 가능

ImageNet Challenge



- AlexNet : 8개 ← CNN 최초 우승
- VGG : 19개
- GoogLeNet : 22개
  - 대전제 : 레이어를 추가할수록 성능이 오른다
    그런데 레이어를 추가하니 막상 성능이 감소한다
  - Gradient Vanishing 아님. Batch Norm, ReLU 등으로도 해결 안됨

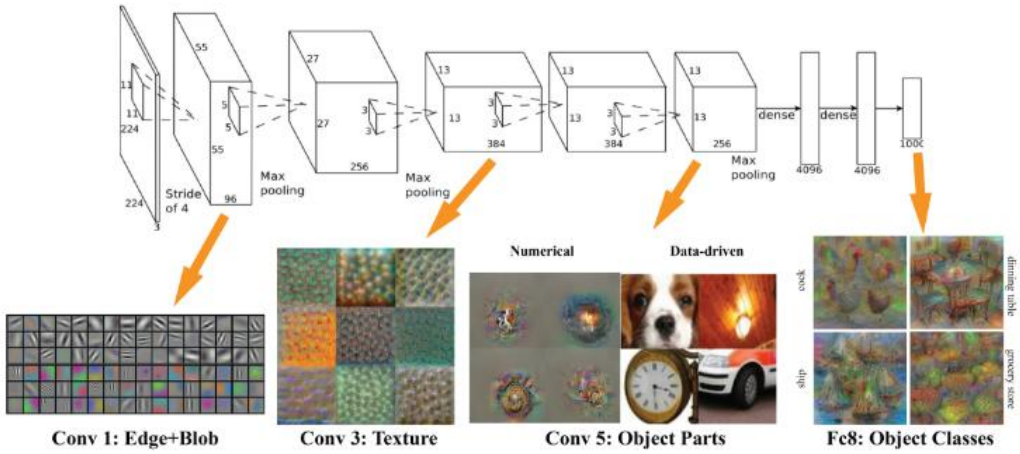
  → Degradation, 깊이의 저주, 퇴화

- ResNet : 152개
  - 대전제 증명 : 깊이 쌓으면 성능이 오른다
  - Deep Neural Network (DNN) 시대의 서막
  - 현대 DNN 모델들의 표준

Source : https://www.researchgate.net/figure/Comparison-between-of-ILSVRC-winners-and-human-error-rate-in-classification-task-55_fig1_353956374

1. AlexNet

ILSVRC 2012. ImageNet Large Scale Visual Recognition Challenge



AlexNet 주요 특징:

- 대형 CNN 구조
  - 5개의 Conv Layer, 3개의 FC Layer로 구성
  - 당시 기준 매우 깊은 대형 CNN 구조
- ReLU 도입
  - ReLU를 도입하여 비선형성 강화, 학습속도 향상
  - 기울기 소실 문제 해결하는데 탁월한 성능
- Dropout
  - Overfitting 해결
  - 학습 과정에서 임의로 일부 뉴런을 비활성화시켜 특정 뉴런에 지나치게 의존하지 않도록
- GPU 사용
  - 대규로 데이터셋을 빠르게 학습하기 위해 사용
- Data Augmentation, Normalization
  - 데이터 증강 기법으로 더 많은 학습 데이터를 생성하고, 학습 성능 향상
  - LRN(Local Response Normalization) 이라는 정규화 기법을 적용하여 CNN이 더 잘 학습되도록 처리

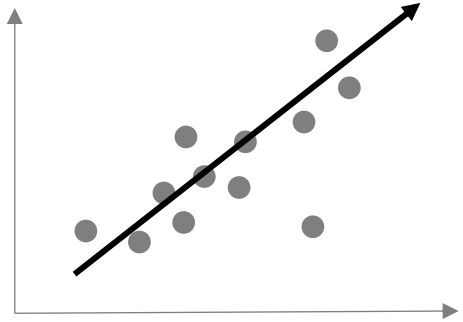
딥러닝이 이미지 처리와 인식 문제에서
탁월한 성능을 발휘할 수 있음을 증명

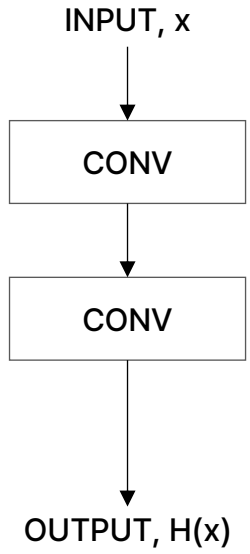
Source : https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

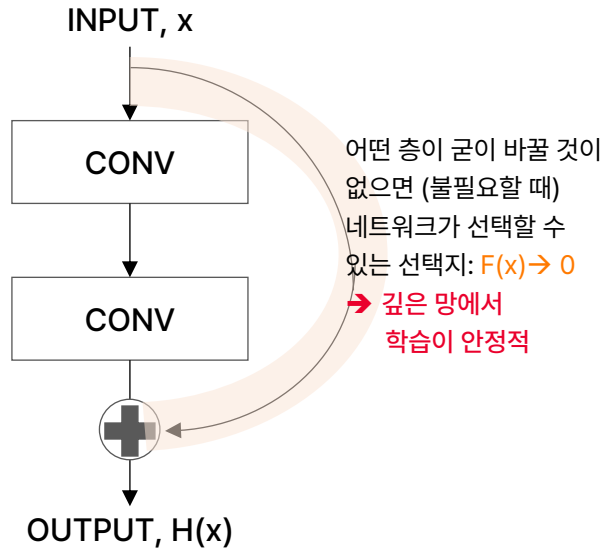
2. ResNet

잔차의 재해석

구분	통계 회귀분석	Vanilla CNN	ResNet
잔차	• 잔차 = 실제값 – 예측값 • 모델 전체 출력 vs 정답 • 모델이 틀린 양 (학습 결과)	• 잔차 = 실제값 – 예측값 • 모델 전체 출력 vs 정답 • 출력을 통째로 학습	• 잔차 = 출력 – 입력 • 블록 출력 vs 블록 입력 • 입력에 더할 변화량 (학습 대상)
학습 대상	• 회귀 계수	• 목표 출력, H(x)	• 변화량, F(x) = H(x) - x
잔차 역할	• 줄여야 할 오차 (결과)	• 줄여야 할 오차 (결과)	• 학습하는 목표 – 변화량
잔차 = 0	• 모델이 데이터를 완벽히 설명 = 학습 성공	• 출력이 정답과 일치 = 학습 성공	• 그 층이 아무것도 안함 = 학습 성공과 무관
핵심 장치	• 최소제곱법	• Conv Layer 쌓기	• Skip Connection (입력을 출력에 더함)





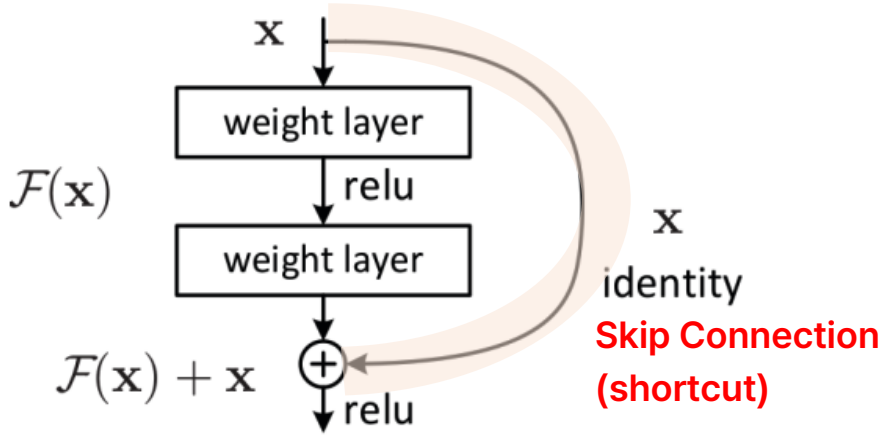


2. ResNet

Residual Block : 지름길 통해 각 블록의 입력 흐름을 보존하고, 필요한 변화량만 더해가며 정보를 점진적으로 정제하는 딥 네트워크

- 성능 향상을 위해 신경망 깊이를 늘리면 학습 오차마저 커지는 현상 – 깊어지면 성능이 나빠진다 (degradation)
- Residual Block = 각 블록의 입력을 그대로 넘겨주자
- 특정 블록이 유용한 변환을 학습하지 못하더라도 입력이 보존되어 최소한 성능이 나빠지지 않도록 안전장치 마련

  ➔ 필요한 것만 정교하게 학습하겠다 : 입력은 skip으로 보존되니, 각 블록은 바꿀 부분(F(x))만 학습하면 된다

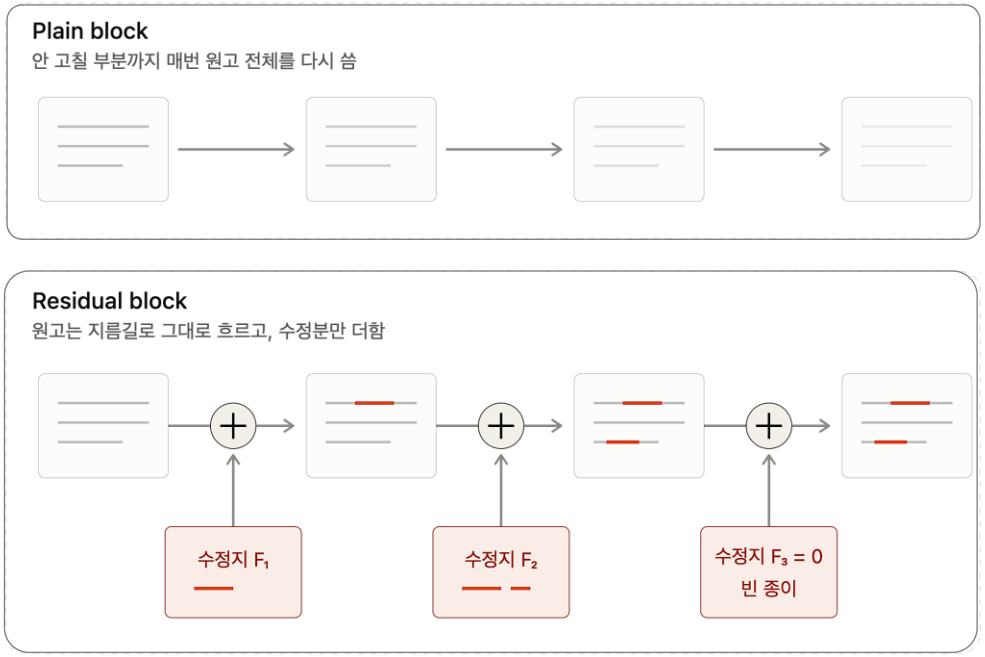


Source : https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

[참고] Residual Block, 쉽게 이해하기

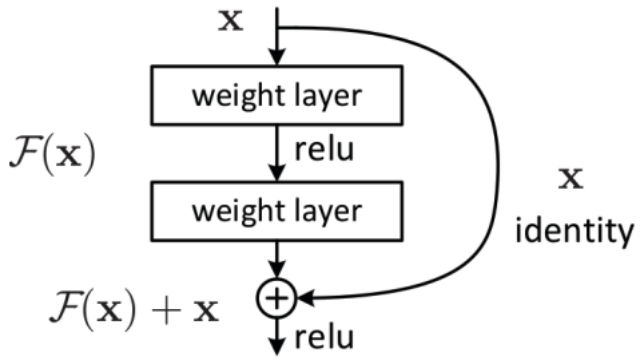
원고 편집에 비유해 보면...

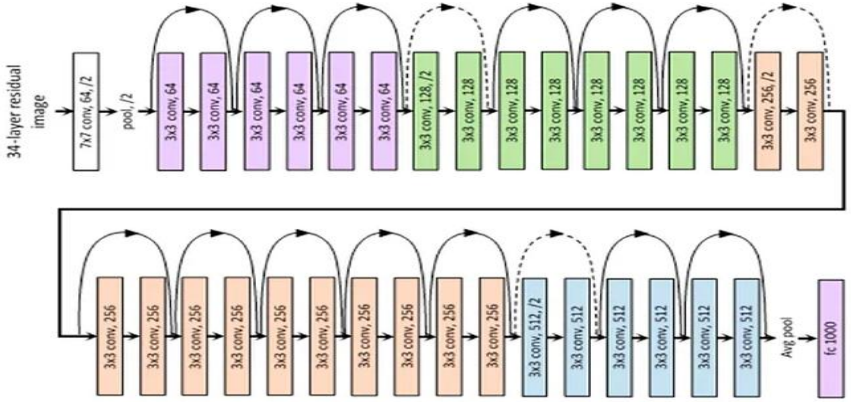
- Plain block : 각 단계마다 원고를 백지에 다시 옮겨 씀. 안 고칠 부분까지 전부 다시 작성 → 옮기는 과정에서 손실 발생 가능
- Residual block : 원본 위에 수정사항만 표시. 고칠 것이 없으면 추가 작업 없음 → 그 단계에 들어온 원고가 그대로 남음
- 편집자는 무엇을 고쳐야 할지 미리 아는 것이 아님. 최종 원고에 대한 평가가 역방향으로 전달되면서 각자 무엇을 쓸지 정해짐.



2. ResNet

CNN 기반으로 Residual Block 제안, 층이 깊어지더라도 과적합이나 기울기 소실 문제 해결





ResNet 주요 특징:

- **Residual Block**
  - 레이어를 깊게 쌓으면 성능이 좋아질 수 있지만 기울기 소실 문제 발생
  - 각 층의 출력을 다음 층으로 직접 전달하는 Skip Connection을 추가하여 문제 해결
  - 기존 정보 유지 + 더 필요한 것만 추가
- 네트워크 안정성
  - 100층 이상의 매우 깊은 네트워크도 학습 가능
    (최대 152층까지 학습할 수 있도록 설계)
  - 깊은 네트워크지만 상대적으로 적은 파라미터로 높은 성능 발휘
  - Residual Learning으로 깊은 네트워크에서도 안정적으로 학습 가능
- 다양한 변형
  - ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-152, ...
  - 더 깊은 모델은 더 복잡한 데이터셋 처리하는데 유용
  - 컴퓨터 비전 외에도 NLP, Speech Recognition 등 다양한 딥러닝 응용 분야에서 활용

딥러닝에서의 네트워크 깊이 한계 극복 &
Computer Vision 표준 아키텍처로 자리매김

Source : https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

4. Attention 등장

순차 처리의 병목을 넘다

- Transformer – self-attention과 병렬화, 그리고 $O(n^2)$ 비용
- BERT – 양방향 사전학습과 전이학습

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

DL Architecture

Expansion History

CNN-based Architecture

COMPUTER VISION		
YEAR	**Architecture**	**Contribution**
1990	**CNN**	• 이미지 처리에 강점을 가진 신경망 • 합성곱 필터로 이미지 특징 자동 추출하는 방법론 제시
1998	LeNet-5	• 초기 CNN 중 하나 • MNIST 위해 설계, 현대 CNN의 기초 다짐
2012	**AlexNet**	• GPU 기반 ReLU, Dropout 사용한 Large CNNs • ImageNet에서 혁신적인 성과 & 딥러닝 대중화
2014	VGGNet	• 작은 필터(3*3)로 아키텍처 단순화 • 객세인식 성능 향상
2014	GoogleNet (Inception)	• Inception 모듈 도입 • 계산 자원 효율화, 높은 정확도 유지
2015	**ResNet**	• Residual Learning으로 Deep Network(최대 152층) • 기울기 소실 문제 해결
2017	DenseNet	• 각 층을 모든 다른 층과 연결 • 파라미터 수를 줄이면서 정확도 향상
2017	YOLO You Only Look Once	• 실시간 객체 탐지 • 이미지 내 객체를 빠르고 효율적으로 탐지
2018	EfficientNet	• 네트워크 깊이, 폭, 해상도를 균형있게 조정 • 적은 파라미터로 성능 극대화
2021	**ViT** Vision Transformer	• Transformer를 이미지 분류에 적용한 첫 사례로 전통적은 CNN 성능 능가 (2~5%) • Vision Task에서도 효과적일 수 있음을 입증

TRANSFORMERs

NLP		
YEAR	**Architecture**	**Contribution**
1985	**RNN**	• 시퀀스 데이터를 처리할 수 있는 첫번째 신경망 • 순차적 데이터 처리에 탁월한 성능 발휘
1997	LSTM Long Short-Term Moemry	• RNN의 기울기 소실 문제 해결 • 언어/음성과 같은 시퀀스에서 뛰어난 성능 발휘
2014	Word2Vec	• 단어 임베딩 혁신 • 단어의 의미지 관계를 연속적인 벡터로 학습
2015	Seq2Seq	• 인코더-디코더 확장 • 기계번역 및 순차 데이터 작업에서 성능 향상
2017	**Transformer**	• Self-Attention 도입, RNN/LSTM 대체 • 병렬 처리와 함께 뛰어난 성능 구현
2018	**BERT**	• Transformer 디코더 구조 확장 (Bidirectional) • Pre-training, Transfer Learning
2019	GPT-2	• Transformer 디코더 구조 확장 (Unidirectional) • 문맥에 맞는 텍스트 생성 가능
2020	GPT-3	• 1750억개 파라미터 가진 대형 Transformer • 미세조정 없이 다양한 NLP Task 수행
2021	DALL-E	• 텍스트 설명으로 이미지 생성 • Cross-Modal, Multi-Modal
2021	CLIP	• 이미지와 텍스트 데이터 쌍을 학습 • 비전 작업에서 Zero Shot 학습 가능하게 함
2023	**GPT-4**	• GPT-3 후속버전 • 향상된 성능으로 깊이 있는 문맥 이해와 자연스러운 텍스트 생성 가능

1. Transformer

중요한 것에 집중하는 방식으로, 문장을 더 정확하고 빠르게 이해할 수 있음

- 자연어 처리에서 RNN이나 LSTM을 대체한 BERT, GPT의 근본이 되는 기본 아키텍처
- Self-Attention 매커니즘을 도입하여 긴 문맥을 효율적으로 처리하여 NLP Task 성능을 크게 향상

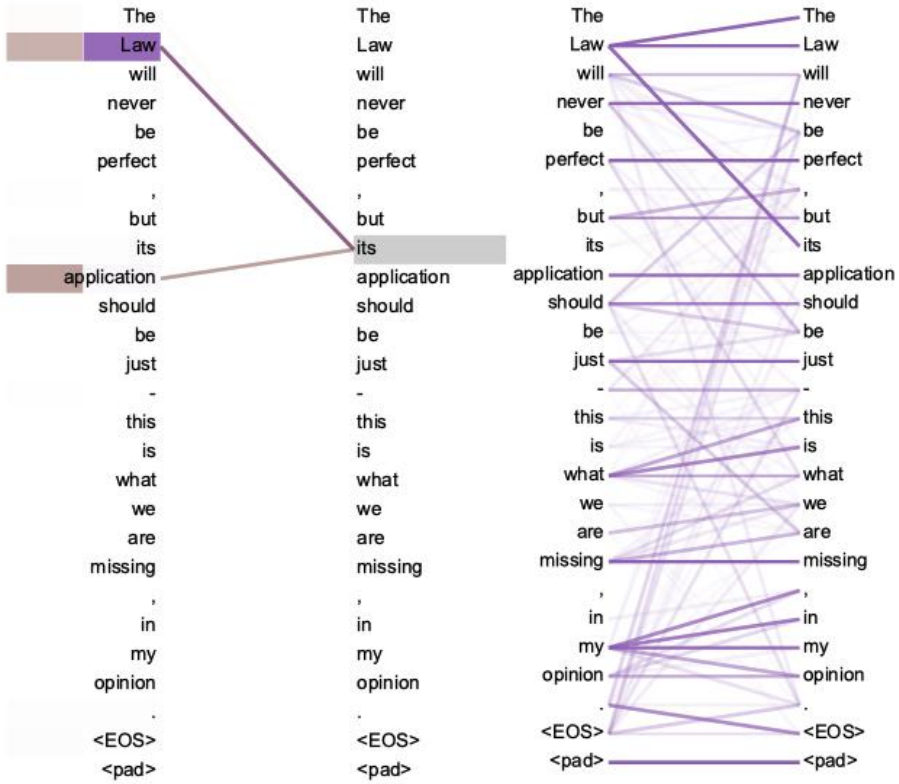
Attention

- 입력 시퀀스(문장)의 각 요소(단어)가 **다른 요소들과 얼마나 중요한지, 어느 부분에 집중할지 결정**하는 매커니즘
- 기계 번역 시 특정 단어를 번역할 때, 문장의 다른 단어들과 얼마나 관련이 있는지를 고려하여 더 정확하게 번역

Self-Attention

- 입력된 문장 내에서 **각 단어가 다른 모든 단어와 관계를 학습**
- 한 단어가 문장 내 다른 단어들과 얼마나 관련 있는지를 계산하여 문맥을 더 잘 이해하도록
- RNN, LSTM이 순차적으로 데이터를 처리하지만, Self-Attention은 문장 내의 모든 단어를 동시에 보고 처리 (동시 = 병렬처리)

Attention 매커니즘을 잘 사용하기 위해 만든 구조,
Transformer



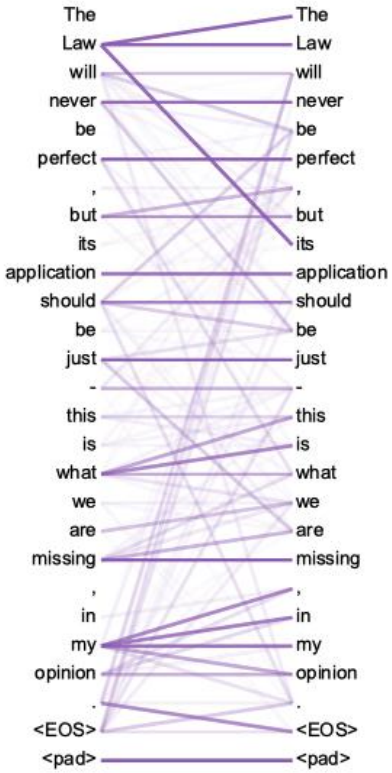
Source : Attention Is All You Need (2017), https://arxiv.org/pdf/1706.03762

2. Attention, 쉽게 이해하기

Key Point

데이터 특징	▪ 기존 RNN, LSTM 한계점 : 문장이 길면 앞의 내용이 잘 전달되지 않음 ➔ 장기 의존성 문제 ▪ 착안 포인트 : 모든 단어를 다 참고하되, 중요한 단어에 더 집중하면 되지 않을까?
Attention	▪ 문장을 읽고 마지막에 정리하는 것이 아니라, 읽으면서 중요한 단어에 형광펜으로 구분 ▪ 중요한 단어는 크게 보고, 덜 중요한 단어는 작게 봄 ▪ 각 단어에 얼마나 집중할지 가중치를 구해서, 중요한 단어일수록 크게 반영 ▪ (e.g.) 고양이가 창문에서 햇살을 받으며 잔다.
Self-Attention	▪ 문장 안에서 각 단어들이 서로 얼마나 중요한지 보는 것 ▪ 각 단어가 문장 속 다른 단어들과 서로 영향 주는 관계를 파악 ▪ (e.g.) 고양이가 창문에서 햇살을 받으며 잔다. → '고양이'와 '잔다' 관계가 강함 → 큰 가중치
Multi-Head Attention	▪ 중요한 관계가 하나만 있는 것은 아니다!! → 여러 개의 관계 ➔ Multi-Head (Head = 하나의 시선) ▪ 문장 내에서 Head를 여러 개 달아서, 각 Head가 다른 관계에서 집중하게 하는 것 → 데이터를 여러 관점에서 보는 것 ▪ (e.g.) 고양이가 창문에서 햇살을 받으며 잔다.
Masked Multi-Head Attention	▪ 미래 단어는 절대 보면 안된다!! → 현재 시점까지만 보고 판단해야 함 ▪ Mask = 가림막 : 현재 단어가 미래 단어에 가중치를 주지 못하도록 처리 ← 미래 단어를 보면 부정 행위 ▪ Multi-Head와 결합 : 각 Head가 미래를 가린 상태에서 다른 관계 분석 ▪ (e.g.) 고양이가 창문에서 햇살을 받으며 잔다. ○ Head1 : 잔다 → 고양이가 (동사가 주어 참조) ○ Head2 : 햇살 → 창문 (햇살이 창문 참조. 반대 반향은 마스크로 차단)

[참고] QKV, 쉽게 이해하기



- Attention은 각 단어가 다른 단어와 "얼마나 관련있는지" 계산하여 가중치를 매기는 것
- 그런데 이걸 어떻게 숫자로 계산할까?

❑ Query
- 질문 - "나와 관련있는 단어가 누구지?" 라고 묻는 역할
- (e.g.) "잔다"가 Query라면 → "누가 자는지" 관련된 단어를 찾으려는 질문
- 현재 단어가 다른 단어들과 얼마나 연관이 있는지를 묻는 값

❑ Key
- 단서 – 각 단어가 "나는 이런 단어야" 라고 스스로 소개하는 역할
- (e.g.) '고양이가'의 Key → "나는 주어야, 잠을 잘 수 있는 대상이야"

※ 문법 분석이 아니라, 학습된 특징 표현 (한국어 조사 – "가")

❑ Value
- 정보 – 그 단어가 실제로 담고 있는 의미/정보 그 자체
- (e.g.) '고양이가'의 value → '잔다'에게 골라졌을 때 넘겨줄 "주어=고양이"라는 내용

❑ 계산 흐름 :
- Query와 Key 비교 → 얼마나 관련있는지 점수(유사도) 계산
- 그 점수를 가중치로 변환 (하나의 Query 기준)
- Value 곱해서 합산

[참고] QKV, 쉽게 이해하기

예문 : "고양이가 창문에서 햇살을 받으며 잔다"

❑ 계산 흐름 :

- Query와 Key 비교 → 얼마나 관련있는지 점수(유사도) 계산

  - 잔다 ↔ 고양이가 • 고양이가 : 90
  - 잔다 ↔ 햇살을 • 햇살을 : 15
  - 잔다 ↔ 창문에서 • 창문에서 : 10
  - … • …

- 그 점수를 가중치로 변환 (하나의 Query 기준)

  ❑ 잔다 (Query) :
  1 = 고양이가 0.55
  + 창문에서 0.05
  + 햇살을 0.05
  + 받으며 0.10
  + 잔다 0.25

Q \ K	고양이가	창문에서	햇살을	받으며	잔다
고양이가	0.30	0.05	0.10	0.10	0.45
창문에서	0.10	0.25	0.35	0.20	0.10
햇살을	0.10	0.25	0.20	0.40	0.05
받으며	0.25	0.10	0.40	0.15	0.10
잔다	0.55	0.05	0.05	0.10	0.25

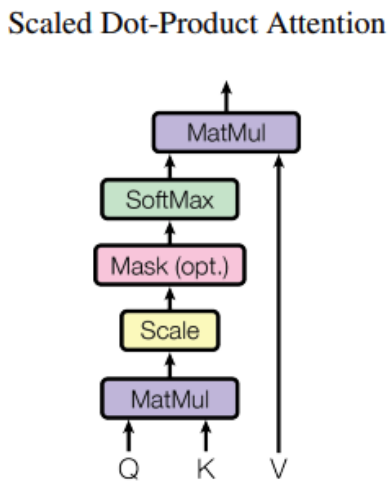
- Value 곱해서 합산

  ❑ 0.55 * (고양이가 value) + 0.05 * (창문에서 value) + 0.05 * (햇살을 value) + 0.10 * (받으며 value) + 0.25 * (잔다 value)
  ← '고양이가'의 정보가 가장 많이 반영됨
  ➔ 이렇게 만들어진 '잔다'의 표현에는 '고양이가'의 정보가 가장 많이 반영
  ➔ '잔다'는 이제 "고양이가 잔다" 라는 문맥을 품은 단어가 됨

2-(1). Scaled Dot-Product Attention

Transformer > Attention > Scaled Dot-Product Attention : 점수를 매겨서 집중하는 방식

- Attention : 중요한 단어에 집중
- Dot-Product : 얼마나 비슷한 방향을 보고 있나? (Product : 단순 곱셈, Dot-Product : 두 벡터의 원소별 곱을 모두 더한 값)
- 계산된 값이 크면 두 단어 간의 관계가 강함을 뜻하므로 좀 더 집중, Attention!!
- Scaled : 벡터 차원이 커지면 dot-product도 커져 softmax가 제대로 작동하지 않음. 이를 해결하기 위해 추가됨



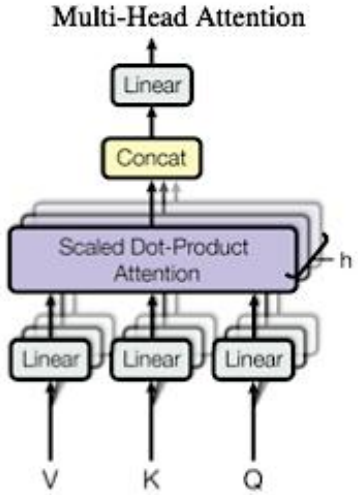
- Query : 현재 단어가 다른 단어들과 얼마나 연관이 있는지를 묻는 값 (질문)
- Key : 각각의 다른 단어들은 어떤 의미를 가지고 있는지 나타내는 값 (단서)
- Value : 해당 단어의 실제 의미를 나타내는 값 (정보)

Source : Attention Is All You Need (2017), https://arxiv.org/pdf/1706.03762

2-(2). Multi-Head Attention

Transformer > Attention > Multi-Head Attention : 병렬적 관계 탐색

- Multi-Head : 여러 개의 Attention 매커니즘이 동시에 작동하는 구조 (병렬 구조: 서로 다른 종류의 관계 포착)
- Linear : 각각의 Head 결과를 합쳐서(Concat) 최종 결과를 만들어 냄



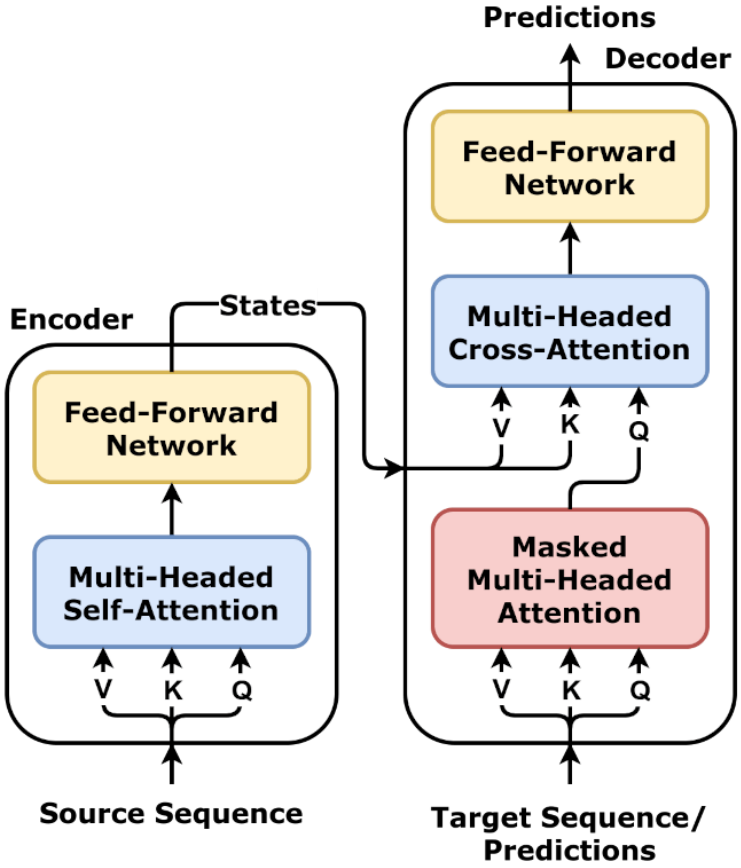
- 각각 다른 QKV 쌍을 통해 여러 질문을 던지고, 그 결과를 모두 모아서 집중해야 하는 단어를 정확하게 찾아내기 위한 방식
- 여러 질문과 다른 관계 유형으로 더 정확한 결과를 확인할 수 있음

- Multi-Head : 여러 헤드가 동시에 작동하여 문장의 의미를 더 풍부하고 정확하게 이해할 수 있도록

$$MultiHead(Q,K,V) = Concat(head_1,,\dots,head_h)W^O$$
$$where\ head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Source : Attention Is All You Need (2017), https://arxiv.org/pdf/1706.03762

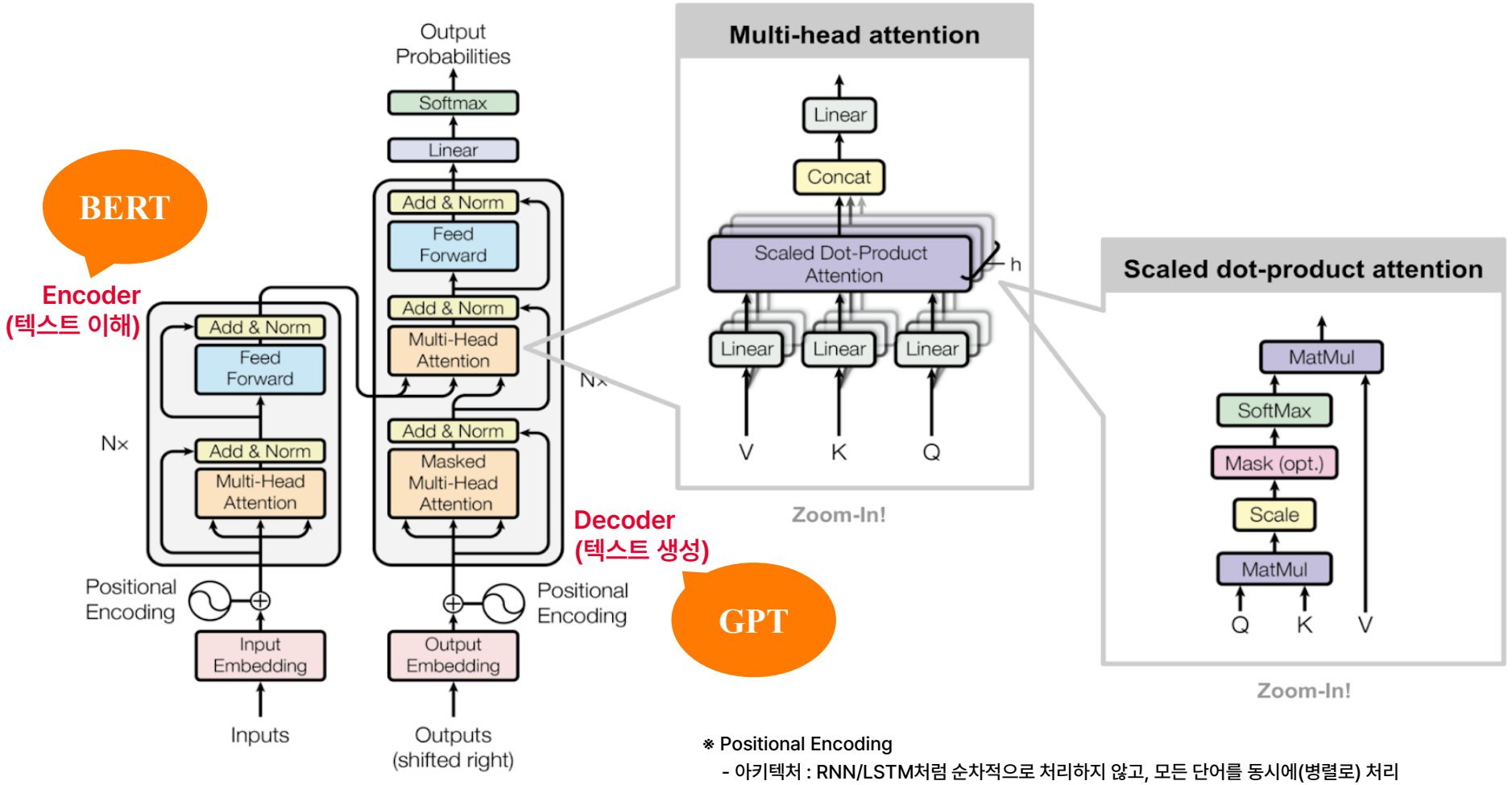
3. Transformer Architecture



Encoder		• 문장을 **이해**하는 부분 • 입력 문장을 이해해서 중요한 정보 추출
	Multi-Headed Self-Attention	• 문장 안에서 단어들끼리 관계를 따져서 • 중요한 단어에 집중
	Feed-Forward	• 그 결과를 다듬는 과정 • 입력 벡터 → 더 큰 차원으로 확장 → ReLU → 축소 • 집중할 각 단어를 벡터 공간에서 잘 반영될 수 있도록 처리
States		• AE의 Latent Space처럼 요약 압축 파일 역할 • 단어별 요약 모음집 성격 • 입력 길에 따라 유동적임 (AE는 고정 크기)
Decoder		• 문장을 만들어내는 부분 • 정보를 활용하여 출력 **문장 생성**
	Masked Multi-Headed	• 문장을 만들어낼 때 앞부분만 보고 다음 단어 예측
	Multi-Headed Cross-Attention	• 인코더가 뽑아놓은 문장 정보(Key, Value : 단서, 정보)와 현재 상태를 연결해서 참고 • 현재 문장만 보면 안되기 때문에 인코더 정보 참고 (요약본을 보고 힌트를 얻는 과정)
	Feed-Forward	• 최종적으로 가공 • 생성 문장을 다듬는 과정

Source : https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)

3. Transformer Architecture



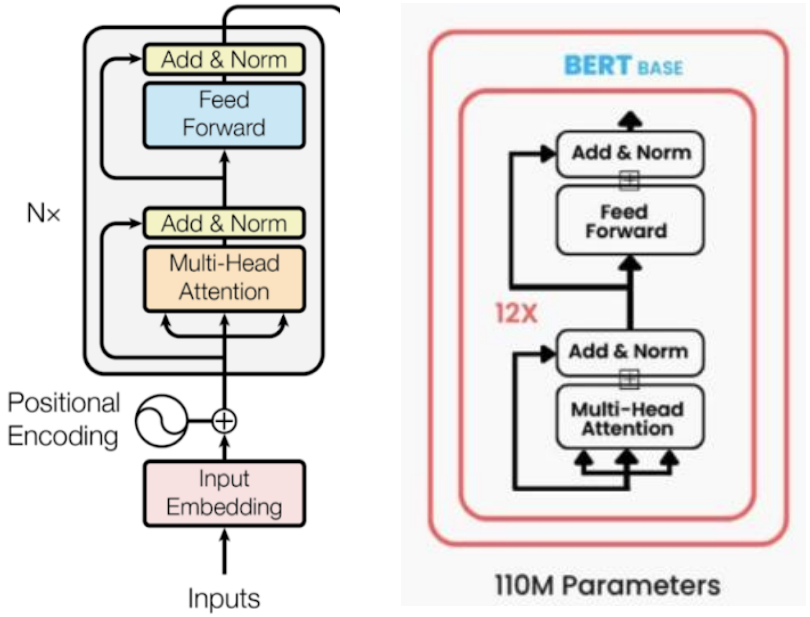
※ Positional Encoding
- 아키텍처 : RNN/LSTM처럼 순차적으로 처리하지 않고, 모든 단어를 동시에(병렬로) 처리
- 상황 : 단어가 숫자로 바뀌어 들어가지만, 순서를 알 수 없음
- 해결 : 각 단어에 위치정보(인코딩 벡터)를 더해서 문장의 순서 정보를 함께 반영
- (e.g.) "고양이가 창문에 앉았다" → [고양이+위치1, 창문에+위치2, 앉았다+위치3]

Source : Attention Is All You Need (2017), https://arxiv.org/pdf/1706.03762

4. BERT

BERT, Bidirectional Encoder Representations from Transformers

- Transformer의 Encoder 부분을 기반으로 양방향 학습
- 입력 문장의 앞뒤 문맥을 모두 고려하여 단어를 이해하는데 중점



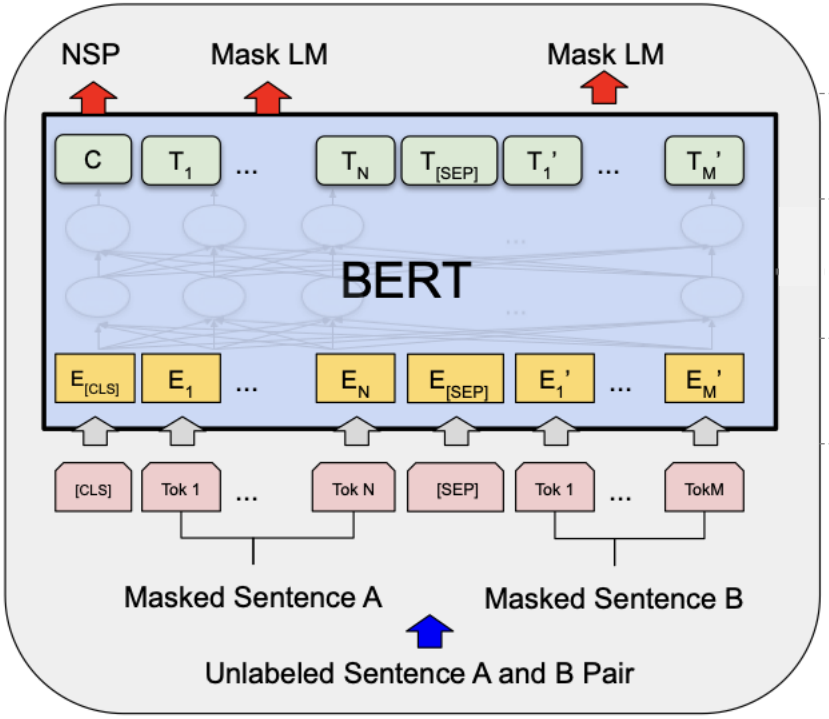
왜 BERT는 Transformer의 Encoder를 사용하는가?

- 인코더 역할
  - 인코더는 텍스트를 입력받아 의미를 분석하고 이해
  - 문장의 각 단어가 어떻게 서로 연결되어 있고, 어떻게 해석되어야 하는지 학습
  - BERT는 문장에서 단어가 어떻게 관련되는지를 깊이 이해하는데 중점
- 인코더 기반의 BERT가 주는 효과
  - 양방향 문맥 이해, Bidirectional Context Understanding
    : 문장의 앞뒤 모든 단어를 고려하여 문맥 파악
  - Masked Language Model
    : BERT는 문장의 일부 단어를 마스킹하고, 그 단어를 맞추는 방식으로 학습
- BERT가 Transformer의 디코더를 사용하지 않는 이유
  - 디코더는 학습한 내용을 토대로 텍스트 생성에 사용
  - BERT는 생성이 아니라 문장 이해에 중점, 텍스트 생성 과정이 포함되지 않아 디코더 기능이 불필요

NLP 분야에서 표준 모델로 자리매김
(문맥 이해의 혁신적 접근과 활용)

Source : BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding (2018), https://arxiv.org/pdf/1810.04805

5. BERT Architecture



문맥을 학습시키는 단계

- NSP, Next Sentence Prediction – 문장 수준 문맥
  - C를 입력으로 "B가 A의 실제 다음 문장인가"를 이진 분류
  - 토큰 수준 문맥을 넘어 문장 간 연결 관계를 [CLS]에 축적시키려는 목적
- Masked LM – 토큰 수준 문맥
  - 입력 토큰의 약 15%를 가린 뒤, 그 자리의 최종 벡터로 원래 단어를 예측
  - 정답 토큰이 입력에서 제거되어 자기 자신을 컨닝하는 문제가 생기지 않음 → 양방향 학습을 가능하게 한 핵심 장치

- T_N : N번째 토큰의 최종 은닉 벡터 = 문맥이 반영된 해당 단어의 의미
- C : [CLS] 최종 은닉 벡터 = 시퀀스 전체 요약 표현 → 문장 단위 작업(분류, 문장쌍 관계 판단)

- 각 층의 Self-Attention에서 모든 토큰이 모든 토큰을 동시에 참조 = 양방향 Self-Attention
- 층을 거칠수록 표현 갱신 : 표층적 어휘 정보 → 구문 → 의미/지시 관계 순으로 문맥 누적
- 같은 단어라도 주변 토큰에 따라 다른 벡터가 나옴 = Contextual Embedding

- 아직 문맥이 반영되지 않은 정적 입력(E)
- E벡터 = 토큰 임베딩 + 세그먼트 임베딩(A/B 구분) + 위치 임베딩

- 문장 두 개를 한 시퀀스로 묶어서 입력
- [CLS] : 모든 시퀀스의 첫 토큰. 그 자체로는 의미가 없는 빈 슬롯
- [SEP] : 문장을 구분하는 특수 토큰. 경계는 표시하되, Attention은 경계를 넘어 흐르게 함
  → 문장 간 문맥까지 학습 대상이 됨

Transformer Encoder 계승

정적 입력(E) → 양방향 Self-Attention 누적 → 문맥 반영 출력(T, C)

↑

Masked LM(단어 수준) + NSP(문장 수준)

양방향 사전학습을 가능하게 만든 목적함수

Source : BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding (2018), https://arxiv.org/pdf/1810.04805

[참고] Masked LM, 쉽게 이해하기

오른쪽 문맥을 쓰지 않으면 풀 수 없는 문제를, 정답 누출 없이 풀 수 있게 만든 목적함수

- 왜 오른쪽 문맥이 필요한가
  - 그는 [MASK]을 열고 우유를 꺼냈다
  - 그는 [MASK]을 열고 여권을 꺼냈다

  → 순방향 LM은 두 문장을 구분할 수 없음. 같은 확률 분포를 낼 수 밖에 없음

  (참고) 한국어는 서술어가 뒤에 있어, 오른쪽 문맥 의존도가 더 큼

- 왜 "가리는" 절차가 필요한가

양방향 + 일반 LM

- 그는 냉장고를 열고 우유를 꺼냈다

  냉장고를 예측하는데, 입력에 냉장고가 그대로 보임

  → 손실은 0에 수렴. 학습되는 것이 없음

Masked LM

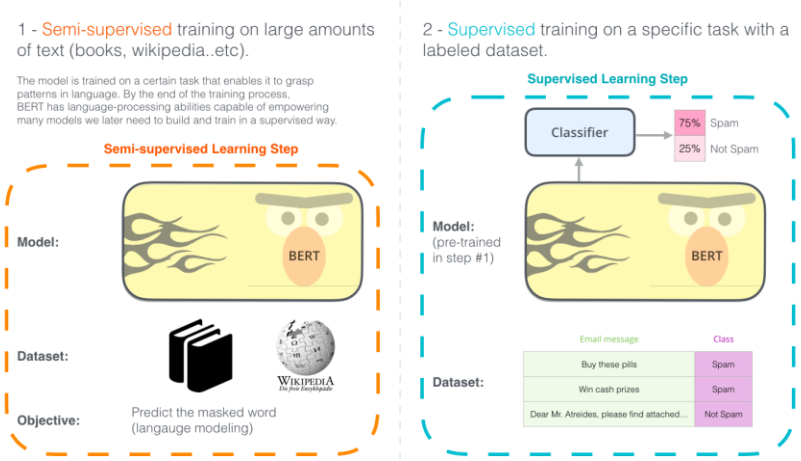
- 그는 [MASK]를 열고 우유를 꺼냈다

  정답이 입력에서 제거됨

  → 주변 문맥으로만 복원

6. BERT 주요 특징

- Pre-training Model : BERT는 대량의 데이터로 언어의 규칙과 의미를 학습한 사전 모델
- Finetuning : 사전 학습된 BERT를 특정한 작업에 맞게 추가 학습
- Transfer Learning : BERT의 학습된 언어 지식을 바탕으로 새로운 작업에 적용. 필요 시 Finetuning 진행



- **Pre-training:**
  기본적인 언어 패턴을 학습하는 단계로 많은 데이터를 사용하여 일반적인 언어의 규칙과 의미 학습

- **Fine-Tuning :**
  사전 학습된 모델을 특정 작업(질문에 답하기, 문장 분류 등)에 맞게 아키텍처를 변형하거나 추가 학습 데이터로 학습
  → 가중치 업데이트 통한 모델 최적화

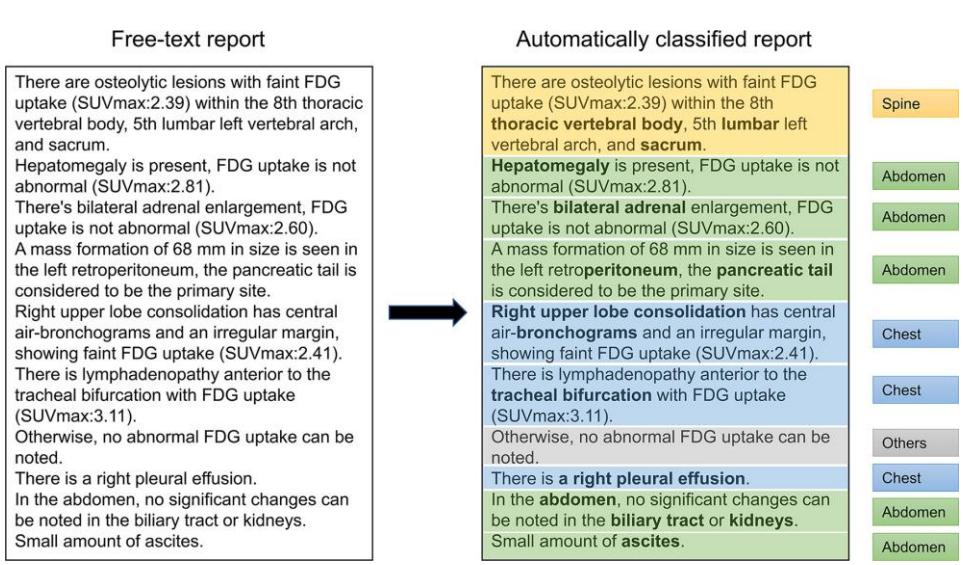
※ Pretraining Details
- Data : Wikipedia, BookCorpus
- 64 TPU chips for 4 days
- BERT-base : 12 Layer, 768 Hidden, 12 Head
- BERT-large : 24 Layer, 1024 Hidden, 16 Head

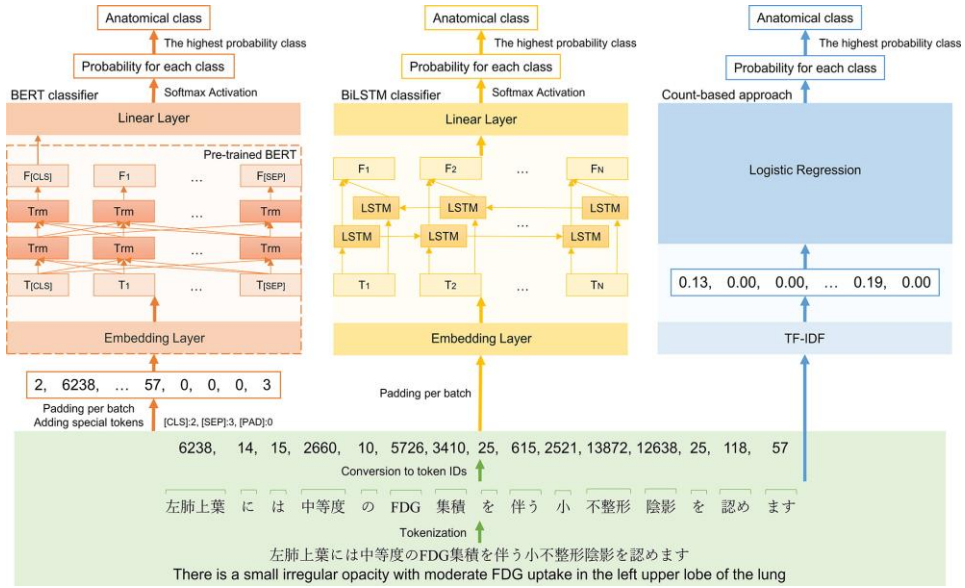
Source : BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding (2018), https://arxiv.org/pdf/1810.04805

[참고] BERT's Transfer Learning

Transfer Learning of Radiology Text Classification

- BERT 기반 전이학습 통해 방사선 보고서의 문장을 해부학 클래스별로 분류
- 구조화되지 않은 텍스트, 매우 적은 해부학 카테고리에서도 의미있는 성능 확인





Methods:

- 6,272 sentences from 900 reports were manually annotated by body part
- BERT-based classification was compared with that of two baseline models: BiLSTM and count-based method

Results:

- The BERT-based approach achieved the highest macro-averaged AUPRC(0.88) and AUC(0.97).
- The BERT-based approach outperformed baselines for most anatomic classes, even for minority classes with few labeled training data.

Source : BERT-based Transfer Learning in Sentence-level Anatomic Classification of Free-Text Radiology Reports (2023), https://pubs.rsna.org/doi/full/10.1148/ryai.220097

5. 규모와 비용의 시대

더 크게, 더 효율적으로

- Beyond Transformer
- MoE – 조건부 연산으로 푸는 규모 확장
- Kimi Linear – KDA & MLA 하이브리드

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

1. Beyond Transformer

Transformer가 뛰어나지만 비용, 메모리 등의 구조적 한계로 이를 해결하기 위한 새로운 아키텍처 제안

NLP		
1985	RNN	• 시퀀스 데이터를 처리할 수 있는 첫번째 신경망 • 순차적 데이터 처리에 탁월한 성능 발휘
1997	LSTM Long Short-Term Moemry	• RNN의 기울기 소실 문제 해결 • 언어/음성과 같은 시퀀스에서 뛰어난 성능 발휘
2017	**Transformer**	• Self-Attention 도입, RNN/LSTM 대체 • 병렬 처리와 함께 뛰어난 성능 구현

Efficient-oriented Track

Attention 구조 자체를 대체하여 장문 처리와 계산 효율을 근본적으로 개선하는 전략

Linear Recurrent Models		
2021-11	S4	• RNN 재설계 가능 • Transformer 병목 일부 해결, 성능은 불안정
2023-06	HyenaDNA	• Long Conv + Gating으로 Attention 대체 가능성 탐색 • 성능, 처리속도 등 여전히 한계
2023-07	RetNet	• Multi-scale Retention Block 통해 Attention 대체 • Transformer와 유사한 성능, 안정성 유지
2023-12	Mamba	• 긴 문맥 대응하기 위해 SSM 도입 • 하드웨어 효율까지 고려한 설계
2024-06	DeltaNet	• Delta update rule로 긴 문맥 유지 및 학습 안정성 개선 • 긴 문맥 유지 + 안정적 가중치 흐름
2024-12	Gated Deltanet	• Delta update rule + Gating + 하드웨어 효율 • Mamba 대비 안정적 학습 및 긴 문맥 성능 확보
2025-11	**Kimi Linear**	• Hybrid (Kimi delta + full attetion) architecture • Transformer 대체가 가능한 수준까지 도달

Scaling-oriented Track

모델 파라미터를 늘리되 실제 계산량은 최소화 하는 확장 전략

Mixture of Experts (MoE)		
2017-01	Sparsely-Gated MoE Layer	• MoE 구조를 딥러닝 맥락에서 정립 • Sparse Gating 개념 도입
2020-06	GShard	• Conditional Computation + MoE • 분산 학습 프레임워크 확립
2021-01	Switch Transformer	• 기존 MoE 보다 단순한 라우팅 구조로 효율 개선 • 1조 파라미터급 모델 가능성 확인
2021-12	GLaM	• MoE 기반 대형 언어모델 설계 • 계산량, 에너지 효율 획기적 개선

※ LLM adapting MoE

- (2023-12) Mistral 8*7B : 오픈소스 MoE 상용화 시작점
- (2024-01) DeepSeekMoE : MoE 구조적 완성도 및 실적 적용 확산
- (2025-09) Qwen3-Next : Hybrid Attention + MoE

[Remind] Transformer

중요한 것에 집중하는 방식으로, 문장을 더 정확하고 빠르게 이해할 수 있음

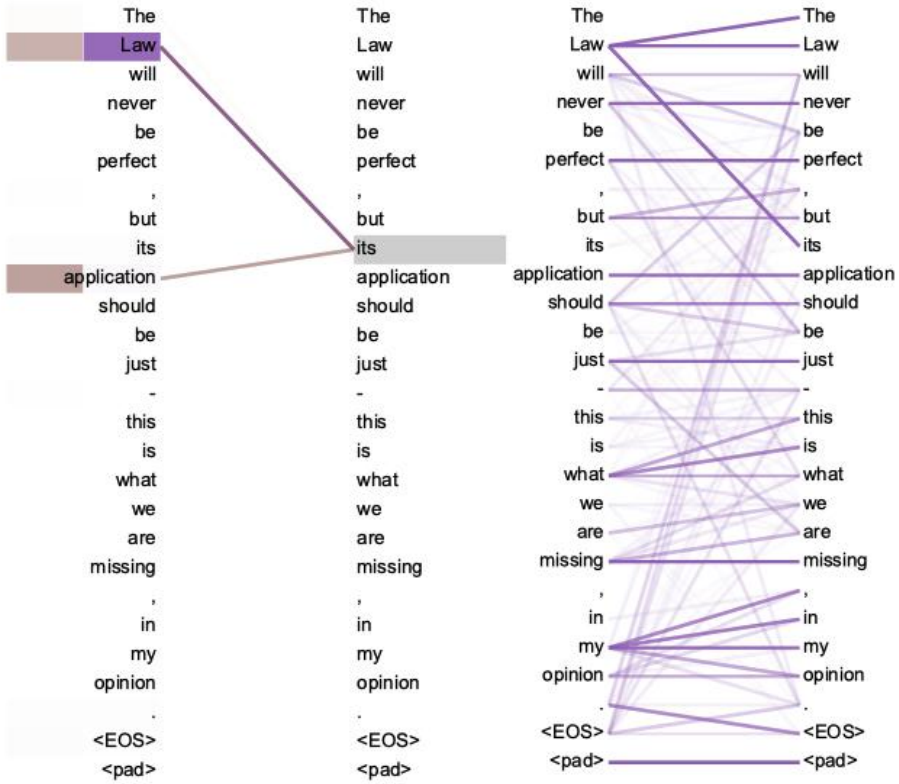
- 자연어 처리에서 RNN이나 LSTM을 대체한 BERT, GPT의 근본이 되는 기본 아키텍처
- Self-Attention 매커니즘을 도입하여 긴 문맥을 효율적으로 처리하여 NLP Task 성능을 크게 향상

Attention

- 입력 시퀀스(문장)의 각 요소(단어)가 **다른 요소들과 얼마나 중요한지, 어느 부분에 집중할지 결정**하는 매커니즘
- 기계 번역 시 특정 단어를 번역할 때, 문장의 다른 단어들과 얼마나 관련이 있는지를 고려하여 더 정확하게 번역

Self-Attention

- 입력된 문장 내에서 **각 단어가 다른 모든 단어와 관계를 학습**
- 한 단어가 문장 내 다른 단어들과 얼마나 관련 있는지를 계산하여 문맥을 더 잘 이해하도록
- RNN, LSTM이 순차적으로 데이터를 처리하지만, Self-Attention은 문장 내의 모든 단어를 **동시**에 보고 처리 (동시 = 병렬처리)



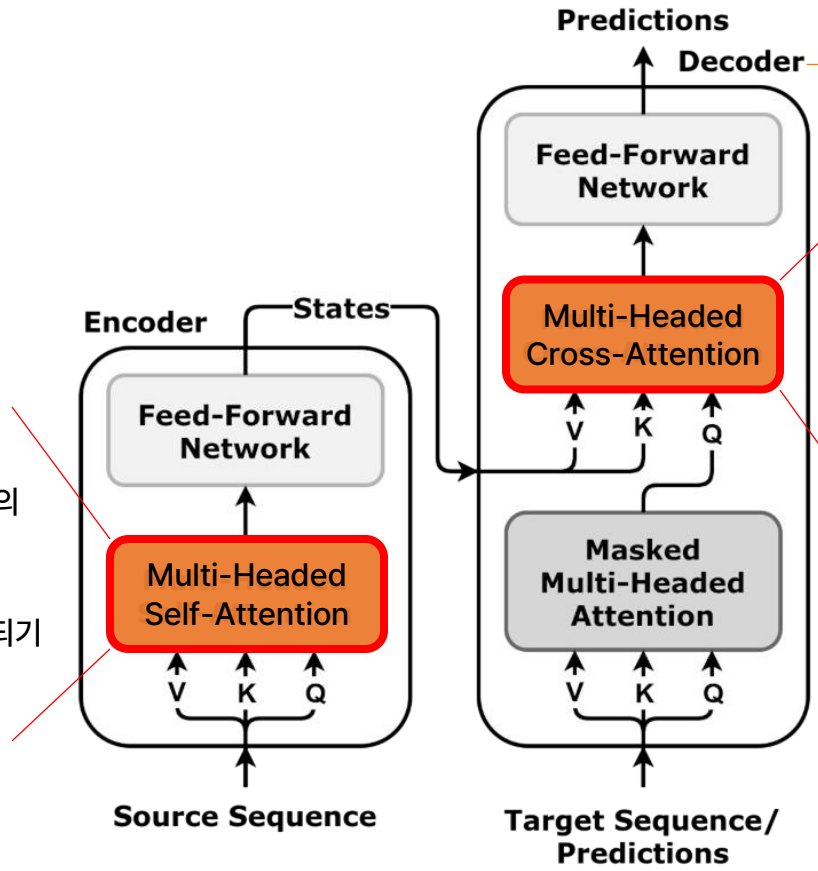
Source : *Attention Is All You Need (2017), https://arxiv.org/pdf/1706.03762*

Key Point

2. Transformer가 풀지 못한 한계

토큰이 늘어날수록 계산량 뿐 아니라 메모리 사용량까지 함께 증가하는 구조

- 각 토큰이 모든 다른 토큰과 상호작용
- 입력 길이가 n개 일 때 각 토큰의 n개의 key-value(KV) 조합을 참조해야 함 → **계산량**이 대략 $O(n^2)$
- KV cache는 입력 길이에 따라 누적되기 때문에 긴 문장을 처리할수록 **메모리 사용량**이 선형적으로 증가



매번 새로운 단어를 하나 말할 때마다, 지금까지 했던 모든 말을 다시 참고하면서 다음 말을 정한다

- Decoding에서 KV cache 누적이 점차 커짐
- 추론(inference) 단계에서 실시간 응답이나 긴 문맥 처리 능력이 떨어짐 → **병목 현상**
- Long-context model이나 Agent 설계 시 Transformer 기반 LLM은 구조적으로 효율성 한계

[참고] Transformer 계산량, 쉽게 이해하기

- 회의에서 모두가 대화해야 하는 상황

  n명이 모인 회의에서 각자가 발언하기 전에 "다른 모든 사람의 의견"을 참조해야 합니다.
  이 때 참고해야 할 수는...
  - 사람이 2명이면 : 참고 관계 4가지 = 2 * 2
  - 사람이 10명이면 : 참고 관계 100가지 = 10 * 10
  - 사람이 100명이면 : 참고 관계 10,000가지 = 100 * 100

- Self-Attention : 문장 안의 토큰 하나하나가 "나머지 모든 토큰과 얼마나 관련 있는지"를 모두 비교해야 함
  토큰 수가 n개이면, 비교해야 할 쌍이 $n*n = n^2$

  (참고)
  - KV cache (메모리) : 토큰 하나가 들어올 때마다 그 토큰의 정보(K, V) 한 줄을 노트에 적어두는 것과 같음.
    토큰이 늘어난 만큼만 노트가 길어지게 되어 선형(n) 증가
  - 계산량 : 새로운 토큰이 하나 들어올 때마다, 지금까지 노트에 적힌 모든 줄을 다시 훑어봐야 함. 계산량은 $O(n^2)$ 형태로 늘어남

3. 왜 RNN을 다시 보는가

Key Point

RNN의 고정 비용이 다시 매력적 – 단, 정확도를 희생한 대가임을 알아야 한다

TRANSFORMER → *그럼 과거를 전부 보관하지 않으면 되지 않나?* → **RNN**

▪ **정확도를 위해 비용을 부담한다**
▪ Transformer의 비용은 버그가 아니라 설계 철학의 직접적 결과
▪ 느리고 메모리를 많이 쓰는 이유는 아무 정보도 버리지 않기 때문

과거 처리 방식	• 전부 원본 KV로 보존 (무압축)
토큰당 생성 비용	• O(L) – 매번 전체 재참조
메모리	• 길이에 비례해 누적
조회 정확도 (recall)	• 최대 – 원본 그대로 꺼냄
치르는 대가	• 비용, 메모리를 전부 떠안음

▪ **비용을 줄이기 위해 정확도를 일부 버린다**
▪ RNN의 한계는 압축이라는 설계 선택의 직접적 결과
▪ 빠르고 메모리가 고정인 이유는 과거를 단일 상태로 압축했기 때문

과거 처리 방식	• 단일 고정 상태로 압축
토큰당 생성 비용	• O(1) – 고정 상태만 참조
메모리	• 고정 (길이 무관)
조회 정확도 (recall)	• 손상 – 덮어써서 복원 불가
치르는 대가	• 정보 손실을 그대로 떠안음

정확도 (recall↑)
Transformer

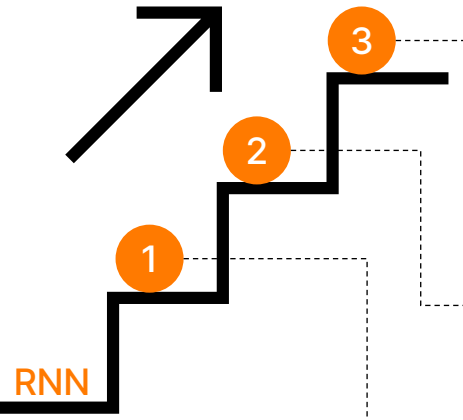
TRADE-OFF

효율 (비용↓)
RNN

RNN으로 돌아가자는 것이 아님
RNN의 효율이라는 장점만 떼어내, 치렀던 단점 없이 가져오기 위한 노력 필요

[참고] 회의록 작성으로 쉽게 이해하기

POV : 긴 회의를 듣고 회의록 한 부를 완성하는 일



RNN 효율 – 회의록 기존 부분 유지

Attention 정보 반영 – 새로운 정보를 제대로 반영

- **바뀐 것만 반영 (DeltaNet)**
- 새로운 발언이 나올 때 마다 회의록을 처음부터 다시 작성하지 않음 Transformer 방식을 따르지 않음
- 기존 회의록은 유지하고, 달라진 부분만 내용 업데이트

State 유지

새로운 입력으로 생긴 변화분(delta)만 State에 갱신

- **나누어서 빠르게 (Chunk-wise)**
- 2시간 회의록을 혼자 담당하면 느림
- 회의를 10분 단위(chunk)로 쪼개어 여러명이 동시에 맡아, 각자 정리 → 마지막에 이어 붙임
  (chunk 내부는 순차, chunk 간은 병렬) 순차 병렬

- **순차의 벽을 깨다 (RNN → SSM)**
- RNN : 1번 발언 듣고 회의록 작성. 그 다음에야 2번 발언으로 넘어갈 수 있음 → 회의가 길수록 회의록 작성이 오래 걸림 순차
- SSM : 회의록 전체 녹취록 확보된 상태. 발언 N개를 순차적으로 보지 않고, N개 발언을 한꺼번에 묶어서 회의록 작성 병렬

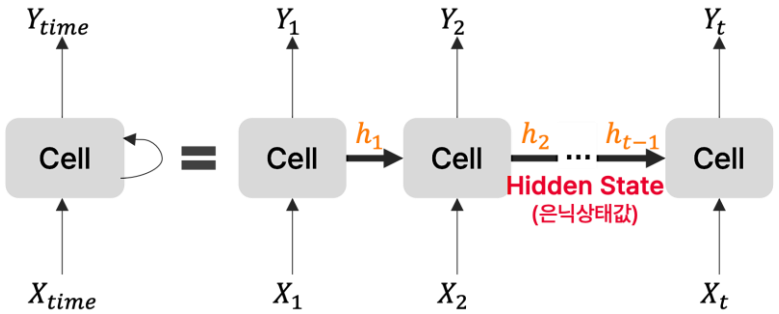
4. Linear Recurrent Models

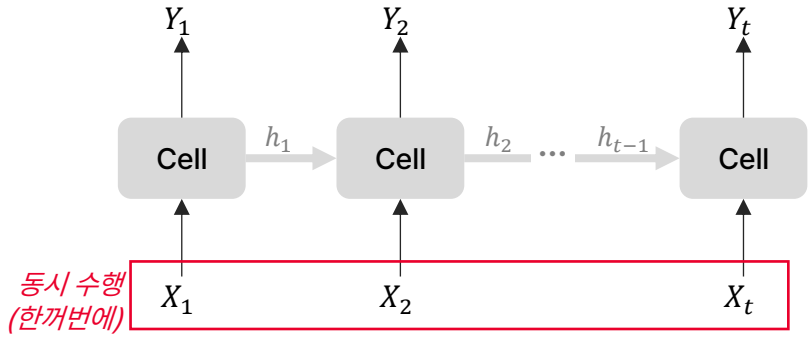
계산 비용을 선형시간(O(n))으로 만들고, 병렬 학습이 가능하도록 발전해온 현재 RNN 계열

State-Space Model (상태공간모델)

(0) RNN-Style 유지 : 순차적으로 state update

(1) SSM : 병렬 처리가 가능하도록 State update 구조 재설계





하나씩 차례대로 ← Hidden State : 계산 순서 의존

회의록 = (((시작 + 발언1) + 발언2) + 발언3)

학습 관점

과거를 미리 펼쳐 한꺼번에 계산 → 동시에, 나란히

위치3 회의록 = 시작 + 발언1 + 발언2 + 발언3
참조 참조

RNN의 Hidden State :
- 순서 : 1번 → 2번 → … → N번을 반드시 차례대로
- 결과 : 길어질수록 느리고, 오래된 정보가 덮어써져 희미해짐

SSM의 Hidden State :
- 순서 : 과거 의존은 유지하되 전체를 펼쳐 한꺼번에 계산 (병렬)
- 결과 : 길어져도 빠르게(병렬 학습), 비용은 선형

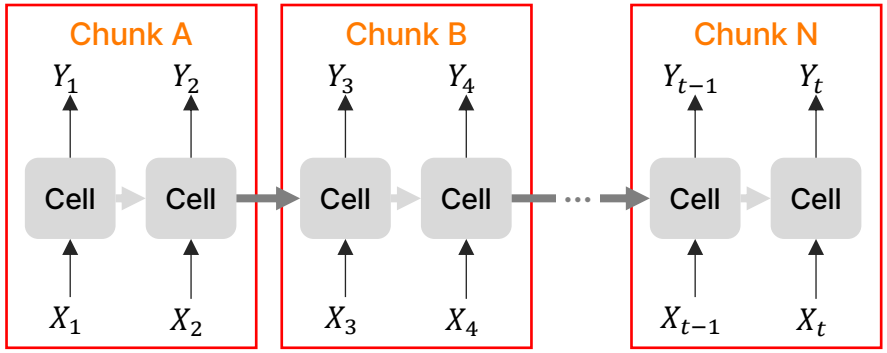
둘 다 과거 정보를 고정 크기의 Hidden State에 압축하여 다음 스텝으로 전달
바뀐 것은 그것을 갱신하는 방식 : 순차 → 선형 병렬

4. Linear Recurrent Models

계산 비용을 선형시간(O(n))으로 만들고, 병렬 학습이 가능하도록 발전해온 현재 RNN 계열

대표 예 : RetNet, Retentive Network

(2) Chunk-wise 병렬화 & GPU-friendly 구조



학습 관점

병렬 Chunk A = 발언1 + 발언2 + 발언3
Chunk B = 발언4 + 발언5 + 발언6

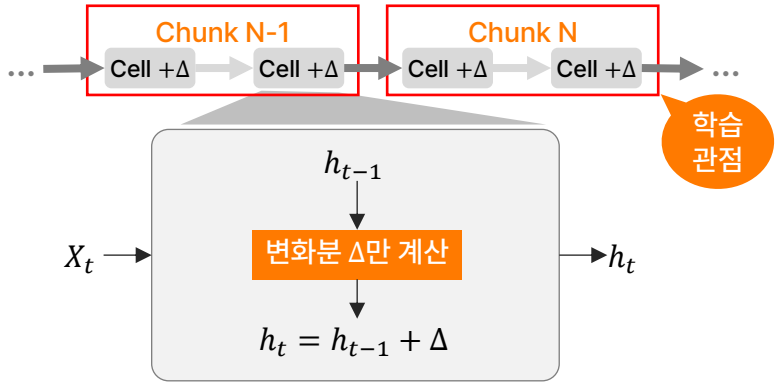
회의록 = 시작 + A → B 순차

추론 관점

← 길어져도 GPU가 구간별로 나눠 빠르게 처리

DeltaNet

(3) Linear-time Attention



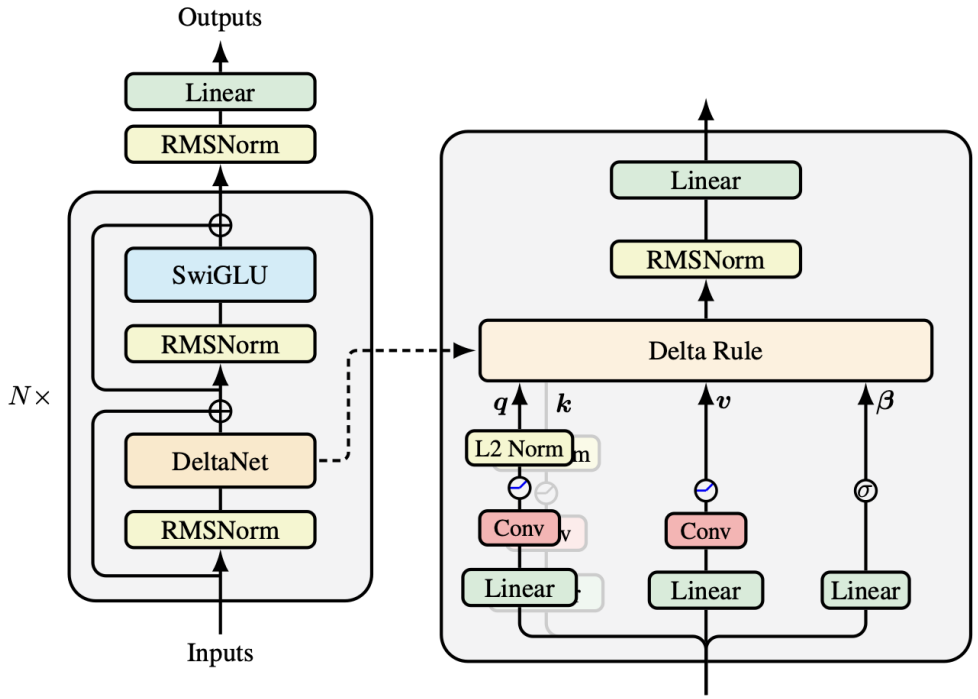
Delta :
새 입력값 – 기존 기억의 예측값
→ 새로운 입력을 만날 때마다 오차만큼 State Update

회의록0 = 빈 회의록(시작)
회의록1 = 회의록0 + Δ_1
회의록2 = 회의록1 + Δ_2
...
회의록6 = 회의록5 + Δ_6

추론 관점

[참고] DeltaNet

- Delta Rule : 노트를 매번 다시 쓰는 것이 아니라 필요한 부분만 수정
- 문장이 길어져도 빠르고, GPU 친화적, 선형 시간으로 문맥 정보 반영
- RNN처럼 State 유지 + Attention처럼 정보 반영



이전 기억에 변화분만 더하는 연산

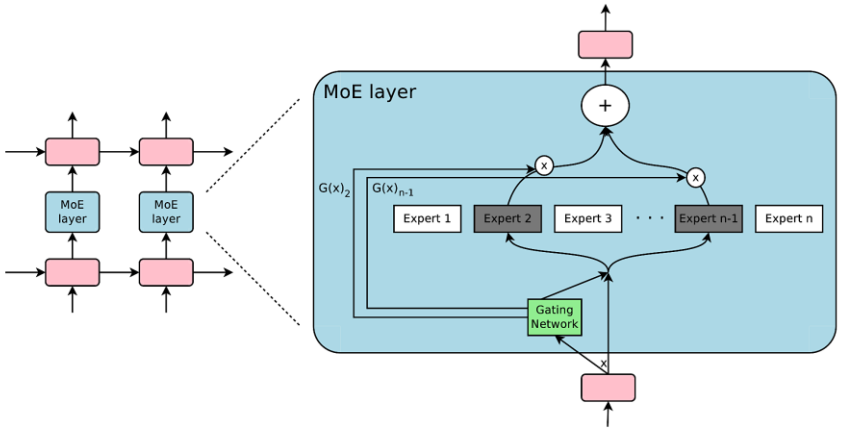
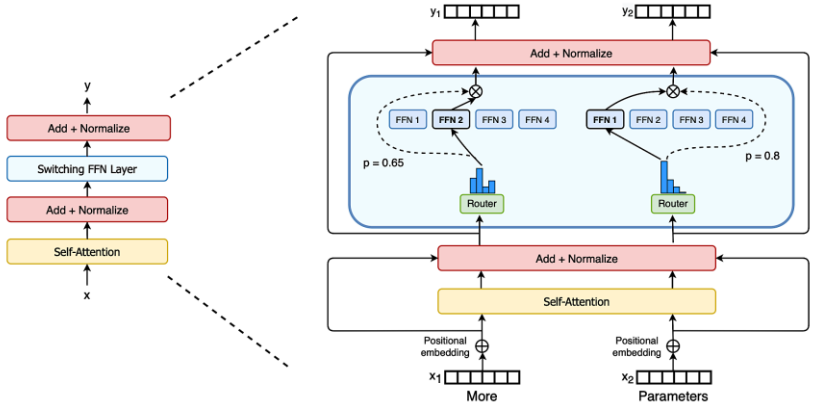
$h_t = h_{t-1} + \Delta$

변화량을 뽑아내는 역할

Source : https://arxiv.org/pdf/2406.06484

5. MoE, Mixture of Experts

모든 계산을 처리하지 않고 필요한 전문가만 선택해 확장성과 효율을 얻는 구조

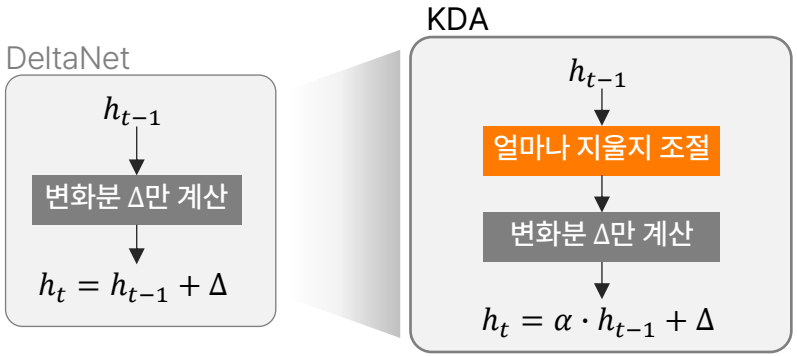
구분	Sparsely-Gated MoE Layer	Switch Transformer
목적	• MoE 개념 최초 제안 – 여러 전문가 중 일부만 사용하자	• 실전에 사용하기 좋게 단순화한 모델 – 더 빨리, 더 크게 확장하자
비유	• 학생을 보고 2명의 선생님 배정, 반반 나누어서 가르침	• 학생마다 딱 1명의 담당 선생님만 배정, 고민 없이 빠르게 배치
구조	 • 전문가 선택 방식 : 토큰마다 **Top-2 Experts** 선택 • Expert간 교차 통신 고려 (특정 토큰을 적절한 전문가에게 보내는데 드는 비용)	 • **Top-1 Expert 선택 (단일 라우팅)** • 통신 부담 극감 → 훈련 효율↑
장점	• 전체 모델이 아닌 일부 전문가만 사용하므로 계산 자원 절감 가능 • 전문가들이 서로 다른 패턴을 학습하면서 표현력 향상	• 통신 비용이 대폭 줄어 1조 파라미터 모델까지 확장 가능 • 단순한 라우팅 구조로 학습 안정성 확보, 구현 및 운영 용이
단점	• 토큰마다 연산량이 여전히 큼 • Expert간 교차 라우팅(통신 비용)이 발생하여 대규모 시스템에서는 비효율	• 부적절한 라우팅이 발생하면 특정 전문가에 부하 (imbalanced routing) • 라우터 품질이 모델 전체 성능에 영향 → 게이트 학습이 중요

Source : https://arxiv.org/pdf/1701.06538, https://arxiv.org/pdf/2101.03961

6. Kimi Linear

KDA로 속도와 메모리를 잡고, MLA로 전체 맥락을 보강

KDA, Kimi Delta Attention



회의록0 = 빈 회의록(시작)
회의록1 = 회의록0 + Δ_1
회의록2 = 회의록1 + Δ_2
...
회의록$_t$ = 회의록$_{t-1}$ + Δ_t

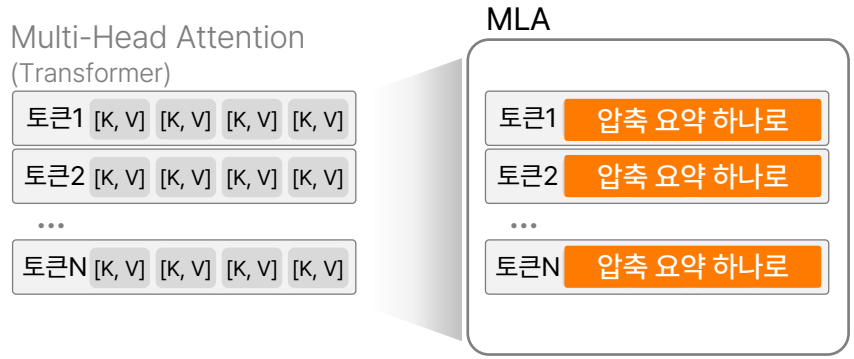
과거를 모두 남기고 변화분을 더함

회의록$_t$ = **α** ·회의록$_{t-1}$ + Δ_t

일부를 덜고 변화분을 더함

무한정 길어지지 않게,
낡은 건 흐려지고 중요한 것은 진하게,
그리고 빠르게

MLA, Multi-head Latent Attention



발언1 보관 : 발언1 원본 전체
발언2 보관 : 발언2 원본 전체
발언3 보관 : 발언3 원본 전체
...

발언마다 원본을 통째로 보관
→ KV cache 부담

발언1 보관 : 발언1 요약본 1개
발언2 보관 : 발언2 요약본 1개
발언3 보관 : 발언3 요약본 1개
...

Latent 압축

핵심만 반영된 **요약본만** 각각 보관
다시 확인할 때에는 **원본 복원/참조**

메모리 공간↓ Trade-Off 복원 부담↑

6. Kimi Linear

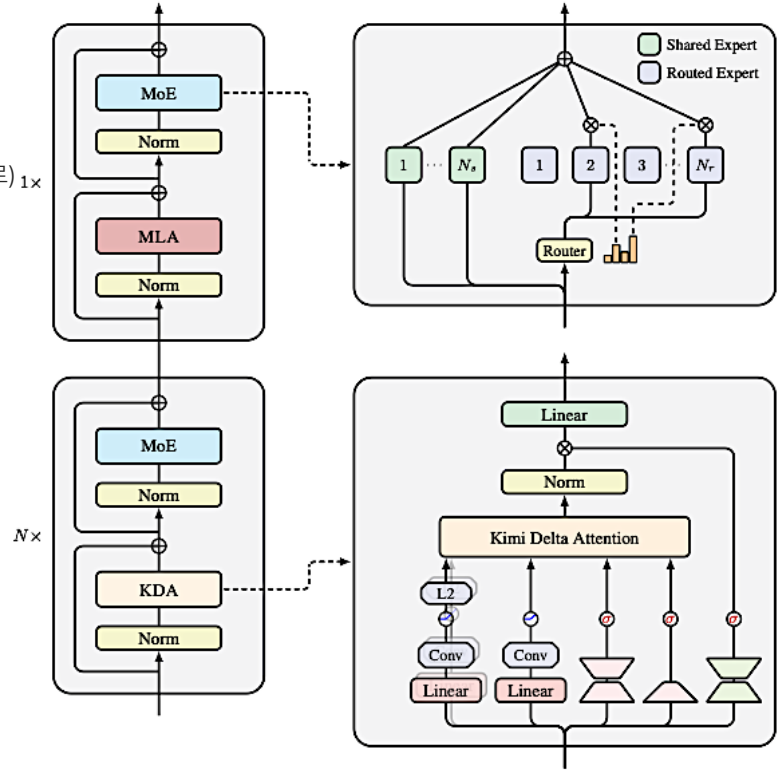
KDA로 속도와 메모리, MLA로 맥락, MoE로 확장을 담당하는 하이브리드 아키텍처

- 순수 선형도, 순수 어텐션도 아닌 하이브리드 구조 – 빠른 방식(KDA)를 주력으로 쓰고, 전체를 보는 방식(MLA)을 섞어 장점 결합
- 긴 문맥에서도 속도와 메모리는 선형 수준으로 가볍게, 품질은 Full Attention급으로 유지

MoE, 확장 담당 :
Feed-Forward 대신 MoE로 대체.
각 토큰을 독립적으로 가공하여 전문가에게 전달 (파라미터 증가, 연산은 그대로)

MLA, 맥락 담당 :
느리지만 전체 관계를 놓치지 않으므로,
KDA만으로 약해지는 장거리 맥락 보강

KDA, 효율 담당 :
전체 토큰을 다시 보지 않으므로 빠르고
메모리 적음



- Shared : 공통 기반 처리, 항상 작동
- Routed : 토큰별 전문 처리

→ MoE = Shared(고정) + Routed(선택)

KDA : MLA = 3 : 1 → KDA-KDA-KDA-MLA
속도, 메모리, 정확도를 실험을 확인한 비율

- Norm/Linear : State에 반영
- KDA : 얼마나 남기고 얼만큼 버릴지 최종 결정
- Conv/L2 : 불필요한 세부 정보 제거, 핵심만 남김
- Linear : 새로운 정보가 들어왔을 때 얼마나 변화시킬지 계산

Source : https://arxiv.org/pdf/2510.26692

[참고] Kimi Linear

MIT TR · 인공지능

트랜스포머 이후 가장 중요한 논문이 나왔다

중국 스타트업 문샷 AI는 문맥이 길어질수록 기하급수적으로 느려지는 트랜스포머의 한계를 새로운 방식으로 해결하려 한다.

박수연 수석기자 2025년 11월 18일

- **아키텍처 평가 :**
  - GPT-o1, Deepseek R1과 같이 추론 능력을 끌어올리기 위해 강화학습 통한 모델 추가학습 흐름
  - Kimi Linear는 강화학습 방식 보다 더 빠르게, 더 높은 정확도 제시
  - AI Agent가 수십만 토큰에 이르는 복잡한 작업을 실시간에 가깝게 처리할 수 있는 가능성 확인
  - 모델 패러다임 변화 가능성 : 큰 모델이 상위 전략 수립, 하위 여러 모델이 세부 작업 처리 (likely Supervisor)

- **시사점 :**
  - 가장 큰 모델이 아니라, 가장 효율적인 모델이 승부를 가를 수 있다
  - 현존하는 LLM은 초거대 단일 모델이나, 문샷AI는 정교하게 설계된 **하이브리드 구조와 연산 효율성을 무기**로 전혀 다른 방향으로 경쟁
  - AI Agent가 복잡한 작업을 수행하고, 수십만 토큰이 실시간으로 오가며 협력하는 시대가 오고 있다

Source : https://www.technologyreview.kr/트랜스포머-이후-가장-중요한-논문이-나왔다/

[참고] Kimi Linear

중국AI혁명 중국 · 문샷 AI · 키미 K3

중국 오픈 모델 '키미 K3' 프런티어 AI 진입했다… 토큰 경제 대전환

박원익
2026.07.16 14:57 PDT

토론 AI 0개 | 공유하기 | 북마크 하기

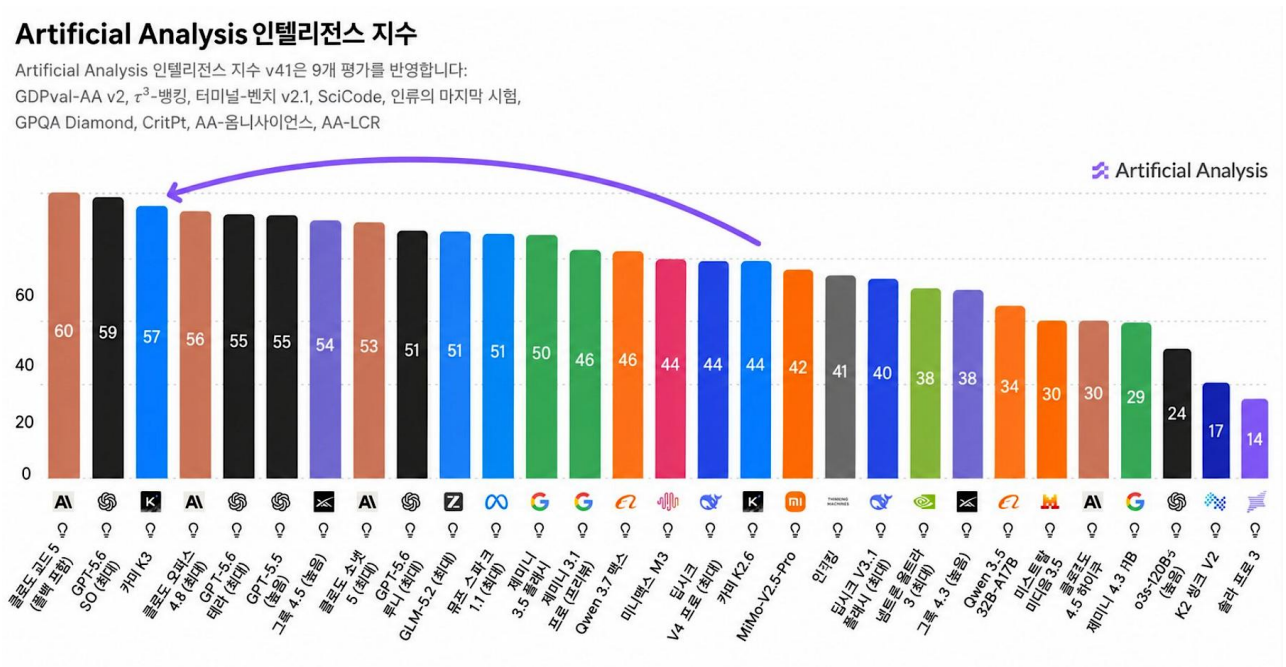
인공지능

China's AI models have Trump's AI world at war with itself

중국 무료 AI 모델 '키미' 충격…트럼프 정부, 대응 방향 놓고 분열

중국 스타트업 문샷AI가 공개한 무료 오픈소스 모델 '키미'가 미국 주요 AI 모델에 버금가는 성능을 보이면서, 트럼프 행정부 내부에서 오픈소스 개방과 정부 규제 강화를 둘러싼 논쟁이 격화되고 있다.

James O'Donnell 2026년 7월 21일



Source : https://www.technologyreview.kr/중국-무료-ai-모델-키미-충격트럼프-정부-대응-방향/, https://themiilk.com/articles/ac462a71e

Part-C. DL Architecture (2017~ Current)

SUMMARY

SUMMARY

Architecture Expansion

CNN-based Architecture

COMPUTER VISION		
YEAR	Architecture	Contribution
1990	CNN	• 이미지 처리에 강점을 가진 신경망 • 합성곱 필터로 이미지 특징 자동 추출하는 방법론 제시
1998	LeNet-5	• 초기 CNN 중 하나 • MNIST 위해 설계, 현대 CNN의 기초 다짐
2012	**AlexNet**	• GPU 기반 ReLU, Dropout 사용한 Large CNNs • ImageNet에서 혁신적인 성과 & 딥러닝 대중화
2014	VGGNet	• 작은 필터(3*3)로 아키텍처 단순화 • 객세인식 성능 향상
2014	GoogleNet (Inception)	• Inception 모듈 도입 • 계산 자원 효율화, 높은 정확도 유지
2015	**ResNet**	• Residual Learning으로 Deep Network(최대 152층) • 기울기 소실 문제 해결
2017	DenseNet	• 각 층을 모든 다른 층과 연결 • 파라미터 수를 줄이면서 정확도 향상
2017	YOLO You Only Look Once	• 실시간 객체 탐지 • 이미지 내 객체를 빠르고 효율적으로 탐지
2018	EfficientNet	• 네트워크 깊이, 폭, 해상도를 균형있게 조정 • 적은 파라미터로 성능 극대화
2021	**ViT** Vision Transformer	• Transformer를 이미지 분류에 적용한 첫 사례로 전통적은 CNN 성능 능가 (2~5%) • Vision Task에서도 효과적일 수 있음을 입증

TRANSFORMERs

NLP		
YEAR	Architecture	Contribution
1985	RNN	• 시퀀스 데이터를 처리할 수 있는 첫번째 신경망 • 순차적 데이터 처리에 탁월한 성능 발휘
1997	LSTM Long Short-Term Moemry	• RNN의 기울기 소실 문제 해결 • 언어/음성과 같은 시퀀스에서 뛰어난 성능 발휘
2014	Word2Vec	• 단어 임베딩 혁신 • 단어의 의미지 관계를 연속적인 벡터로 학습
2015	Seq2Seq	• 인코더-디코더 확장 • 기계번역 및 순차 데이터 작업에서 성능 향상
2017	**Transformer**	• Self-Attention 도입, RNN/LSTM 대체 • 병렬 처리와 함께 뛰어난 성능 구현
2018	**BERT**	• Transformer 디코더 구조 확장 (Bidirectional) • Pre-training, Transfer Learning
2019	GPT-2	• Transformer 디코더 구조 확장 (Unidirectional) • 문맥에 맞는 텍스트 생성 가능
2020	GPT-3	• 1750억개 파라미터 가진 대형 Transformer • 미세조정 없이 다양한 NLP Task 수행
2021	DALL-E	• 텍스트 설명으로 이미지 생성 • Cross-Modal, Multi-Modal
2021	CLIP	• 이미지와 텍스트 데이터 쌍을 학습 • 비전 작업에서 Zero Shot 학습 가능하게 함
2023	**GPT-4**	• GPT-3 후속버전 • 향상된 성능으로 깊이 있는 문맥 이해와 자연스러운 텍스트 생성 가능

SUMMARY

Beyond Transformer

Transformer가 뛰어나지만 비용, 메모리 등의 구조적 한계로 이를 해결하기 위한 새로운 아키텍처 등장

NLP		
1985	RNN	• 시퀀스 데이터를 처리할 수 있는 첫번째 신경망 • 순차적 데이터 처리에 탁월한 성능 발휘
1997	LSTM Long Short-Term Moemry	• RNN의 기울기 소실 문제 해결 • 언어/음성과 같은 시퀀스에서 뛰어난 성능 발휘
2017	**Transformer**	• Self-Attention 도입, RNN/LSTM 대체 • 병렬 처리와 함께 뛰어난 성능 구현

Efficient-oriented Track

Attention 구조 자체를 대체하여 장문 처리와 계산 효율을 근본적으로 개선하는 전략

Linear Recurrent Models		
2021-11	S4	• RNN 재설계 가능 • Transformer 병목 일부 해결, 성능은 불안정
2023-06	HyenaDNA	• Long Conv + Gating으로 Attention 대체 가능성 탐색 • 성능, 처리속도 등 여전히 한계
2023-07	RetNet	• Multi-scale Retention Block 통해 Attention 대체 • Transformer와 유사한 성능, 안정성 유지
2023-12	Mamba	• 긴 문맥 대응하기 위해 SSM 도입 • 하드웨어 효율까지 고려한 설계
2024-06	DeltaNet	• Delta update rule로 긴 문맥 유지 및 학습 안정성 개선 • 긴 문맥 유지 + 안정적 가중치 흐름
2024-12	Gated Deltanet	• Delta update rule + Gating + 하드웨어 효율 • Mamba 대비 안정적 학습 및 긴 문맥 성능 확보
2025-11	**Kimi Linear**	• Hybrid (Kimi delta + full attetion) architecture • Transformer 대체가 가능한 수준까지 도달

Scaling-oriented Track

모델 파라미터를 늘리되 실제 계산량은 최소화 하는 확장 전략

Mixture of Experts (MoE)		
2017-01	Sparsely-Gated MoE Layer	• MoE 구조를 딥러닝 맥락에서 정립 • Sparse Gating 개념 도입
2020-06	GShard	• Conditional Computation + MoE • 분산 학습 프레임워크 확립
2021-01	Switch Transformer	• 기존 MoE 보다 단순한 라우팅 구조로 효율 개선 • 1조 파라미터급 모델 가능성 확인
2021-12	GLaM	• MoE 기반 대형 언어모델 설계 • 계산량, 에너지 효율 획기적 개선

※ LLM adapting MoE

- (2023-12) Mistral 8*7B : 오픈소스 MoE 상용화 시작점
- (2024-01) DeepSeekMoE : MoE 구조적 완성도 및 실적 적용 확산
- (2025-09) Qwen3-Next : Hybrid Attention + MoE



수고하셨습니다.

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX